

```
// Tangent strength based on distance and orientation
const curvature = Math.min(Math.max(dist * 0.4, 30), 200);

const cx1 = p1.x + p1.nx * curvature;
const cy1 = p1.y + p1.ny * curvature;
const cx2 = p2.x + p2.nx * curvature;
const cy2 = p2.y + p2.ny * curvature;

return `M ${p1.x},${p1.y} C ${cx1},${cy1} ${cx2},${cy2} ${p2.x},${p2.y}`;
}

/**
 * 2. Orthogonal Step (Elbow) Path with Smooth Rounded Corners
 */
export function getOrthogonalPath(
  p1: AnchorPoint,
  p2: AnchorPoint
): string {
  const dx = p2.x - p1.x;
  const dy = p2.y - p1.y;
  const dist = Math.hypot(dx, dy);

  if (dist < 4) {
    return `M ${p1.x},${p1.y} L ${p2.x},${p2.y}`;
  }

  const r = Math.min(12, dist / 4); // fillet radius
  const margin = Math.min(24, dist * 0.2);

  const startOutX = p1.x + p1.nx * margin;
  const startOutY = p1.y + p1.ny * margin;
  const endOutX = p2.x + p2.nx * margin;
  const endOutY = p2.y + p2.ny * margin;

  const midX = (startOutX + endOutX) / 2;
  const midY = (startOutY + endOutY) / 2;

  // If mostly horizontal or vertical orientation
  if (Math.abs(p1.nx) > 0.5 && Math.abs(p2.nx) > 0.5) {
    return `M ${p1.x},${p1.y} L ${midX},${p1.y} L ${midX},${p2.y} L ${p2.x},${p2.y}`;
  } else if (Math.abs(p1.ny) > 0.5 && Math.abs(p2.ny) > 0.5) {
    return `M ${p1.x},${p1.y} L ${p1.x},${midY} L ${p2.x},${midY} L ${p2.x},${p2.y}`;
  } else {
    // Corner turn
    if (Math.abs(p1.nx) > 0.5) {
      return `M ${p1.x},${p1.y} L ${p2.x},${p1.y} L ${p2.x},${p2.y}`;
    } else {
      return `M ${p1.x},${p1.y} L ${p1.x},${p2.y} L ${p2.x},${p2.y}`;
    }
  }
}

/**
 * 3. Straight Directed Line
 */
export function getStraightPath(
  p1: AnchorPoint,
  p2: AnchorPoint
): string {
  return `M ${p1.x},${p1.y} L ${p2.x},${p2.y}`;
}

export function getConnectionPath(
  p1: AnchorPoint,
  p2: AnchorPoint,
  routing: "curved" | "orthogonal" | "straight" = "curved"
): string {
  switch (routing) {
    case "orthogonal":
      return getOrthogonalPath(p1, p2);
    case "straight":
      return getStraightPath(p1, p2);
    case "curved":
    default:
      return getCurvedBezierPath(p1, p2);
  }
}
```

```
// =====
// Arrowhead Polygon Generators
// =====

export function getArrowheadPath(
  tipX: number,
  tipY: number,
  fromX: number,
  fromY: number,
  type: ArrowheadType = "arrow",
  size: number = 10
): { path: string; fill: string; stroke: string } | null {
  if (type === "none") return null;

  const dx = tipX - fromX;
  const dy = tipY - fromY;
  const len = Math.hypot(dx, dy);
  if (len < 0.001) return null;

  const ux = dx / len;
  const uy = dy / len;
  const px = -uy;
  const py = ux;

  switch (type) {
    case "triangle":
      case "arrow": {
        const p1x = tipX - ux * size + px * (size * 0.5);
        const p1y = tipY - uy * size + py * (size * 0.5);
        const p2x = tipX - ux * size - px * (size * 0.5);
        const p2y = tipY - uy * size - py * (size * 0.5);
        return {
          path: `M ${tipX},${tipY} L ${p1x},${p1y} L ${p2x},${p2y} Z`,
          fill: "currentColor",
          stroke: "none",
        };
      }
    case "stealth": {
      const p1x = tipX - ux * (size * 1.2) + px * (size * 0.5);
      const p1y = tipY - uy * (size * 1.2) + py * (size * 0.5);
      const notchX = tipX - ux * (size * 0.8);
      const notchY = tipY - uy * (size * 0.8);
      const p2x = tipX - ux * (size * 1.2) - px * (size * 0.5);
      const p2y = tipY - uy * (size * 1.2) - py * (size * 0.5);
      return {
        path: `M ${tipX},${tipY} L ${p1x},${p1y} L ${notchX},${notchY} L ${p2x},${p2y} Z`,
        fill: "currentColor",
        stroke: "none",
      };
    }
    case "diamond": {
      const midX = tipX - ux * size;
      const midY = tipY - uy * size;
      const backX = tipX - ux * (size * 2);
      const backY = tipY - uy * (size * 2);
      const p1x = midX + px * (size * 0.45);
      const p1y = midY + py * (size * 0.45);
      const p2x = midX - px * (size * 0.45);
      const p2y = midY - py * (size * 0.45);
      return {
        path: `M ${tipX},${tipY} L ${p1x},${p1y} L ${backX},${backY} L ${p2x},${p2y} Z`,
        fill: "currentColor",
        stroke: "none",
      };
    }
    case "circle": {
      const cx = tipX - ux * (size * 0.5);
      const cy = tipY - uy * (size * 0.5);
      const r = size * 0.4;
      return {
        path: `M ${cx} - ${r},${cy} a ${r},${r} 0 1,0 ${r} * 2,0 a ${r},${r} 0 1,0 ${-r} * 2,0`,
        fill: "currentColor",
        stroke: "none",
      };
    }
    default:
      return null;
  }
}
```

```

    }
}

// =====
// Hit Testing
// =====

export function hitTestElement(
  el: WhiteboardElement,
  worldX: number,
  worldY: number,
  tolerance: number = HIT_THRESHOLD,
  allElements?: WhiteboardElement[]
): boolean {
  if (el.locked) return false;

  const b = getElementBounds(el);
  const cx = b.x + b.width / 2;
  const cy = b.y + b.height / 2;
  const rot = el.rotation || 0;

  // Un-rotate the point into element's local frame
  const local = rotatePoint(worldX, worldY, cx, cy, -rot);

  switch (el.type) {
    case "rectangle":
    case "text":
    case "sticky":
    case "image":
      return (
        local.x >= el.x - tolerance &&
        local.x <= el.x + el.width + tolerance &&
        local.y >= el.y - tolerance &&
        local.y <= el.y + el.height + tolerance
      );
    case "ellipse": {
      const rx = el.width / 2 + tolerance;
      const ry = el.height / 2 + tolerance;
      if (rx <= 0 || ry <= 0) return false;
      const nx = (local.x - cx) / rx;
      const ny = (local.y - cy) / ry;
      return nx * nx + ny * ny <= 1;
    }
    case "diamond": {
      const dx = Math.abs(local.x - cx) / (el.width / 2 + tolerance);
      const dy = Math.abs(local.y - cy) / (el.height / 2 + tolerance);
      return dx + dy <= 1;
    }
    case "triangle":
    case "star":
    case "cloud": {
      // Conservative bounding box test
      return (
        local.x >= el.x - tolerance &&
        local.x <= el.x + el.width + tolerance &&
        local.y >= el.y - tolerance &&
        local.y <= el.y + el.height + tolerance
      );
    }
    case "line":
    case "arrow": {
      const x1 = el.x;
      const y1 = el.y;
      const x2 = el.x + el.x2;
      const y2 = el.y + el.y2;
      // Use the un-rotated query point so rotated lines hit-test correctly
      const dist = distanceToSegment(local.x, local.y, x1, y1, x2, y2);
      return dist <= (el.strokeWidth || 2) / 2 + tolerance;
    }
    case "stroke": {
      if (!el.points || el.points.length === 0) return false;
      const radius = Math.max((el.strokeWidth || 3) / 2, 2) + tolerance;
      // Query point was un-rotated into the element's local frame above
      // (`local`), so rotated strokes hit-test against their VISIBLE geometry.
      if (el.points.length === 1) {
        const [px, py] = el.points[0];
        return Math.hypot(local.x - (el.x + px), local.y - (el.y + py)) <= radius;
      }
    }
  }
}

```

```

for (let i = 0; i < el.points.length - 1; i++) {
  const p1 = el.points[i];
  const p2 = el.points[i + 1];
  const dist = distanceToSegment(
    local.x,
    local.y,
    el.x + p1[0],
    el.y + p1[1],
    el.x + p2[0],
    el.y + p2[1]
  );
  if (dist <= radius) return true;
}
return false;
}
case "connection": {
  if (!allElements) return false;
  const conn = el as ConnectionElement;
  const sourceEl = allElements.find((e) => e.id === conn.source.elementId);
  const targetEl = allElements.find((e) => e.id === conn.target.elementId);
  if (!sourceEl || !targetEl) return false;

  const { source, target } = getOptimalConnectionAnchors(
    sourceEl,
    targetEl,
    conn.source.anchor,
    conn.target.anchor
  );

  const hitRadius = (conn.strokeWidth || 2.5) / 2 + tolerance + 4;
  const routing = conn.routing || "curved";

  if (routing === "straight") {
    return distanceToSegment(worldX, worldY, source.x, source.y, target.x, target.y) <= hitRadius;
  }

  if (routing === "orthogonal") {
    const dx = target.x - source.x;
    const dy = target.y - source.y;
    const dist = Math.hypot(dx, dy);
    const margin = Math.min(24, dist * 0.2);
    const startOutX = source.x + source.nx * margin;
    const startOutY = source.y + source.ny * margin;
    const endOutX = target.x + target.nx * margin;
    const endOutY = target.y + target.ny * margin;
    const midX = (startOutX + endOutX) / 2;
    const midY = (startOutY + endOutY) / 2;

    let waypoints: Array<{ x: number; y: number }> = [];
    if (Math.abs(source.nx) > 0.5 && Math.abs(target.nx) > 0.5) {
      waypoints = [
        { x: source.x, y: source.y },
        { x: midX, y: source.y },
        { x: midX, y: target.y },
        { x: target.x, y: target.y },
      ];
    } else if (Math.abs(source.ny) > 0.5 && Math.abs(target.ny) > 0.5) {
      waypoints = [
        { x: source.x, y: source.y },
        { x: source.x, y: midY },
        { x: target.x, y: midY },
        { x: target.x, y: target.y },
      ];
    } else {
      if (Math.abs(source.nx) > 0.5) {
        waypoints = [
          { x: source.x, y: source.y },
          { x: target.x, y: source.y },
          { x: target.x, y: target.y },
        ];
      } else {
        waypoints = [
          { x: source.x, y: source.y },
          { x: source.x, y: target.y },
          { x: target.x, y: target.y },
        ];
      }
    }
  }
}
}

```

```

    for (let i = 0; i < waypoints.length - 1; i++) {
      if (
        distanceToSegment(
          worldX,
          worldY,
          waypoints[i].x,
          waypoints[i].y,
          waypoints[i + 1].x,
          waypoints[i + 1].y
        ) <= hitRadius
      ) {
        return true;
      }
    }
    return false;
  }

  // Curved Bezier (sample 24 intervals along cubic curve)
  const dx = target.x - source.x;
  const dy = target.y - source.y;
  const dist = Math.hypot(dx, dy);
  const curvature = Math.min(Math.max(dist * 0.4, 30), 200);

  const cx1 = source.x + source.nx * curvature;
  const cy1 = source.y + source.ny * curvature;
  const cx2 = target.x + target.nx * curvature;
  const cy2 = target.y + target.ny * curvature;

  let prevX = source.x;
  let prevY = source.y;
  const steps = 24;
  for (let i = 1; i <= steps; i++) {
    const t = i / steps;
    const omt = 1 - t;
    const currX =
      omt * omt * omt * source.x +
      3 * omt * omt * t * cx1 +
      3 * omt * t * t * cx2 +
      t * t * t * target.x;
    const currY =
      omt * omt * omt * source.y +
      3 * omt * omt * t * cy1 +
      3 * omt * t * t * cy2 +
      t * t * t * target.y;

    if (distanceToSegment(worldX, worldY, prevX, prevY, currX, currY) <= hitRadius) {
      return true;
    }
    prevX = currX;
    prevY = currY;
  }
  return false;
}
default:
  return false;
}
}

/**
 * Test if a 2D point falls inside a closed 2D polygon using the ray-casting algorithm (even-odd rule).
 */
export function isPointInPolygon(
  px: number,
  py: number,
  polygon: Array<{ x: number; y: number }>
): boolean {
  if (polygon.length < 3) return false;
  let inside = false;
  for (let i = 0, j = polygon.length - 1; i < polygon.length; j = i++) {
    const xi = polygon[i].x;
    const yi = polygon[i].y;
    const xj = polygon[j].x;
    const yj = polygon[j].y;

    const intersect =
      yi > py !== yj > py &&
      px < ((xj - xi) * (py - yi)) / (yj - yi) + xi;

```

```

    if (intersect) inside = !inside;
  }
  return inside;
}

/**
 * Every visible point of an element, in world units: rotation pivot first,
 * then stroke samples / line endpoints, then the visible bounds corners.
 * Loose lasso selects when ANY is inside; strict encircle requires ALL.
 */
export function sampleElementPoints(el: WhiteboardElement): Array<{ x: number; y: number }> {
  const b = getElementBounds(el);
  const rot = el.rotation || 0;
  const bcx = b.x + b.width / 2;
  const bcy = b.y + b.height / 2;
  const place =
    rot !== 0 ? (x: number, y: number) => rotatePoint(x, y, bcx, bcy, rot) : (x: number, y: number) => ({ x, y
    });

  // 1. Center containment (the center is the rotation pivot, so it is
  // rotation-invariant).
  const center = getElementCenter(el);
  const pts = [place(center.x, center.y)];

  // 2. Stroke sample points (rotated into visible position)
  if (el.type === "stroke" && el.points) {
    for (const [px, py] of el.points) {
      pts.push(place(el.x + px, el.y + py));
    }
  }

  // 3. Line / Arrow endpoints (rotated into visible position)
  if (el.type === "line" || el.type === "arrow") {
    pts.push(place(el.x, el.y));
    pts.push(place(el.x + el.x2, el.y + el.y2));
  }

  // 4. Bounding box corners – rotated so lasso matches the VISIBLE quad
  for (const corner of [
    { x: b.x, y: b.y },
    { x: b.x + b.width, y: b.y },
    { x: b.x + b.width, y: b.y + b.height },
    { x: b.x, y: b.y + b.height },
  ]) {
    pts.push(place(corner.x, corner.y));
  }

  return pts;
}

/**
 * Test if a whiteboard element is contained in or intersects with a freeform polygon (e.g. for Lasso selectio
 * n).
 */
export function isElementInPolygon(
  el: WhiteboardElement,
  polygon: Array<{ x: number; y: number }>
): boolean {
  if (polygon.length < 3) return false;
  return sampleElementPoints(el).some((p) => isPointInPolygon(p.x, p.y, polygon));
}

/**
 * Where an element actually IS, for a containment test – which is not the
 * same set of points as "any part of it", and the difference decides whether
 * strict encircle is usable on handwriting at all.
 *
 * A stroke's bounding box is a rectangle around a curve: for a diagonal line
 * the box corners sit in empty space, far off the ink. Requiring THOSE inside
 * the loop means the Samsung gesture this mode exists for – snaking a loop
 * between two lines of writing to lift one word – always fails, because the
 * corners of the word's box are up in the line above. So ink is tested as
 * ink, and only elements with no finer geometry fall back to their box.
 */
export function outlineElementPoints(el: WhiteboardElement): Array<{ x: number; y: number }> {
  const b = getElementBounds(el);
  const rot = el.rotation || 0;
  const bcx = b.x + b.width / 2;

```

```

const bcy = b.y + b.height / 2;
const place =
  rot !== 0 ? (x: number, y: number) => rotatePoint(x, y, bcx, bcy, rot) : (x: number, y: number) => ({ x, y
    });

// Handwriting: the ink itself, sampled along the path.
if (el.type === "stroke" && el.points && el.points.length > 0) {
  return el.points.map(([px, py]) => place(el.x + px, el.y + py));
}

// A line or arrow is its two ends; the box around a diagonal is mostly air.
if (el.type === "line" || el.type === "arrow") {
  return [place(el.x, el.y), place(el.x + el.x2, el.y + el.y2)];
}

// Rectangles, ellipses, text, stickies, images: the box IS the extent.
return [
  { x: b.x, y: b.y },
  { x: b.x + b.width, y: b.y },
  { x: b.x + b.width, y: b.y + b.height },
  { x: b.x, y: b.y + b.height },
].map((c) => place(c.x, c.y));
}

/**
 * Samsung-style strict encircle: the whole element must lie inside the loop.
 * A neighbour the loop only grazes stays unselected – the surgical precision
 * the loose lasso trades away for speed.
 */
export function isElementFullyInPolygon(
  el: WhiteboardElement,
  polygon: Array<{ x: number; y: number }>
): boolean {
  if (polygon.length < 3) return false;
  const pts = outlineElementPoints(el);
  if (pts.length === 0) return false;
  return pts.every((p) => isPointInPolygon(p.x, p.y, polygon));
}

function distanceToSegment(
  px: number,
  py: number,
  x1: number,
  y1: number,
  x2: number,
  y2: number
): number {
  const dx = x2 - x1;
  const dy = y2 - y1;
  const lenSq = dx * dx + dy * dy;
  if (lenSq === 0) return Math.hypot(px - x1, py - y1);
  const t = clamp(((px - x1) * dx + (py - y1) * dy) / lenSq, 0, 1);
  const projX = x1 + t * dx;
  const projY = y1 + t * dy;
  return Math.hypot(px - projX, py - projY);
}

// =====
// Resize Handles
// =====

export function getResizeHandles(
  bounds: BoundingBox,
  rotation: number,
  camera: { x: number; y: number; zoom: number },
  viewport: Viewport
): Array<{ handle: string; x: number; y: number }> {
  const cx = bounds.x + bounds.width / 2;
  const cy = bounds.y + bounds.height / 2;
  const hw = bounds.width / 2;
  const hh = bounds.height / 2;

  const localHandles: Array<{ handle: string; x: number; y: number }> = [
    { handle: "nw", x: cx - hw, y: cy - hh },
    { handle: "n", x: cx, y: cy - hh },
    { handle: "ne", x: cx + hw, y: cy - hh },
    { handle: "e", x: cx + hw, y: cy },
    { handle: "se", x: cx + hw, y: cy + hh },
  ]

```

```

    { handle: "s", x: cx, y: cy + hh },
    { handle: "sw", x: cx - hw, y: cy + hh },
    { handle: "w", x: cx - hw, y: cy },
    // 44 screen px above the top edge: far enough that the rotate handle's
    // touch target does not swallow the top-centre resize handle's.
    { handle: "rotate", x: cx, y: cy - hh - 44 / camera.zoom },
  ];

  return localHandles.map((h) => {
    const rotated = rotatePoint(h.x, h.y, cx, cy, rotation);
    return {
      handle: h.handle,
      x: (rotated.x - camera.x) * camera.zoom + viewport.width / 2,
      y: (rotated.y - camera.y) * camera.zoom + viewport.height / 2,
    };
  });
}

// =====
// Viewport Culling
// =====

export function isVisibleInViewport(
  bounds: BoundingBox,
  rotation: number,
  camera: { x: number; y: number; zoom: number },
  viewport: Viewport
): boolean {
  const halfW = viewport.width / 2 / camera.zoom;
  const halfH = viewport.height / 2 / camera.zoom;
  const margin = 80;
  const viewLeft = camera.x - halfW - margin;
  const viewRight = camera.x + halfW + margin;
  const viewTop = camera.y - halfH - margin;
  const viewBottom = camera.y + halfH + margin;

  return !(
    bounds.x + bounds.width < viewLeft ||
    bounds.x > viewRight ||
    bounds.y + bounds.height < viewTop ||
    bounds.y > viewBottom
  );
}

// =====
// Alignment & Distribution Utilities
// =====

export type AlignmentType = "left" | "center" | "right" | "top" | "middle" | "bottom";

export function alignElements(
  elements: WhiteboardElement[],
  alignment: AlignmentType
): WhiteboardElement[] {
  if (elements.length <= 1) return elements;
  const union = getUnionBounds(elements);
  if (!union) return elements;

  return elements.map((el) => {
    if (el.type === "connection") return el;
    const b = getElementBounds(el);
    let dx = 0;
    let dy = 0;

    switch (alignment) {
      case "left":
        dx = union.x - b.x;
        break;
      case "center":
        dx = union.x + (union.width - b.width) / 2 - b.x;
        break;
      case "right":
        dx = union.x + union.width - b.width - b.x;
        break;
      case "top":
        dy = union.y - b.y;
        break;
      case "middle":

```

```
        dy = union.y + (union.height - b.height) / 2 - b.y;
        break;
      case "bottom":
        dy = union.y + union.height - b.height - b.y;
        break;
    }

    return {
      ...el,
      x: el.x + dx,
      y: el.y + dy,
      updatedAt: Date.now(),
    };
  });
}

export function distributeElements(
  elements: WhiteboardElement[],
  axis: "horizontal" | "vertical"
): WhiteboardElement[] {
  const validElements = elements.filter((el) => el.type !== "connection");
  if (validElements.length < 3) return elements;

  if (axis === "horizontal") {
    const sorted = [...validElements].sort((a, b) => {
      const ba = getElementBounds(a);
      const bb = getElementBounds(b);
      return ba.x - bb.x;
    });

    const first = getElementBounds(sorted[0]);
    const last = getElementBounds(sorted[sorted.length - 1]);
    const totalWidth = sorted.reduce((sum, el) => sum + getElementBounds(el).width, 0);
    const span = last.x + last.width - first.x;
    const gap = (span - totalWidth) / (sorted.length - 1);

    let currentX = first.x;
    const updatedMap = new Map<string, number>();

    for (let i = 0; i < sorted.length; i++) {
      const el = sorted[i];
      const b = getElementBounds(el);
      const dx = currentX - b.x;
      updatedMap.set(el.id, el.x + dx);
      currentX += b.width + gap;
    }

    return elements.map((el) => {
      if (updatedMap.has(el.id)) {
        return { ...el, x: updatedMap.get(el.id)!, updatedAt: Date.now() };
      }
      return el;
    });
  } else {
    const sorted = [...validElements].sort((a, b) => {
      const ba = getElementBounds(a);
      const bb = getElementBounds(b);
      return ba.y - bb.y;
    });

    const first = getElementBounds(sorted[0]);
    const last = getElementBounds(sorted[sorted.length - 1]);
    const totalHeight = sorted.reduce((sum, el) => sum + getElementBounds(el).height, 0);
    const span = last.y + last.height - first.y;
    const gap = (span - totalHeight) / (sorted.length - 1);

    let currentY = first.y;
    const updatedMap = new Map<string, number>();

    for (let i = 0; i < sorted.length; i++) {
      const el = sorted[i];
      const b = getElementBounds(el);
      const dy = currentY - b.y;
      updatedMap.set(el.id, el.y + dy);
      currentY += b.height + gap;
    }

    return elements.map((el) => {
```

```

        if (updatedMap.has(el.id)) {
            return { ...el, y: updatedMap.get(el.id)!, updatedAt: Date.now() };
        }
        return el;
    });
}
}

=====
// === END FILE: src/lib/whiteboard/geometry.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/grid.ts ===
// File 101 of 194 | Lines: 31 | Size: 843 bytes
=====

/**
 * Grid and Snapping Math (v3 – Obsidian Canvas & Vector Grade)
 * =====
 * Standard grid size and point snapping utilities.
 */

export const GRID_SIZE = 24;

/**
 * Snap a single scalar value to the nearest multiple of `gridSize`.
 * Normalizes -0 to 0 for clean numerical precision.
 */
export function snapToGrid(val: number, gridSize: number = GRID_SIZE): number {
    if (!gridSize || gridSize <= 0) return val;
    const res = Math.round(val / gridSize) * gridSize;
    return res === 0 ? 0 : res;
}

/**
 * Snap 2D coordinate point (x, y) to the nearest grid intersection.
 */
export function snapPointToGrid(
    x: number,
    y: number,
    gridSize: number = GRID_SIZE
): { x: number; y: number } {
    return {
        x: snapToGrid(x, gridSize),
        y: snapToGrid(y, gridSize),
    };
}

=====
// === END FILE: src/lib/whiteboard/grid.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/imageEdit.ts ===
// File 102 of 194 | Lines: 253 | Size: 9737 bytes
=====

/**
 * Image edits
 * =====
 * Crop, flip, a look (filter) and a frame shape for pictures on the page.
 *
 * Every edit is a DESCRIPTION stored on the element, never new pixels: the
 * photo in IndexedDB stays exactly as it was taken. That keeps each edit
 * undoable, lets "Reset" bring back the whole photo months later, and costs
 * no extra storage however many times a picture is re-cropped.
 *
 * The canvas renderer and the exporter both draw from `imageLayout`, so a
 * picture looks the same on screen, in a PNG and in a PDF.
 */

import type { ImageCrop, ImageElement, ImageFilter, ImageFrame } from "../types";

export const FULL_CROP: ImageCrop = { x: 0, y: 0, w: 1, h: 1 };

```

```

/** The smallest crop, as a fraction of the picture: always something left to grab. */
export const MIN_CROP = 0.04;

export const IMAGE_FILTERS: Array<{ id: ImageFilter; label: string; hint: string; css: string }> = [
  { id: "none", label: "Original", hint: "As taken", css: "" },
  {
    id: "document",
    label: "Document",
    hint: "Whiter paper, darker ink – for photos of pages and boards",
    css: "grayscale(1) contrast(1.6) brightness(1.12)",
  },
  { id: "mono", label: "Gray", hint: "Black and white", css: "grayscale(1)" },
  { id: "vivid", label: "Vivid", hint: "Stronger colour", css: "saturate(1.45) contrast(1.08)" },
  {
    id: "faded",
    label: "Faded",
    hint: "Pale, so your writing stands out on top of it",
    css: "opacity(0.45) grayscale(0.3)",
  },
];

export const IMAGE_FRAMES: Array<{ id: ImageFrame; label: string }> = [
  { id: "square", label: "Square" },
  { id: "rounded", label: "Rounded" },
  { id: "circle", label: "Circle" },
];

export function filterCss(filter: ImageFilter | undefined): string {
  return IMAGE_FILTERS.find((f) => f.id === filter)?.css ?? "";
}

const clamp = (v: number, lo: number, hi: number) => Math.min(hi, Math.max(lo, v));

/** A crop that is always inside the picture and never smaller than MIN_CROP. */
export function normalizeCrop(c: ImageCrop | undefined): ImageCrop {
  if (!c || ![c.x, c.y, c.w, c.h].every(Number.isFinite)) return { ...FULL_CROP };
  const w = clamp(c.w, MIN_CROP, 1);
  const h = clamp(c.h, MIN_CROP, 1);
  return { x: clamp(c.x, 0, 1 - w), y: clamp(c.y, 0, 1 - h), w, h };
}

export function isFullCrop(c: ImageCrop | undefined): boolean {
  if (!c) return true;
  const n = normalizeCrop(c);
  return n.x < 1e-4 && n.y < 1e-4 && n.w > 1 - 1e-4 && n.h > 1 - 1e-4;
}

/**
 * Mirrors a crop with the picture. Crops are measured on the picture as you
 * SEE it, so flipping the picture must move the kept window with it, or a
 * flip would silently show a different part of the photo.
 */
export function flipCrop(c: ImageCrop, axis: "x" | "y"): ImageCrop {
  return axis === "x" ? { ...c, x: 1 - c.x - c.w } : { ...c, y: 1 - c.y - c.h };
}

/** Whether the picture is drawn exactly as a plain, unedited image. */
export function isUnedited(el: ImageElement): boolean {
  return (
    isFullCrop(el.crop) &&
    !el.flipX &&
    !el.flipY &&
    (!el.filter || el.filter === "none") &&
    (!el.frame || el.frame === "square") &&
    !(el.borderWidth && el.borderWidth > 0)
  );
}

/** The picture's own pixel size, with a fallback for old elements that lack it. */
export function naturalSize(el: ImageElement): { nw: number; nh: number } {
  const nw = el.naturalWidth > 0 ? el.naturalWidth : el.width || 1;
  const nh = el.naturalHeight > 0 ? el.naturalHeight : el.height || 1;
  return { nw, nh };
}

export interface ImageLayout {
  /** viewBox of the inner <svg>, in the picture's own pixels. */
  viewBox: { x: number; y: number; w: number; h: number };
}

```

```

    nw: number;
    nh: number;
    /** Transform for the <image> inside the viewBox (the flip). */
    flip: string;
    filter: string;
    frame: ImageFrame;
    /** Corner radius of the frame, in element units. */
    radius: number;
}

export function imageLayout(el: ImageElement): ImageLayout {
    const { nw, nh } = naturalSize(el);
    const c = normalizeCrop(el.crop);
    const sx = el.flipX ? -1 : 1;
    const sy = el.flipY ? -1 : 1;
    const flip =
        sx === 1 && sy === 1 ? "" : `translate(${sx === -1 ? nw : 0} ${sy === -1 ? nh : 0}) scale(${sx} ${sy})`;
    const frame = el.frame ?? "square";
    return {
        viewBox: { x: c.x * nw, y: c.y * nh, w: c.w * nw, h: c.h * nh },
        nw,
        nh,
        flip,
        filter: filterCss(el.filter),
        frame,
        radius: frame === "rounded" ? Math.min(el.width, el.height) * 0.08 : 0,
    };
}

/**
 * The element's new box after a crop, keeping the picture at the SAME scale
 * and centred where it was: cropping takes away, it never zooms.
 */
export function boxForCrop(
    el: ImageElement,
    next: ImageCrop
): { x: number; y: number; width: number; height: number } {
    const { nw, nh } = naturalSize(el);
    const prev = normalizeCrop(el.crop);
    const n = normalizeCrop(next);
    // Scale on screen per picture pixel. "meet" fits the crop inside the box,
    // so the smaller ratio is what is actually drawn.
    const scale = Math.min(el.width / (prev.w * nw), el.height / (prev.h * nh));
    const width = n.w * nw * scale;
    const height = n.h * nh * scale;
    const cx = el.x + el.width / 2;
    const cy = el.y + el.height / 2;
    return { x: cx - width / 2, y: cy - height / 2, width, height };
}

/** The next quarter turn, snapping a hand-rotated picture to the grid first. */
export function quarterTurn(rotation: number, dir: 1 | -1): number {
    const q = Math.PI / 2;
    const snapped = Math.round((rotation || 0) / q) * q;
    let next = snapped + dir * q;
    // Keep the stored angle in  $(-\pi, \pi]$  so it never grows without bound.
    while (next > Math.PI + 1e-9) next -= 2 * Math.PI;
    while (next <= -Math.PI + 1e-9) next += 2 * Math.PI;
    return Math.abs(next) < 1e-9 ? 0 : next;
}

/**
 * Moves or resizes a crop from one handle, in crop fractions.
 * `handle` is one of n/s/e/w/ne/nw/se/sw, or "move". `aspect` (w/h in
 * PICTURE pixels, i.e. already divided by nw/nh) holds a ratio when set.
 */
export type CropHandle = "move" | "n" | "s" | "e" | "w" | "ne" | "nw" | "se" | "sw";

export function dragCrop(
    start: ImageCrop,
    handle: CropHandle,
    dx: number,
    dy: number,
    aspect: number | null
): ImageCrop {
    if (handle === "move") {
        return {
            ...start,

```

```

    x: clamp(start.x + dx, 0, 1 - start.w),
    y: clamp(start.y + dy, 0, 1 - start.h),
  };
}
let x0 = start.x;
let y0 = start.y;
let x1 = start.x + start.w;
let y1 = start.y + start.h;
if (handle.includes("w")) x0 = clamp(x0 + dx, 0, x1 - MIN_CROP);
if (handle.includes("e")) x1 = clamp(x1 + dx, x0 + MIN_CROP, 1);
if (handle.includes("n")) y0 = clamp(y0 + dy, 0, y1 - MIN_CROP);
if (handle.includes("s")) y1 = clamp(y1 + dy, y0 + MIN_CROP, 1);
let out: ImageCrop = { x: x0, y: y0, w: x1 - x0, h: y1 - y0 };
if (aspect && aspect > 0) out = fitAspect(out, aspect, handle);
return out;
}

/**
 * Forces a crop to a ratio (in fractions: w/h where 1 means the picture's own
 * shape), shrinking the free side and anchoring the edge opposite the handle.
 */
export function fitAspect(c: ImageCrop, aspect: number, handle: CropHandle = "se"): ImageCrop {
  let w = c.w;
  let h = c.h;
  if (w / h > aspect) w = h * aspect;
  else h = w / aspect;
  // Too big for the picture: shrink both, keeping the ratio.
  if (w > 1) {
    w = 1;
    h = w / aspect;
  }
  if (h > 1) {
    h = 1;
    w = h * aspect;
  }
  const x = handle.includes("w") ? c.x + c.w - w : handle === "n" || handle === "s" ? c.x + (c.w - w) / 2 : c.x;
  const y = handle.includes("n") ? c.y + c.h - h : handle === "e" || handle === "w" ? c.y + (c.h - h) / 2 : c.y;
  return normalizeCrop({ x, y, w, h });
}

/** A centred crop of the given ratio (w/h in fractions), as large as fits. */
export function centeredAspectCrop(aspect: number): ImageCrop {
  const w = aspect >= 1 ? 1 : aspect;
  const h = aspect >= 1 ? 1 / aspect : 1;
  return normalizeCrop({ x: (1 - w) / 2, y: (1 - h) / 2, w, h });
}

const esc = (s: string) =>
  s.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;").replace(/"/g, "&quot;");

/**
 * The picture as standalone SVG markup, placed at the element's position.
 * Used by export, so a file shows exactly what the canvas shows.
 */
export function imageSvgMarkup(el: ImageElement, href: string, extraAttrs = ""): string {
  const L = imageLayout(el);
  const vb = L.viewBox;
  const clipId = `imgclip-${el.id}`;
  const clipShape =
    L.frame === "circle"
      ? `

```

```

    `<svg x="0" y="0" width="${el.width}" height="${el.height}" viewBox="${vb.x} ${vb.y} ${vb.w} ${vb.h}" preserveAspectRatio="xMidYMid meet">` +
    `<image href="${esc(href)}" x="0" y="0" width="${L.nw}" height="${L.nh}" preserveAspectRatio="none"${flip} ${style} />` +
    `</svg></g>${border}</g>`
  );
}

```

```

// === END FILE: src/lib/whiteboard/imageEdit.ts ===

```

```

// === START FILE: src/lib/whiteboard/imageImport.ts ===
// File 103 of 194 | Lines: 59 | Size: 2346 bytes

```

```

/**
 * Getting a picture ready to go on the page.
 *
 * A tablet camera photo is 12-50 megapixels and 4-12 MB. Stored as taken, a
 * dozen of them fill the app's storage, and every one costs memory each time
 * the page is drawn - for detail no one can see on a notebook page. So a
 * picture larger than MAX_SIDE is scaled down once, here, before it is saved.
 * Small pictures, and anything that is not a plain photo (GIF, SVG), pass
 * through untouched.
 */

export const MAX_SIDE = 2560;

export function readAsDataURL(file: Blob): Promise<string> {
  return new Promise((resolve, reject) => {
    const r = new FileReader();
    r.onload = () => resolve(String(r.result ?? ""));
    r.onerror = () => reject(r.error ?? new Error("Could not read the picture."));
    r.readAsDataURL(file);
  });
}

function loadImg(src: string): Promise<HTMLImageElement> {
  return new Promise((resolve, reject) => {
    const img = new Image();
    img.onload = () => resolve(img);
    img.onerror = () => reject(new Error("This file is not a picture ProNotes can open."));
    img.src = src;
  });
}

/** The size a picture is scaled to so its longer side is at most `max`. */
export function fitWithin(w: number, h: number, max = MAX_SIDE): { w: number; h: number } {
  const s = Math.min(1, max / Math.max(w, h, 1));
  return { w: Math.max(1, Math.round(w * s)), h: Math.max(1, Math.round(h * s)) };
}

export async function prepareImage(file: Blob): Promise<string> {
  const dataUrl = await readAsDataURL(file);
  if (/^data:image\/(gif|svg)/i.test(dataUrl)) return dataUrl;
  const img = await loadImg(dataUrl);
  const { naturalWidth: w, naturalHeight: h } = img;
  if (Math.max(w, h) <= MAX_SIDE) return dataUrl;
  const size = fitWithin(w, h);
  const canvas = document.createElement("canvas");
  canvas.width = size.w;
  canvas.height = size.h;
  const ctx = canvas.getContext("2d");
  if (!ctx) return dataUrl;
  ctx.imageSmoothingQuality = "high";
  ctx.drawImage(img, 0, 0, size.w, size.h);
  // PNGs may be screenshots or diagrams with transparency: keep them PNG.
  const png = /^data:image\/png/i.test(dataUrl);
  return canvas.toDataURL(png ? "image/png" : "image/jpeg", 0.9);
}

export function isImageFile(file: File): boolean {
  return file.type.startsWith("image/") || /\. (png|jpe?g|gif|webp|bmp|svg|heic|heif)$/i.test(file.name);
}

```

```

=====
// === END FILE: src/lib/whiteboard/imageImport.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/imagestore.ts ===
// File 104 of 194 | Lines: 147 | Size: 5782 bytes
=====

/**
 * Image storage (IndexedDB)
 * =====
 * Image pixels live here, NOT in the workspace that goes to localStorage.
 *
 * The measured problem: a single pasted image on the demo board was **908 KB
 * of a 918 KB workspace** – 99% of it. localStorage allows roughly 5 MB, so a
 * handful of photos fills it, and the failure lands on SAVE: you would work
 * for an hour and then discover nothing had been written.
 *
 * When that happened the fallback dropped every image's `src` to `` to get
 * under the limit. The pixels were gone for good, because the data URL in the
 * workspace was the only copy anyone had.
 *
 * So images follow the same rule as imported PDFs: the bytes go to IndexedDB
 * once, and the element keeps only `asset:<id>`. That keeps the workspace
 * small, and the pixels survive a quota event because they were never in the
 * thing that overflowed.
 */

const DB_NAME = "pronotes-assets";
const DB_VERSION = 1;
const STORE = "images";

/** Marks a src as a reference into this store rather than inline pixels. */
export const ASSET_PREFIX = "asset:";

export interface StoredImage {
  id: string;
  /** The original data URL, kept verbatim so re-encoding never degrades it. */
  dataUrl: string;
  addedAt: number;
}

export function isAssetRef(src: string | undefined | null): boolean {
  return typeof src === "string" && src.startsWith(ASSET_PREFIX);
}

export function assetIdOf(src: string): string {
  return src.slice(ASSET_PREFIX.length);
}

function openDb(): Promise<IDBDatabase> {
  return new Promise((resolve, reject) => {
    if (typeof indexedDB === "undefined") {
      reject(new Error("This browser has no IndexedDB, so images cannot be stored."));
      return;
    }
    const req = indexedDB.open(DB_NAME, DB_VERSION);
    req.onupgradeneeded = () => {
      const db = req.result;
      if (!db.objectStoreNames.contains(STORE)) db.createObjectStore(STORE, { keyPath: "id" });
    };
    req.onsuccess = () => resolve(req.result);
    req.onerror = () => reject(req.error ?? new Error("Could not open the image store"));
  });
}

function tx<T>(mode: IDBTransactionMode, run: (s: IDBObjectStore) => IDBRequest<T>): Promise<T> {
  return openDb().then(
    (db) =>
    new Promise<T>((resolve, reject) => {
      const t = db.transaction(STORE, mode);
      const req = run(t.objectStore(STORE));
      req.onsuccess = () => resolve(req.result);
      req.onerror = () => reject(req.error ?? new Error("Image store request failed"));
      t.oncomplete = () => db.close();
    })
  );
}

```

```

    });
}

/**
 * Stores a data URL and returns the `asset:<id>` src to put on the element.
 *
 * If IndexedDB is unavailable the ORIGINAL data URL is returned instead. That
 * keeps the picture – an inline image that risks the quota is still far better
 * than losing it outright – and the degraded-save path remains as the backstop.
 */
export async function storeImage(dataUrl: string): Promise<string> {
  if (!dataUrl.startsWith("data:")) return dataUrl;
  const id = `img_${Date.now().toString(36)}_${Math.random().toString(36).slice(2, 9)}`;
  try {
    await tx("readwrite", (s) =>
      s.put({ id, dataUrl, addedAt: Date.now() } as StoredImage) as unknown as IDBRequest<void>
    );
    return `${ASSET_PREFIX}${id}`;
  } catch (err) {
    // Keeping the picture inline is still the right call - it beats losing it
    // - but it puts the image back into the ~5 MB workspace, which is the exact
    // quota problem this store exists to avoid. A few of these and saving
    // starts failing, so the user is told NOW rather than at the failed save.
    console.warn("[imagestore] Could not store an image in IndexedDB; keeping it inline.", err);
    if (typeof window !== "undefined") {
      window.dispatchEvent(new CustomEvent("wb:image-inline", { detail: { bytes: dataUrl.length } }));
    }
    return dataUrl;
  }
}

export async function loadImage(id: string): Promise<string | null> {
  try {
    const rec = await tx<StoredImage | undefined>("readonly", (s) => s.get(id));
    return rec?.dataUrl ?? null;
  } catch {
    return null;
  }
}

/**
 * The newest stored pictures, for "use again" in the image sheet. Only ids
 * still referenced by some note are worth offering; the caller filters.
 */
export async function listRecentImages(limit = 12): Promise<StoredImage[]> {
  try {
    // Keys only: getAll() would pull every photo's pixels into memory. Ids
    // start with their creation time (see storeImage), so they sort by age.
    const keys = await tx<IDBValidKey[]>("readonly", (s) => s.getAllKeys());
    const born = (k: string) => parseInt(k.split("_")[1] ?? "0", 36) || 0;
    const newest = (keys as string[]).sort((a, b) => born(b) - born(a)).slice(0, limit);
    const out: StoredImage[] = [];
    for (const id of newest) {
      const rec = await tx<StoredImage | undefined>("readonly", (s) => s.get(id));
      if (rec) out.push(rec);
    }
    return out;
  } catch {
    return [];
  }
}

/** Removes images nothing references any more. */
export async function pruneUnusedImages(referenced: Set<string>): Promise<number> {
  try {
    const all = await tx<StoredImage[]>("readonly", (s) => s.getAll() as IDBRequest<StoredImage[]>);
    const orphans = all.filter((r) => !referenced.has(r.id));
    for (const o of orphans) {
      await tx("readwrite", (s) => s.delete(o.id) as unknown as IDBRequest<void>);
    }
    return orphans.length;
  } catch (err) {
    // Pruning is housekeeping and must never break the delete that triggered
    // it - but a prune that always fails means storage only ever grows, so
    // it is logged rather than swallowed.
    console.warn("[imagestore] Pruning unused images failed; storage was not reclaimed.", err);
    return 0;
  }
}

```

```

}

=====
// === END FILE: src/lib/whiteboard/imagestore.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/input.ts ===
// File 105 of 194 | Lines: 241 | Size: 9286 bytes
=====

/**
 * Wheel input interpretation
 * =====
 * Turns a raw wheel event into "zoom by this factor" or "pan by this much",
 * kept out of the component so it can be unit-tested against what real devices
 * actually send.
 *
 * Two things browsers get you wrong if you ignore them:
 *
 * 1. `deltaMode`. Chrome sends pixels (0), but Firefox sends LINES (1) – about
 *    3 per wheel notch – and page mode (2) exists too. Treating 3 lines as 3
 *    pixels makes the wheel feel inert and pinch-zoom crawl at 0.2% per event.
 *
 * 2. A trackpad pinch arrives as a wheel event with `ctrlKey` set. That is a
 *    browser convention, not a real Ctrl press, so pinch and Ctrl+wheel are
 *    indistinguishable – and both should zoom.
 */

export type WheelMode = "auto" | "trackpad" | "mouse";

export interface RawWheel {
  deltaX: number;
  deltaY: number;
  deltaMode: number;
  ctrlKey: boolean;
  metaKey: boolean;
  shiftKey: boolean;
}

export type WheelAction =
  | { kind: "zoom"; factor: number }
  | { kind: "pan"; dx: number; dy: number };

/** One wheel notch in Firefox's line units is roughly this many pixels. */
const PIXELS_PER_LINE = 16;
const PIXELS_PER_PAGE = 800;

export function normalizeWheelDelta(e: RawWheel): { dx: number; dy: number } {
  const scale =
    e.deltaMode === 1 ? PIXELS_PER_LINE : e.deltaMode === 2 ? PIXELS_PER_PAGE : 1;
  return { dx: e.deltaX * scale, dy: e.deltaY * scale };
}

/**
 * Is this event from a mouse wheel rather than a trackpad?
 *
 * Mouse wheels emit large, quantised steps on one axis (100/120px in Chrome,
 * 3 lines in Firefox) and never a horizontal component. Trackpads emit small,
 * often fractional deltas and usually some horizontal drift. This is a
 * heuristic, which is why the user can override it in Settings.
 */
export function looksLikeMouseWheel(e: RawWheel): boolean {
  if (e.deltaMode !== 0) return true; // line/page units only ever come from a wheel
  if (e.deltaX !== 0) return false; // trackpads drift sideways; wheels do not
  const abs = Math.abs(e.deltaY);
  return abs >= 50 && Number.isInteger(e.deltaY);
}

// -----
// Which device is this, really?
// -----
//
// `looksLikeMouseWheel` judges ONE event, and that is what made two-finger
// scrolling zoom "sometimes": a Windows precision touchpad flicked straight
// down sends deltaX exactly 0 with an integer deltaY over 50 – identical to a
// wheel notch – while the same gesture with a little sideways drift reads as a

```

```

// trackpad. One gesture, two answers, so the canvas zoomed at random.
//
// A device does not change between events. So the source is decided across a
// short run of events and then held, and only re-decided after a pause long
// enough to mean a new gesture.

export type WheelSource = "mouse" | "trackpad";

export interface WheelSample {
  deltaX: number;
  deltaY: number;
  deltaMode: number;
  time: number;
}

export interface WheelDeviceState {
  samples: WheelSample[];
  verdict: WheelSource | null;
}

/** A gap longer than this means a new gesture; the verdict is re-decided. */
export const WHEEL_GESTURE_GAP_MS = 400;
/** Trackpads stream; wheels click. Consecutive events closer than this are a stream. */
export const TRACKPAD_STREAM_GAP_MS = 40;

const MAX_SAMPLES = 12;

export function createWheelDeviceState(): WheelDeviceState {
  return { samples: [], verdict: null };
}

/** Decisive evidence from the samples themselves, or null when unsure. */
function decideSource(samples: WheelSample[]): WheelSource | null {
  for (let i = samples.length - 1; i >= 0; i--) {
    const s = samples[i];
    // Line and page units only ever come from a real wheel.
    if (s.deltaMode !== 0) return "mouse";
    // Wheels have no horizontal axis, and cannot produce fractions.
    if (s.deltaX !== 0) return "trackpad";
    if (!Number.isInteger(s.deltaY)) return "trackpad";
  }

  // Cadence is what separates the ambiguous case: a touchpad reports a
  // continuous stream every few milliseconds, a wheel sends discrete notches.
  if (samples.length >= 3) {
    let rapid = 0;
    for (let i = 1; i < samples.length; i++) {
      if (samples[i].time - samples[i - 1].time < TRACKPAD_STREAM_GAP_MS) rapid++;
    }
    if (rapid >= 2) return "trackpad";
  }

  return null;
}

export function observeWheel(state: WheelDeviceState, sample: WheelSample): WheelDeviceState {
  const last = state.samples[state.samples.length - 1];
  const newGesture = !last || sample.time - last.time > WHEEL_GESTURE_GAP_MS;

  const samples = (newGesture ? [] : state.samples).concat(sample).slice(-MAX_SAMPLES);
  const carried = newGesture ? null : state.verdict;
  return { samples, verdict: decideSource(samples) ?? carried };
}

/** The device behind this event: the held verdict, else the single-event guess. */
export function wheelSourceOf(state: WheelDeviceState, e: RawWheel): WheelSource {
  return state.verdict ?? (looksLikeMouseWheel(e) ? "mouse" : "trackpad");
}

/**
 * Turns a scroll pan into the arguments `panCamera` expects.
 *
 * Two opposite conventions meet here, which is exactly why scrolling came out
 * inverted:
 * - a DRAG delta is how far a finger moved, and the content follows the hand
 *   (`panCamera` subtracts it from the camera);
 * - a SCROLL delta means "show me what is further down", so the camera must
 *   move the OTHER way.
 */

```

```

*
* Passing a scroll delta straight to `panCamera` therefore scrolls backwards:
* swiping up on the touchpad moved the view up instead of down.
*/
export function cameraPanForScroll(pan: { dx: number; dy: number }): { dx: number; dy: number } {
  return { dx: -pan.dx, dy: -pan.dy };
}

export function resolveWheelAction(
  e: RawWheel,
  mode: WheelMode,
  source?: WheelSource
): WheelAction {
  const { dx, dy } = normalizeWheelDelta(e);

  // A pinch (ctrlKey) always zooms, in every mode – otherwise pinching would
  // scroll the canvas away, which is never what the gesture means.
  const pinchOrModifier = e.ctrlKey || e.metaKey;

  const treatAsMouse =
    mode === "mouse" ||
    (mode === "auto" && (source ? source === "mouse" : looksLikeMouseWheel(e)));

  if (pinchOrModifier || (treatAsMouse && !e.shiftKey)) {
    // 0.0015 per pixel gives ~14% per mouse notch and a smooth trackpad pinch,
    // and being exponential it is symmetric: zooming in then out returns
    // exactly where you started.
    return { kind: "zoom", factor: Math.exp(-dy * 0.0015) };
  }

  // Shift swaps the axis, the long-standing convention for horizontal scroll
  // on a device that only has a vertical wheel.
  if (e.shiftKey && dx === 0) return { kind: "pan", dx: dy, dy: 0 };

  return { kind: "pan", dx, dy };
}

// -----
// Stylus buttons
// -----

/**
 * Bits in PointerEvent.buttons that mean "the user wants to erase with this
 * stylus", per the Pointer Events spec:
 * 2 – barrel (side) button held
 * 32 – the eraser end of the pen is in contact
 *
 * Note on the Samsung S Pen specifically: its BLE *remote* button (the one used
 * for Air Actions – single click, double click, gestures) is NOT delivered to
 * web content at all. It is only readable through the native S Pen Remote SDK,
 * so a double-click-for-undo cannot be implemented from inside a WebView. The
 * barrel button and the eraser end below ARE delivered, and hold-to-erase is
 * the behaviour OneNote and Samsung Notes both use.
 */
export const STYLUS_BARREL_BUTTON = 2;
export const STYLUS_ERASER_TIP = 32;

export interface StylusButtonState {
  pointerType: string;
  buttons: number;
}

/** True when a pen event carries the barrel button or the eraser end. */
export function isStylusEraseRequest(e: StylusButtonState): boolean {
  if (e.pointerType !== "pen") return false;
  return (e.buttons & STYLUS_BARREL_BUTTON) !== 0 || (e.buttons & STYLUS_ERASER_TIP) !== 0;
}

/**
 * What a pen event should do to the borrowed-eraser state.
 *
 * Keyed on the BUTTON, not on the nib. Holding the barrel button while merely
 * hovering has to arm the eraser already, and releasing it has to hand the pen
 * back at once – even with the nib still on the glass. Driving this from
 * pointerdown/pointerup alone, as the first version did, meant the eraser only
 * appeared once you touched the paper and you stayed stuck in it until you
 * lifted off.

```

```

*
* `borrowed` is whether the eraser is currently on loan, so the caller gets an
* edge rather than a level and never re-enters the eraser it is already in
* (which would overwrite the tool being held for you).
*/
export type StylusEraseAction = "engage" | "release" | "none";

export function stylusEraseTransition(
  borrowed: boolean,
  e: StylusButtonState
): StylusEraseAction {
  // A finger or a mouse says nothing about the pen's button, and must never
  // release an eraser the pen is holding.
  if (e.pointerType !== "pen") return "none";
  const wants = isStylusEraseRequest(e);
  if (wants && !borrowed) return "engage";
  if (!wants && borrowed) return "release";
  return "none";
}

=====
// === END FILE: src/lib/whiteboard/input.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/links.ts ===
// File 106 of 194 | Lines: 725 | Size: 22114 bytes
=====

/**
 * Wikilinks & Knowledge Graph Intelligence (v3 – Obsidian Canvas Grade)
 * =====
 * Handles:
 * - Parsing [[WikiLinks]], [[Board|Alias]], and #tags
 * - Bi-directional backlink aggregation across all boards
 * - 2D Knowledge Graph data generation with typed semantic edges
 * - Autocomplete search for WikiLinks & Connections
 */

import type {
  Board,
  TextElement,
  StickyElement,
  ConnectionElement,
  WhiteboardElement,
} from "../types";
import { getLinkTypeColor, getLinkTypeLabel } from "../types";
import { allElements } from "../notebook";

// =====
// Types
// =====

export interface Wikilink {
  target: string;
  display: string;
  start: number;
  end: number;
}

export interface Tag {
  name: string;
  start: number;
  end: number;
}

export interface ParsedText {
  links: Wikilink[];
  tags: Tag[];
}

export interface TextSegment {
  type: "text" | "link" | "tag";
  text: string;
  value?: string; // target board name or tag name
}

```

```
// =====
// Parsing
// =====

const WIKILINK_REGEX = /\[\[([^\]]+)\](?:\|([^\]]*))?\]\]/g;
// Obsidian semantics: only the first "|segment" is the display alias
const TAG_REGEX = /(?:^|\s)#[a-zA-Z][a-zA-Z0-9_-]*/g;

export function parseText(text: string): ParsedText {
  const links: Wikilink[] = [];
  const tags: Tag[] = [];

  if (!text) return { links, tags };

  // Find wikilinks
  let match: RegExpExecArray | null;
  WIKILINK_REGEX.lastIndex = 0;
  while ((match = WIKILINK_REGEX.exec(text)) !== null) {
    const target = match[1].trim();
    if (!target) continue; // "[[ ]]" / "[[|alias]]" have no target – skip
    // "[[Target]]" (empty alias) displays the target itself
    const alias = match[2]?.split("|")[0]?.trim();
    links.push({
      target,
      display: alias || target,
      start: match.index,
      end: match.index + match[0].length,
    });
  }

  // Find tags
  TAG_REGEX.lastIndex = 0;
  while ((match = TAG_REGEX.exec(text)) !== null) {
    const tagName = match[1].trim();
    const insideLink = links.some(
      (l) => match!.index >= l.start && match!.index < l.end
    );
    if (!insideLink) {
      tags.push({
        name: tagName,
        start: match.index,
        end: match.index + match[0].length,
      });
    }
  }

  return { links, tags };
}

export function segmentText(text: string): TextSegment[] {
  if (!text) return [];
  const { links, tags } = parseText(text);

  interface Token {
    type: "link" | "tag";
    start: number;
    end: number;
    text: string;
    value: string;
  }

  const tokens: Token[] = [
    ...links.map((l) => ({
      type: "link" as const,
      start: l.start,
      end: l.end,
      text: l.display,
      value: l.target,
    })),
    ...tags.map((t) => ({
      type: "tag" as const,
      start: t.start,
      end: t.end,
      text: `#${t.name}`,
      value: t.name,
    })),
  ].sort((a, b) => a.start - b.start);
}
```

```

const segments: TextSegment[] = [];
let cursor = 0;

for (const token of tokens) {
  if (token.start > cursor) {
    segments.push({
      type: "text",
      text: text.slice(cursor, token.start),
    });
  }
  segments.push({
    type: token.type,
    text: token.text,
    value: token.value,
  });
  cursor = token.end;
}

if (cursor < text.length) {
  segments.push({
    type: "text",
    text: text.slice(cursor),
  });
}

return segments;
}

export function getBoardLinks(board: Board): string[] {
  const targets = new Set<string>();
  // allElements, not board.elements: a notebook keeps its content on its
  // pages and leaves the canvas field empty, so links written in a notebook
  // were invisible to backlinks, tags and the graph.
  for (const el of allElements(board)) {
    if (el.type === "text" || el.type === "sticky") {
      const text = (el as TextElement | StickyElement).text || "";
      const parsed = parseText(text);
      for (const link of parsed.links) {
        targets.add(link.target.toLowerCase());
      }
    }
  }
  return Array.from(targets);
}

export function getBoardTags(board: Board): string[] {
  const tags = new Set<string>();
  if (board.tags) {
    board.tags.forEach((t) => tags.add(t.toLowerCase()));
  }
  {
    for (const el of allElements(board)) {
      if (el.type === "text" || el.type === "sticky") {
        const text = (el as TextElement | StickyElement).text || "";
        const parsed = parseText(text);
        for (const tag of parsed.tags) {
          tags.add(tag.name.toLowerCase());
        }
      }
    }
  }
  return Array.from(tags);
}

// =====
// Backlinks Aggregator
// =====

export interface BacklinkEntry {
  sourceBoardId: string;
  sourceBoardName: string;
  elementId: string;
  target: string;
  snippet?: string;
  relationship?: string;
  color?: string;
}

```

```

export function getBacklinks(
  boards: Board[],
  targetBoardName: string,
  targetBoardId?: string,
  customLinkTypes: Array<{ id: string; name: string; color: string }> = []
): BacklinkEntry[] {
  const targetName = targetBoardName.toLowerCase();
  const backlinks: BacklinkEntry[] = [];

  for (const board of boards) {
    for (const el of allElements(board)) {
      // 1. Text & Sticky note wikilinks
      if (el.type === "text" || el.type === "sticky") {
        const text = (el as TextElement | StickyElement).text || "";
        const parsed = parseText(text);
        for (const link of parsed.links) {
          if (link.target.toLowerCase() === targetName) {
            backlinks.push({
              sourceBoardId: board.id,
              sourceBoardName: board.name,
              elementId: el.id,
              target: link.target,
              snippet: text.slice(0, 80),
              relationship: "WikiLink",
              color: "#6366f1",
            });
          }
        }
      }
      // 2. Direct semantic connection arrows
      if (el.type === "connection") {
        const conn = el as ConnectionElement;
        if (targetBoardId && (conn.target.boardId === targetBoardId || conn.source.boardId === targetBoardId)) {
          backlinks.push({
            sourceBoardId: board.id,
            sourceBoardName: board.name,
            elementId: el.id,
            target: targetBoardName,
            relationship: getLinkTypeLabel(conn.linkType, customLinkTypes),
            color: getLinkTypeColor(conn.linkType, customLinkTypes),
          });
        }
      }
    }
  }

  return backlinks;
}

// =====
// Knowledge Graph Engine
// =====

/**
 * A graph vertex is not always a whole board. When a single text box (or
 * sticky, image, shape...) is what actually carries the link, it earns its own
 * vertex, so the graph shows WHICH note points at the target rather than
 * collapsing every link on a board into one dot.
 */
export type GraphNodeKind = "board" | "folder" | "element" | "tag";

export interface GraphNode {
  id: string;
  name: string;
  outDegree: number;
  inDegree: number;
  orphan: boolean;
  starred: boolean;
  tagCount: number;
  folderId: string | null;
  color?: string;
  kind?: GraphNodeKind;
  /** For element vertices: where to navigate, and what shape to draw. */
  boardId?: string;
  elementId?: string;
  elementType?: string;
  snippet?: string;
}

```

```

    tags?: string[];
    folderName?: string;
  }

/**
 * Element vertex ids are namespaced. Element ids are nanoid and would not
 * realistically collide with board ids, but an accidental collision would make
 * a click navigate to the wrong thing, and the prefix also makes an id
 * self-describing in a debug dump.
 */
export function elementNodeId(boardId: string, elementId: string): string {
  return `el:${boardId}:${elementId}`;
}

export function parseElementNodeId(
  id: string
): { boardId: string; elementId: string } | null {
  if (!id.startsWith("el:")) return null;
  const rest = id.slice(3);
  const cut = rest.indexOf(":");
  if (cut < 1) return null;
  return { boardId: rest.slice(0, cut), elementId: rest.slice(cut + 1) };
}

/** A short, human label for an element vertex. */
function elementLabel(el: WhiteboardElement): string {
  if (el.type === "text" || el.type === "sticky") {
    const raw = ((el as TextElement | StickyElement).text || "").trim();
    // Show what the note SAYS, with link markup rendered as its display text -
    // "[[Physics|see here]]" reads as "see here", not as markup.
    const plain = segmentText(raw)
      .map((seg) => seg.text)
      .join("")
      .replace(/[#>~]/g, "")
      .replace(/\\s+/g, " ")
      .trim();
    if (plain) return plain.length > 28 ? plain.slice(0, 27) + "..." : plain;
  }
  if (el.type === "image") return "Image";
  return el.type.charAt(0).toUpperCase() + el.type.slice(1);
}

export interface GraphEdge {
  id: string;
  source: string;
  target: string;
  label: string;
  linkType?: string;
  color?: string;
}

export interface GraphData {
  nodes: GraphNode[];
  edges: GraphEdge[];
}

export interface BuildGraphOptions {
  includeTags?: boolean;
  includeElements?: boolean;
}

export function buildGraph(
  boards: Board[],
  customLinkTypes: Array<{ id: string; name: string; color: string }> = [],
  folders: Array<{ id: string; name: string }> = [],
  options?: BuildGraphOptions
): GraphData {
  const nodes: GraphNode[] = [];
  const edges: GraphEdge[] = [];
  const boardNameMap = new Map<string, Board>();

  for (const board of boards) {
    boardNameMap.set(board.name.toLowerCase(), board);
  }

  const folderMap = new Map<string, string>();
  for (const f of folders) {
    folderMap.set(f.id, f.name);
  }

```

```

}

// Every id the graph can draw a node for. The Link Manager stores the REAL
// endpoint (a board id OR a folder id) in `elementId`; `boardId` is only the
// board the connection is filed under. Resolving through this set is what
// lets folder endpoints reach the graph at all.
const graphNodeIds = new Set<string>([
  ...boards.map((b) => b.id),
  ...folders.map((f) => f.id),
]);

const inDegrees = new Map<string, number>();
for (const board of boards) {
  inDegrees.set(board.id, 0);
}

// Element vertices: any element that actually carries a link gets its own
// node, so the graph answers "which node points at this?" instead of only
// "which board". Elements with no links stay out – a vertex per shape would
// bury the structure under noise.
const elementNodes = new Map<string, GraphNode>();
const addElementNode = (board: Board, el: WhiteboardElement): string => {
  const id = elementNodeId(board.id, el.id);
  if (!elementNodes.has(id)) {
    const rawText = (el.type === "text" || el.type === "sticky")
      ? ((el as TextElement | StickyElement).text || "").trim()
      : "";
    const snippet = rawText ? (rawText.length > 120 ? rawText.slice(0, 117) + "..." : rawText) : undefined;
    const parsedEl = rawText ? parseText(rawText) : { links: [], tags: [] };
    const elTags = parsedEl.tags.map((t) => t.name);

    elementNodes.set(id, {
      id,
      name: elementLabel(el),
      outDegree: 0,
      inDegree: 0,
      orphan: false,
      starred: false,
      tagCount: elTags.length,
      folderId: board.folderId,
      folderName: board.folderId ? folderMap.get(board.folderId) : undefined,
      kind: "element",
      boardId: board.id,
      elementId: el.id,
      elementType: el.type,
      snippet,
      tags: elTags,
    });
    inDegrees.set(id, inDegrees.get(id) || 0);
  }
  return id;
};

// 1. Edges from WikiLinks – sourced at the ELEMENT that contains the link.
for (const board of boards) {
  for (const el of allElements(board)) {
    if (el.type !== "text" && el.type !== "sticky") continue;
    const text = (el as TextElement | StickyElement).text || "";
    const { links } = parseText(text);
    for (const link of links) {
      const targetBoard = boardNameMap.get(link.target.toLowerCase());
      if (!targetBoard || targetBoard.id === board.id) continue;

      const sourceId = addElementNode(board, el);
      const edgeId = `wl-${el.id}-${targetBoard.id}-${link.target.toLowerCase()}`;
      if (edges.some((e) => e.id === edgeId)) continue;
      edges.push({
        id: edgeId,
        source: sourceId,
        target: targetBoard.id,
        label: link.display,
        linkType: "wikilink",
        color: "#6366f1",
      });
      inDegrees.set(targetBoard.id, (inDegrees.get(targetBoard.id) || 0) + 1);
    }
  }
}

```

```

// 2. Edges from connection arrows.
for (const board of boards) {
  const docElements = allElements(board);
  const byId = new Map(docElements.map((e) => [e.id, e]));
  for (const el of docElements) {
    if (el.type !== "connection") continue;
    const conn = el as ConnectionElement;

    const endpointNode = (ep: { boardId: string; elementId: string }): string | null => {
      if (ep.elementId && graphNodeIds.has(ep.elementId)) return ep.elementId;
      const target = byId.get(ep.elementId);
      if (target && target.type !== "connection") return addElementNode(board, target);
      return ep.boardId || null;
    };

    const sNode = endpointNode(conn.source);
    const tNode = endpointNode(conn.target);
    if (!sNode || !tNode || sNode === tNode) continue;

    const edgeId = `conn-${conn.id}`;
    if (edges.some((e) => e.id === edgeId)) continue;
    edges.push({
      id: edgeId,
      source: sNode,
      target: tNode,
      label: conn.label || getLinkTypeLabel(conn.linkType, customLinkTypes),
      linkType: conn.linkType,
      color: getLinkTypeColor(conn.linkType, customLinkTypes),
    });
    inDegrees.set(tNode, (inDegrees.get(tNode) || 0) + 1);
  }
}

// 3. Board vertices
for (const board of boards) {
  const outDegree = edges.filter((e) => e.source === board.id).length;
  const inDegree = inDegrees.get(board.id) || 0;
  let snippet: string | undefined;
  for (const el of allElements(board)) {
    if (el.type === "text" || el.type === "sticky") {
      const raw = ((el as TextElement | StickyElement).text || "").trim();
      if (raw) {
        snippet = raw.length > 120 ? raw.slice(0, 117) + "..." : raw;
        break;
      }
    }
  }

  nodes.push({
    id: board.id,
    name: board.name,
    outDegree,
    inDegree,
    orphan: outDegree === 0 && inDegree === 0,
    starred: !!board.starred,
    tagCount: (board.tags || []).length,
    folderId: board.folderId,
    folderName: board.folderId ? folderMap.get(board.folderId) : undefined,
    color: board.color || "#3b82f6",
    kind: "board",
    snippet,
    tags: board.tags || [],
  });
}

// 4. Element vertices
for (const node of elementNodes.values()) {
  node.outDegree = edges.filter((e) => e.source === node.id).length;
  node.inDegree = inDegrees.get(node.id) || 0;
  nodes.push(node);
  edges.push({
    id: `owns-${node.id}`,
    source: node.boardId!,
    target: node.id,
    label: "",
    linkType: "contains",
    color: "#9ca3af",
  });
}

```

```

    });
  }

// 5. Tag vertices (Obsidian Canvas Grade Tag Clustering)
if (options?.includeTags) {
  const tagToSources = new Map<string, Set<string>>();
  for (const board of boards) {
    for (const t of board.tags || []) {
      const norm = t.toLowerCase().replace(/^#/, "");
      if (!tagToSources.has(norm)) tagToSources.set(norm, new Set());
      tagToSources.get(norm)!.add(board.id);
    }
    for (const el of allElements(board)) {
      if (el.type === "text" || el.type === "sticky") {
        const parsed = parseText((el as TextElement | StickyElement).text || "");
        for (const t of parsed.tags) {
          const norm = t.name.toLowerCase().replace(/^#/, "");
          if (!tagToSources.has(norm)) tagToSources.set(norm, new Set());
          tagToSources.get(norm)!.add(board.id);
        }
      }
    }
  }
}

for (const [tagNorm, sourceIds] of tagToSources.entries()) {
  if (sourceIds.size === 0) continue;
  const tagNodeId = `tag:${tagNorm}`;
  nodes.push({
    id: tagNodeId,
    name: `#${tagNorm}`,
    outDegree: 0,
    inDegree: sourceIds.size,
    orphan: false,
    starred: false,
    tagCount: 0,
    folderId: null,
    color: "#10b981", // vibrant emerald
    kind: "tag",
  });

  for (const sourceId of sourceIds) {
    edges.push({
      id: `tag-edge-${sourceId}-${tagNorm}`,
      source: sourceId,
      target: tagNodeId,
      label: `#${tagNorm}`,
      linkType: "tag",
      color: "#10b981",
    });
  }
}

return { nodes, edges };
}

/**
 * Filter graph down to a neighborhood around a focus node up to a given hop depth.
 */
export function filterGraphNeighborhood(
  graph: GraphData,
  focusNodeId: string,
  depth: number = 1
): GraphData {
  if (!focusNodeId || depth <= 0) return graph;
  const includedNodeIds = new Set<string>([focusNodeId]);
  let frontier = new Set<string>([focusNodeId]);

  // A board and its note elements are one entity: seed all owned elements
  for (const edge of graph.edges) {
    if (edge.linkType === "contains") {
      if (edge.source === focusNodeId) {
        includedNodeIds.add(edge.target);
        frontier.add(edge.target);
      } else if (edge.target === focusNodeId) {
        includedNodeIds.add(edge.source);
        frontier.add(edge.source);
      }
    }
  }

```

```

    }
  }

  for (let d = 0; d < depth; d++) {
    const nextFrontier = new Set<string>();
    for (const edge of graph.edges) {
      if (frontier.has(edge.source) && !includedNodeIds.has(edge.target)) {
        includedNodeIds.add(edge.target);
        nextFrontier.add(edge.target);
        // Include elements owned by newly reached board
        for (const c of graph.edges) {
          if (c.linkType === "contains" && c.source === edge.target) {
            includedNodeIds.add(c.target);
            nextFrontier.add(c.target);
          }
        }
      }
    }
    if (frontier.has(edge.target) && !includedNodeIds.has(edge.source)) {
      includedNodeIds.add(edge.source);
      nextFrontier.add(edge.source);
      // Include board owning newly reached element
      for (const c of graph.edges) {
        if (c.linkType === "contains" && c.target === edge.source) {
          includedNodeIds.add(c.source);
          nextFrontier.add(c.source);
        }
      }
    }
  }
  frontier = nextFrontier;
}

const nodes = graph.nodes.filter((n) => includedNodeIds.has(n.id));
const edges = graph.edges.filter(
  (e) => includedNodeIds.has(e.source) && includedNodeIds.has(e.target)
);

return { nodes, edges };
}

/**
 * Local Board Elements Graph
 * Generates an interactive graph of all cards, sticky notes, shapes, and connections on a specific board.
 */
export function buildBoardLocalGraph(
  board: Board | null | undefined,
  customLinkTypes: Array<{ id: string; name: string; color: string }> = []
): GraphData {
  const docElements = allElements(board);
  if (!board || docElements.length === 0) {
    return { nodes: [], edges: [] };
  }

  const nodes: GraphNode[] = [];
  const edges: GraphEdge[] = [];
  const inDegrees = new Map<string, number>();

  const elementMap = new Map<string, string>(); // elementId -> displayName

  // 1. Create nodes for all non-connection elements
  for (const el of docElements) {
    if (el.type === "connection") continue;

    let name = "Element";
    let color = "#3b82f6";

    if (el.type === "text") {
      const text = (el as TextElement).text || "Text Note";
      name = text.split("\n")[0].slice(0, 30) || "Text Note";
      color = (el as TextElement).color || "#6366f1";
    } else if (el.type === "sticky") {
      const text = (el as StickyElement).text || "Sticky Note";
      name = text.split("\n")[0].slice(0, 30) || "Sticky Note";
      color = "#eab308";
    } else if (el.type === "image") {
      name = "Image";
      color = "#ec4899";
    } else {

```

```

    name = el.type.charAt(0).toUpperCase() + el.type.slice(1);
    color = (el as { strokeColor?: string }).strokeColor || "#10b981";
  }

  elementMap.set(el.id, name);
  inDegrees.set(el.id, 0);

  nodes.push({
    id: el.id,
    name,
    outDegree: 0,
    inDegree: 0,
    orphan: true,
    starred: false,
    tagCount: 0,
    folderId: null,
    color,
  });
}

// 2. Build edges from connections
for (const el of docElements) {
  if (el.type === "connection") {
    const conn = el as ConnectionElement;
    const sId = conn.source.elementId;
    const tId = conn.target.elementId;

    if (sId && tId && elementMap.has(sId) && elementMap.has(tId)) {
      const edgeId = `conn-${conn.id}`;
      edges.push({
        id: edgeId,
        source: sId,
        target: tId,
        label: conn.label || getLinkTypeLabel(conn.linkType, customLinkTypes),
        linkType: conn.linkType,
        color: getLinkTypeColor(conn.linkType, customLinkTypes),
      });
      inDegrees.set(tId, (inDegrees.get(tId) || 0) + 1);
    }
  }
}

// 3. Update degree and orphan status
for (const node of nodes) {
  const outDegree = edges.filter((e) => e.source === node.id).length;
  const inDegree = inDegrees.get(node.id) || 0;
  node.outDegree = outDegree;
  node.inDegree = inDegree;
  node.orphan = outDegree === 0 && inDegree === 0;
}

return { nodes, edges };
}

```

```

=====
// === END FILE: src/lib/whiteboard/links.ts ===
=====

```

```

=====
// === START FILE: src/lib/whiteboard/notebook.ts ===
// File 107 of 194 | Lines: 1034 | Size: 37369 bytes
=====

```

```

/**
 * Notebook pages
 * =====
 * Pure helpers for paged documents. No React, no store – so the page model can
 * be tested on its own.
 *
 * The one rule that keeps notebooks and canvases from drifting apart: nothing
 * outside this file may read or write a board's element list directly. Reads go
 * through `getBoardElements`, writes through `mapBoardElements`. A canvas
 * resolves to `board.elements`; a notebook resolves to its active page. Every
 * existing tool then works on both without knowing which it is drawing on.
 */

```

```

import {
  BOARD_PATTERN_MAX_SIZE,
  BOARD_PATTERN_MIN_SIZE,
  DEFAULT_PAGE_SIZE,
  PAGE_SIZES,
  type Board,
  type BoardBackground,
  type BoardPattern,
  type ConnectionElement,
  type NotebookPage,
  type PageSizeId,
  type PageTemplate,
  type WhiteboardElement,
} from "./types";
import { GRID_SIZE } from "./grid";

/** Ids are generated here so the helpers stay usable outside the browser. */
function pageId(): string {
  return `pg_${Date.now().toString(36)}_${Math.random().toString(36).slice(2, 9)}`;
}

export function isNotebook(board: Board | null | undefined): boolean {
  return board?.kind === "notebook";
}

export function pageSizeOf(board: Board | null | undefined) {
  // A PDF-backed notebook carries the document's own page size, so a slide
  // deck is not letterboxed into A4.
  const custom = board?.customPageSize;
  if (custom && custom.width > 0 && custom.height > 0) {
    return { label: "PDF", width: custom.width, height: custom.height };
  }
  return PAGE_SIZES[board?.pageSize ?? DEFAULT_PAGE_SIZE];
}

/** Every PDF a notebook's pages reference, deduplicated. */
export function pdfDocIdsOf(board: Board | null | undefined): string[] {
  const ids = new Set<string>();
  for (const p of getPages(board)) if (p.pdf?.docId) ids.add(p.pdf.docId);
  return [...ids];
}

export function createPage(background?: BoardBackground): NotebookPage {
  const now = Date.now();
  return { id: pageId(), elements: [], background, createdAt: now, updatedAt: now };
}

/** A notebook's pages, tolerating a malformed or absent array. */
export function getPages(board: Board | null | undefined): NotebookPage[] {
  if (!board || !isNotebook(board)) return [];
  return Array.isArray(board.pages) ? board.pages : [];
}

/**
 * The page currently being edited. Falls back to the first page when
 * `activePageId` points at a page that no longer exists, so a stale id can
 * never leave the notebook with nothing to draw on.
 */
export function getActivePage(board: Board | null | undefined): NotebookPage | null {
  const pages = getPages(board);
  if (pages.length === 0) return null;
  return pages.find((p) => p.id === board?.activePageId) ?? pages[0];
}

export function getActivePageIndex(board: Board | null | undefined): number {
  const pages = getPages(board);
  if (pages.length === 0) return -1;
  const i = pages.findIndex((p) => p.id === board?.activePageId);
  return i === -1 ? 0 : i;
}

/** The live element list: the page's for a notebook, the board's for a canvas. */
export function getBoardElements(board: Board | null | undefined): WhiteboardElement[] {
  if (!board) return [];
  if (!isNotebook(board)) return board.elements ?? [];
  return getActivePage(board)?.elements ?? [];
}

```

```

/**
 * Applies `fn` to whichever element list is live, returning a new board.
 *
 * Every element mutation in the store funnels through this. Without a single
 * funnel, each of the store's mutation sites would need its own notebook
 * branch, and the one that got missed would silently write a page's strokes
 * onto the canvas field where nothing would ever render them again.
 */
export function mapBoardElements(
  board: Board,
  fn: (elements: WhiteboardElement[]) => WhiteboardElement[]
): Board {
  const now = Date.now();

  if (!isNotebook(board)) {
    return { ...board, elements: fn(board.elements ?? []), updatedAt: now };
  }

  const pages = getPages(board);
  if (pages.length === 0) {
    // A notebook with no pages is malformed; give it one rather than dropping
    // whatever the caller was trying to add.
    const page = createPage();
    return {
      ...board,
      pages: [{ ...page, elements: fn([]), updatedAt: now }],
      activePageId: page.id,
      updatedAt: now,
    };
  }

  const activeId = (getActivePage(board) as NotebookPage).id;
  return {
    ...board,
    pages: pages.map((p) =>
      p.id === activeId ? { ...p, elements: fn(p.elements ?? []), updatedAt: now } : p
    ),
    activePageId: activeId,
    updatedAt: now,
  };
}

/**
 * Like `mapBoardElements`, but applies to EVERY page rather than the active
 * one. Used for document-wide sweeps such as stripping connections that point
 * at a deleted board: a dangling link on page 7 is just as broken as one on
 * the page you happen to be looking at.
 */
export function mapAllBoardElements(
  board: Board,
  fn: (elements: WhiteboardElement[]) => WhiteboardElement[]
): Board {
  const now = Date.now();

  if (!isNotebook(board)) {
    return { ...board, elements: fn(board.elements ?? []), updatedAt: now };
  }

  return {
    ...board,
    pages: getPages(board).map((p) => ({ ...p, elements: fn(p.elements ?? []), updatedAt: now })),
    updatedAt: now,
  };
}

/**
 * Deep-copies a document for "duplicate". `remap` rewrites one element — the
 * caller owns id generation, because connections also have to have their
 * endpoints repointed at the copy. Pages are copied too, so duplicating a
 * notebook does not hand two documents the same page objects.
 */
export function cloneBoardContent(
  board: Board,
  remap: (el: WhiteboardElement) => WhiteboardElement
): Pick<Board, "elements" | "pages" | "activePageId"> {
  if (!isNotebook(board)) {
    return { elements: (board.elements ?? []).map(remap) };
  }
}

```

```

const pages = getPages(board).map((p) => ({
  ...p,
  id: pageId(),
  elements: (p.elements ?? []).map(remap),
}));
const activeIndex = Math.max(0, getActivePageIndex(board));

return {
  elements: [],
  pages,
  activePageId: pages[activeIndex]?.id ?? pages[0]?.id,
};
}

// -----
// Continuous scroll layout
// -----
//
// Pages are stacked down the canvas with a gap between them, so a notebook
// scrolls the way GoodNotes and Notability do rather than making you click a
// next button. Page `i` occupies world Y from `pageOffsetY(i)` to
// `pageOffsetY(i) + height`; the gap between sheets is dead space.
//
// Each page's elements stay stored in PAGE-LOCAL coordinates. Only rendering
// and hit-testing add the offset. Storing world coordinates instead would
// mean every insert, delete or reorder of a page had to rewrite the
// coordinates of every element below it.

/** Dead space between sheets, in world units. Small enough to waste no screen,
 * large enough to read as a separator and to host the gap "+" button.
 * Pinned at 14 by `notebook_camera.test.ts` ("the join is a seam, not a
 * band"): at the zoom floor it must render under 12 screen px on phone,
 * tablet and desktop alike. */
export const PAGE_GAP = 14;

export function pageOffsetY(index: number, pageHeight: number): number {
  return index * (pageHeight + PAGE_GAP);
}

/** Total world height of the whole stack, excluding the trailing gap. */
export function notebookContentHeight(pageCount: number, pageHeight: number): number {
  if (pageCount <= 0) return 0;
  return pageCount * pageHeight + (pageCount - 1) * PAGE_GAP;
}

/**
 * Which page a world Y belongs to.
 *
 * A point in the gap between two sheets belongs to the page above it, so a
 * stroke started just off the bottom edge lands on the page you were writing
 * on rather than jumping to the next one. Points above the first page and
 * below the last clamp to those pages.
 */
export function pageIndexAtY(y: number, pageHeight: number, pageCount: number): number {
  if (pageCount <= 0) return -1;
  if (y < 0) return 0;
  const stride = pageHeight + PAGE_GAP;
  const raw = Math.floor(y / stride);
  return Math.max(0, Math.min(pageCount - 1, raw));
}

/** World Y -> the same point measured from the top of its own page. */
export function toPageLocalY(y: number, index: number, pageHeight: number): number {
  return y - pageOffsetY(index, pageHeight);
}

/** True when a world point falls on the sheet itself rather than in a gap. */
export function isOnSheet(
  x: number,
  y: number,
  index: number,
  pageWidth: number,
  pageHeight: number
): boolean {
  const localY = toPageLocalY(y, index, pageHeight);
  return x >= 0 && x <= pageWidth && localY >= 0 && localY <= pageHeight;
}

```

```
// -----
// Keeping ink on the paper
// -----
//
// A notebook is a stack of sheets, not an infinite canvas, and it is the only
// thing on screen that looks like paper. Before this, a notebook was drawn on
// the SAME unbounded world as a whiteboard: you could write a metre to the
// right of the page, or into the dead gap between two sheets, and the ink
// stayed there – visible, saved, and impossible to see again once you scrolled,
// because nothing outside the sheet is ever printed or exported. GoodNotes,
// Notability and OneNote all refuse that, and so do we now.
//
// The ACTIVE page always sits at world origin: re-anchoring slides the camera
// by exactly one stride whenever the next sheet takes over, so the page you are
// writing on is always the rectangle (0,0)-(width,height). That is what makes
// these two functions as simple as they look.

/**
 * How far inside the sheet edge ink is allowed to land.
 *
 * Not zero: a stroke has width, and a point exactly on the boundary paints a
 * half-width smear over the page border. Two world units keeps the nib inside
 * the paper at every zoom without being visible as a margin.
 */
export const SHEET_INK_INSET = 2;

/** True when a world point is on the ACTIVE sheet (which sits at the origin). */
export function isOnActiveSheet(
  x: number,
  y: number,
  pageWidth: number,
  pageHeight: number,
  margin = 0
): boolean {
  return (
    x >= -margin && x <= pageWidth + margin && y >= -margin && y <= pageHeight + margin
  );
}

/**
 * Pulls a world point back onto the active sheet.
 *
 * Used for every point that becomes content – stroke samples, shape corners,
 * where a text box lands. Clamping rather than dropping the point matters for
 * strokes: if you run off the edge mid-word the line stops at the margin
 * instead of tearing a hole in the middle of the stroke.
 */
export function clampToSheet(
  x: number,
  y: number,
  pageWidth: number,
  pageHeight: number,
  inset = SHEET_INK_INSET
): { x: number; y: number } {
  const lo = inset;
  const hiX = Math.max(lo, pageWidth - inset);
  const hiY = Math.max(lo, pageHeight - inset);
  return {
    x: Math.min(hiX, Math.max(lo, x)),
    y: Math.min(hiY, Math.max(lo, y)),
  };
}

/**
 * Clamps a whole element's bounding box onto the sheet by TRANSLATION only.
 *
 * Moving is the right correction for a drag: resizing an element because it
 * reached the edge would silently reshape the user's drawing. An element too
 * big for the page is pinned to the top-left rather than squashed, so it stays
 * whole and reachable.
 */
export function clampBoxToSheet(
  x: number,
  y: number,
  w: number,
  h: number,
  pageWidth: number,
```

```

    pageHeight: number
  ): { x: number; y: number } {
    const maxX = pageWidth - w;
    const maxY = pageHeight - h;
    return {
      x: maxX < 0 ? 0 : Math.min(maxX, Math.max(0, x)),
      y: maxY < 0 ? 0 : Math.min(maxY, Math.max(0, y)),
    };
  }
}

/**
 * The pages whose sheets intersect a visible world Y range, as an index range.
 * A 200-page notebook must not render 200 sheets and every stroke on them.
 */
export function visiblePageRange(
  topY: number,
  bottomY: number,
  pageHeight: number,
  pageCount: number,
  overscan = 1
): { start: number; end: number } {
  if (pageCount <= 0) return { start: 0, end: -1 };
  const stride = pageHeight + PAGE_GAP;
  const start = Math.max(0, Math.floor(topY / stride) - overscan);
  const end = Math.min(pageCount - 1, Math.floor(bottomY / stride) + overscan);
  return { start, end: Math.max(start, end) };
}

/**
 * Turns a 1-based, inclusive page range a person typed into page INDICES that
 * actually exist.
 *
 * People type ranges loosely - "5 to 2" when they mean 2 to 5, "3 to 999" for
 * "to the end", an empty box while they are still typing. Every one of those
 * must come back as a sensible range rather than an empty export or a throw,
 * so this is the single place that decides what a range means. It returns the
 * clamped `from`/`to` as well, so the UI can show what it actually used.
 */
export function resolvePageRange(
  pageCount: number,
  from: number,
  to: number
): { from: number; to: number; indices: number[] } {
  if (pageCount <= 0) return { from: 0, to: 0, indices: [] };
  const clamp = (n: number, fallback: number) => {
    const v = Math.round(Number.isFinite(n) ? n : fallback);
    return Math.min(Math.max(1, v), pageCount);
  };
  let lo = clamp(from, 1);
  let hi = clamp(to, pageCount);
  if (lo > hi) [lo, hi] = [hi, lo];
  const indices: number[] = [];
  for (let i = lo; i <= hi; i++) indices.push(i - 1);
  return { from: lo, to: hi, indices };
}

/**
 * Reads a page selection the way people write one: "2-5, 8", "1,3", "4-",
 * "-6", "7". Returns 1-based page numbers, sorted, deduplicated and clamped
 * to what exists.
 *
 * Every note-taking app that offers this accepts the same shorthand, so the
 * parser is deliberately forgiving: stray spaces, en-dashes pasted from a PDF,
 * trailing commas and a half-typed "2-" all mean something sensible rather
 * than an error. An empty or entirely unparseable string returns no pages, and
 * the caller decides whether that means "nothing" or "fall back to all".
 */
export function parsePageSelection(input: string, pageCount: number): number[] {
  if (pageCount <= 0) return [];
  const clamp = (n: number) => Math.min(Math.max(1, n), pageCount);
  const chosen = new Set<number>();

  // En-dash and em-dash arrive from anything pasted out of a document, and
  // people space their ranges out ("4 - 2"), so spaces around a dash are
  // closed up BEFORE splitting - otherwise that reads as three separate pages.
  const normalised = input.replace(/[\u2012-\u2015]/g, "-").replace(/\s*-\s*/g, "-");
  for (const rawPart of normalised.split(/[,;\s]+/)) {
    const part = rawPart.trim();

```

```

    if (!part) continue;

    const range = /^(\\d*)-(\\d*)$/.exec(part);
    if (range) {
        const [, a, b] = range;
        if (!a && !b) continue; // a lone "-" says nothing
        let lo = a ? clamp(parseInt(a, 10)) : 1;
        let hi = b ? clamp(parseInt(b, 10)) : pageCount;
        if (lo > hi) [lo, hi] = [hi, lo];
        for (let i = lo; i <= hi; i++) chosen.add(i);
        continue;
    }

    if (/^\\d+$/.test(part)) chosen.add(clamp(parseInt(part, 10)));
}

return [...chosen].sort((x, y) => x - y);
}

/**
 * Describes a set of page numbers the way it would be typed: [1,2,3,5] reads
 * "1-3, 5". Used for filenames and for showing back what was understood.
 */
export function formatPageSelection(pages: number[]): string {
    const sorted = [...new Set(pages)].sort((a, b) => a - b);
    const parts: string[] = [];
    let i = 0;
    while (i < sorted.length) {
        const start = sorted[i];
        let end = start;
        while (i + 1 < sorted.length && sorted[i + 1] === end + 1) { end = sorted[++i]; }
        parts.push(start === end ? `${start}` : `${start}-${end}`);
        i++;
    }
    return parts.join(", ");
}

// -----
// Page operations
// -----

/**
 * Fresh copies of the pages at `indices`, ready to be inserted into
 * `targetBoardId`.
 *
 * Every element gets a new id, because the same element existing twice under
 * one id makes selection, deletion and undo act on both at once. Connections
 * are remapped to the new ids; one whose other end was NOT copied is dropped
 * rather than carried over pointing at nothing, which would draw a line to the
 * origin of the page.
 *
 * A page that shows a PDF keeps its `{ docId, pageNumber }` reference. That is
 * safe without copying any bytes: the PDF garbage collector scans every
 * board's pages for referenced docIds, so the import stays alive exactly as
 * long as some page still points at it.
 */
export function copyPagesFrom(
    source: Board,
    indices: number[],
    targetBoardId: string,
    newId: () => string
): NotebookPage[] {
    const pages = getPages(source);
    const chosen = indices.map((i) => pages[i]).filter(Boolean);

    const idMap = new Map<string, string>();
    for (const page of chosen) {
        for (const el of page.elements ?? []) idMap.set(el.id, newId());
    }

    const remap = (el: WhiteboardElement): WhiteboardElement | null => {
        const id = idMap.get(el.id) ?? newId();
        if (el.type === "connection") {
            const conn = el as ConnectionElement;
            const ends = [conn.source, conn.target];
            for (const end of ends) {
                // An end on THIS board that was not copied has nothing to attach to.
                if (end.boardId === source.id && !idMap.has(end.elementId)) return null;
            }
        }
    }

```

```

    }
    return {
      ...conn,
      id,
      source: {
        ...conn.source,
        boardId: conn.source.boardId === source.id ? targetBoardId : conn.source.boardId,
        elementId: idMap.get(conn.source.elementId) ?? conn.source.elementId,
      },
      target: {
        ...conn.target,
        boardId: conn.target.boardId === source.id ? targetBoardId : conn.target.boardId,
        elementId: idMap.get(conn.target.elementId) ?? conn.target.elementId,
      },
      updatedAt: Date.now(),
    } as WhiteboardElement;
  }
  return { ...el, id, updatedAt: Date.now() };
};

const now = Date.now();
return chosen.map((page) => ({
  ...page,
  id: pageId(),
  elements: (page.elements ?? [])
    .map(remap)
    .filter((el) => el is WhiteboardElement => el !== null),
  createdAt: now,
  updatedAt: now,
})));
}

/** A notebook page that shows one page of an imported PDF. */
export function pageFromPdf(docId: string, pageNumber: number): NotebookPage {
  const now = Date.now();
  return {
    id: pageId(),
    elements: [],
    pdf: { docId, pageNumber },
    // A PDF page IS the paper; drawing dots over a lecture slide is noise.
    background: { pattern: "plain" },
    createdAt: now,
    updatedAt: now,
  };
}

/**
 * Inserts pages at `atIndex` (0-based, clamped), selecting the first one.
 *
 * `atIndex` is the position the first new page ENDS UP at, so 0 means "before
 * everything" and pageCount means "at the end" - the two answers people
 * actually want when they say where to put something.
 */
export function insertPages(board: Board, incoming: NotebookPage[], atIndex: number): Board {
  if (!isNotebook(board) || incoming.length === 0) return board;
  const pages = getPages(board);
  const at = Math.min(Math.max(0, Math.round(atIndex)), pages.length);
  const next = [...pages.slice(0, at), ...incoming, ...pages.slice(at)];
  return {
    ...board,
    pages: next,
    activePageId: incoming[0].id,
    updatedAt: Date.now(),
  };
}

/** Inserts a blank page after `afterPageId` (or at the end) and selects it. */
export function addPage(board: Board, afterPageId?: string): Board {
  const pages = getPages(board);
  // A new page starts with the NOTEBOOK's paper (the Settings default), not
  // the neighbour's: per-page overrides stay exactly where you put them, and
  // a fresh sheet in a mixed notebook is predictable instead of a copy of
  // whatever it landed next to.
  const anchorIndex = afterPageId ? pages.findIndex((p) => p.id === afterPageId) : pages.length - 1;
  const page = createPage(board.background);
  const at = anchorIndex >= 0 ? anchorIndex + 1 : pages.length;

  return {

```

```

    ...board,
    pages: [...pages.slice(0, at), page, ...pages.slice(at)],
    activePageId: page.id,
    updatedAt: Date.now(),
  };
}

export function duplicatePage(board: Board, pageId_: string): Board {
  const pages = getPages(board);
  const index = pages.findIndex((p) => p.id === pageId_);
  if (index === -1) return board;

  const source = pages[index];
  const now = Date.now();
  const copy: NotebookPage = {
    ...source,
    id: pageId(),
    // Elements carry ids; cloning without fresh ones would give two pages
    // elements that selection and deletion treat as the same object.
    elements: source.elements.map((el) => ({
      ...el,
      id: `${el.id}_c${Math.random().toString(36).slice(2, 7)}`,
    })),
    createdAt: now,
    updatedAt: now,
  };

  return {
    ...board,
    pages: [...pages.slice(0, index + 1), copy, ...pages.slice(index + 1)],
    activePageId: copy.id,
    updatedAt: now,
  };
}

/** Deletes a page. A notebook always keeps at least one, so the last delete clears instead. */
export function deletePage(board: Board, pageId_: string): Board {
  const pages = getPages(board);
  const index = pages.findIndex((p) => p.id === pageId_);
  if (index === -1) return board;

  if (pages.length === 1) {
    const blank = createPage(pages[0].background);
    return { ...board, pages: [blank], activePageId: blank.id, updatedAt: Date.now() };
  }

  const remaining = pages.filter((p) => p.id !== pageId_);
  // Select the page that takes the deleted one's place, or the new last page.
  const next = remaining[Math.min(index, remaining.length - 1)];
  return { ...board, pages: remaining, activePageId: next.id, updatedAt: Date.now() };
}

/** Moves a page to `toIndex`, clamped into range. */
export function movePage(board: Board, pageId_: string, toIndex: number): Board {
  const pages = getPages(board);
  const from = pages.findIndex((p) => p.id === pageId_);
  if (from === -1) return board;

  const to = Math.max(0, Math.min(pages.length - 1, toIndex));
  if (to === from) return board;

  const next = [...pages];
  const [moved] = next.splice(from, 1);
  next.splice(to, 0, moved);
  return { ...board, pages: next, updatedAt: Date.now() };
}

/**
 * Which sheet is under a world point, in the ANCHORED frame (the active page
 * sits at y = 0 and page i at (i - activeIndex) * stride). Null in the gap
 * between sheets, above the first, below the last, or beside the paper.
 *
 * Needed because several sheets can be on screen at once when zoomed out, but
 * only the anchored one is live. Without this a pen landing on the sheet
 * above or below was treated as "off the page" and panned instead of writing.
 */
export function sheetIndexAt(
  wx: number,

```

```

wy: number,
activeIndex: number,
pageCount: number,
pageWidth: number,
pageHeight: number,
marginX = 0
): number | null {
  if (pageCount <= 0) return null;
  const stride = pageHeight + PAGE_GAP;
  const offset = Math.floor(wy / stride);
  const index = activeIndex + offset;
  if (index < 0 || index >= pageCount) return null;
  const localY = wy - offset * stride;
  if (localY > pageHeight) return null; // in the gap below this sheet
  if (wx < -marginX || wx > pageWidth + marginX) return null;
  return index;
}

/**
 * Inserts a blank page so that it ends up AT `index` and selects it.
 *
 * `0` is above the first page and `pages.length` is below the last – the
 * three places a "+" can appear between sheets. Like addPage, the new sheet
 * starts with the notebook default paper.
 */
export function insertPageAt(board: Board, index: number): Board {
  const pages = getPages(board);
  const at = Math.max(0, Math.min(pages.length, Math.round(index)));
  if (at === 0) return pages.length > 0 ? addPageBefore(board, pages[0].id) : addPage(board);
  return addPage(board, pages[at - 1].id);
}

/** Inserts a blank page BEFORE `pageId` and selects it. */
export function addPageBefore(board: Board, pageId_: string): Board {
  const pages = getPages(board);
  const index = pages.findIndex((p) => p.id === pageId_);
  if (index === -1) return addPage(board);

  const page = createPage(board.background);
  return {
    ...board,
    pages: [...pages.slice(0, index), page, ...pages.slice(index)],
    activePageId: page.id,
    updatedAt: Date.now(),
  };
}

// -----
// Bookmarks
// -----

export function togglePageBookmark(board: Board, pageId_: string): Board {
  const pages = getPages(board);
  if (!pages.some((p) => p.id === pageId_)) return board;
  return {
    ...board,
    pages: pages.map((p) =>
      p.id === pageId_ ? { ...p, bookmarked: !p.bookmarked, updatedAt: Date.now() } : p
    ),
    updatedAt: Date.now(),
  };
}

/** Starred pages with their real page numbers, for the Favourites list. */
export function bookmarkedPages(
  board: Board | null | undefined
): Array<{ page: NotebookPage; index: number }> {
  return getPages(board)
    .map((page, index) => ({ page, index }))
    .filter((e) => e.page.bookmarked === true);
}

// -----
// Templates
// -----

/** Captures a page as a reusable template. Ink is copied, not referenced. */
export function templateFromPage(

```

```

    page: NotebookPage,
    name: string,
    width: number,
    height: number
  ): PageTemplate {
    return {
      id: `tpl_${Date.now().toString(36)}_${Math.random().toString(36).slice(2, 7)}`,
      name: name.trim() || "Untitled template",
      createdAt: Date.now(),
      width,
      height,
      background: page.background,
      elements: (page.elements ?? []).map((el) => ({ ...el })),
    };
  }

  /**
   * A new page from a template.
   *
   * Every element gets a fresh id – reusing the template's would make two pages
   * built from it share identities, so selecting on one would select on the
   * other and deleting from one would delete from both.
   */
  export function pageFromTemplate(template: PageTemplate): NotebookPage {
    const now = Date.now();
    return {
      id: pageId(),
      background: template.background,
      elements: (template.elements ?? []).map((el) => ({
        ...el,
        id: `${el.id}_t${Math.random().toString(36).slice(2, 7)}`,
      })),
      createdAt: now,
      updatedAt: now,
    };
  }

  /** Inserts a template-built page after `afterPageId` (or at the end). */
  export function insertTemplatePage(
    board: Board,
    template: PageTemplate,
    afterPageId?: string
  ): Board {
    const pages = getPages(board);
    const page = pageFromTemplate(template);
    const anchor = afterPageId ? pages.findIndex((p) => p.id === afterPageId) : pages.length - 1;
    const at = anchor >= 0 ? anchor + 1 : pages.length;
    return {
      ...board,
      pages: [...pages.slice(0, at), page, ...pages.slice(at)],
      activePageId: page.id,
      updatedAt: Date.now(),
    };
  }

  export function setActivePage(board: Board, pageId_: string): Board {
    if (!getPages(board).some((p) => p.id === pageId_)) return board;
    return { ...board, activePageId: pageId_ };
  }

  /** Steps `delta` pages from the current one, stopping at the ends. */
  export function stepPage(board: Board, delta: number): Board {
    const pages = getPages(board);
    if (pages.length === 0) return board;
    const index = getActivePageIndex(board);
    const next = Math.max(0, Math.min(pages.length - 1, index + delta));
    return next === index ? board : { ...board, activePageId: pages[next].id };
  }

  /** Per-page paper, falling back to the notebook's own. */
  export function setPageBackground(
    board: Board,
    pageId_: string,
    patch: Partial<BoardBackground>
  ): Board {
    const pages = getPages(board);
    if (!pages.some((p) => p.id === pageId_)) return board;

```

```

return {
  ...board,
  pages: pages.map((p) =>
    p.id === pageId_
    ? {
      ...p,
      background: { ...(p.background ?? board.background ?? { pattern: "dots" }), ...patch },
      updatedAt: Date.now(),
    }
    : p
  ),
  updatedAt: Date.now(),
};
}

export function backgroundForPage(board: Board, page: NotebookPage | null): BoardBackground | undefined {
  return page?.background ?? board.background;
}

const VALID_PATTERNS: ReadonlySet<string> = new Set(["dots", "grid", "lines", "plain"]);

export interface SheetPaper {
  pattern: BoardPattern;
  /** Spacing in px, clamped to the sane range. */
  size: number;
  /** Raw stored colour (preset id or literal); undefined follows the theme. */
  color?: string;
}

/**
 * The paper one sheet is painted with: the page's own, falling back to the
 * notebook's. Every renderer (canvas live sheet, neighbour sheets, thumbnails,
 * export) must go through this – reading only the ACTIVE page's paper and
 * painting it on every sheet is exactly how a grid page "turned blank" the
 * moment you navigated to it while showing grid from the page above.
 */
export function paperForPage(
  board: Board | null | undefined,
  page: NotebookPage | null
): SheetPaper {
  const bg = board ? backgroundForPage(board, page) ?? board.background : undefined;
  const pattern: BoardPattern =
    bg && VALID_PATTERNS.has(bg.pattern) ? bg.pattern : "dots";
  const size = Math.min(
    BOARD_PATTERN_MAX_SIZE,
    Math.max(BOARD_PATTERN_MIN_SIZE, bg?.size ?? GRID_SIZE)
  );
  return { pattern, size, color: bg?.color };
}

/** Turns a fresh board into a notebook with one blank page. */
export function makeNotebook(board: Board, pageSize: PageSizeId = DEFAULT_PAGE_SIZE): Board {
  const first = createPage(board.background);
  // Anything already drawn on the canvas becomes the first page rather than
  // being stranded on a field the notebook view never reads.
  const seeded: NotebookPage =
    board.elements && board.elements.length > 0 ? { ...first, elements: board.elements } : first;

  return {
    ...board,
    kind: "notebook",
    pageSize,
    pages: [seeded],
    activePageId: seeded.id,
    elements: [],
    updatedAt: Date.now(),
  };
}

/** Every element in the document, for search, export and the knowledge graph. */
export function allElements(board: Board | null | undefined): WhiteboardElement[] {
  if (!board) return [];
  if (!isNotebook(board)) return board.elements ?? [];
  return getPages(board).flatMap((p) => p.elements ?? []);
}

// -----
// How much empty desk may surround the paper

```

```
// -----

/**
 * Screen pixels of bare desk allowed past the edge of the paper.
 *
 * Stated in SCREEN pixels on purpose. A bound in world units looks completely
 * different at each zoom - the same 400-unit slack is a thin strip at 200% and
 * most of the screen at 44% - and what the eye judges is the strip of grey it
 * can see, not the number behind it. A small strip is wanted: it is the handle
 * you grab to pan, and it shows where the sheet ends.
 */
export const DESK_MARGIN_PX = 80;

/** clamp(), or the midpoint when the interval is empty (content fits: centre it). */
function boundAxis(v: number, lo: number, hi: number): number {
  if (lo > hi) return (lo + hi) / 2;
  return Math.min(Math.max(v, lo), hi);
}

/**
 * Where a notebook's camera is allowed to sit.
 *
 * The camera is the world point at the centre of the viewport, so the visible
 * world band is `camera +/- half the viewport in world units`. The rule is
 * simply: no more than `DESK_MARGIN_PX` of desk beyond the paper on any side.
 *
 * Horizontally the paper is one sheet wide; vertically the "paper" is the whole
 * stack, measured in the anchored frame where the ACTIVE page starts at y = 0,
 * so the bound moves with the page you are on and never fights re-anchoring.
 *
 * When the paper is smaller than the viewport - a zoomed-out sheet, or a
 * one-page notebook on a tall screen - the interval is empty and the midpoint
 * centres the paper instead, which is the only position with equal margins.
 */
/**
 * How far out a notebook may be zoomed: until the whole sheet fits, and no
 * further.
 *
 * Capping the desk at DESK_MARGIN_PX does nothing while the paper itself can
 * be shrunk to a quarter of the screen - at 44% an A4 page is 349px wide on a
 * 1250px tablet, so two thirds of the glass is grey no matter where the camera
 * sits, and the margin rule can only centre the waste rather than remove it.
 * The cure is to stop the zoom at the point where more zooming out buys
 * nothing but grey: the page already fits, with the same margin around it.
 *
 * Derived from the viewport rather than being a fixed percentage, because
 * "40%" means a full page on a phone and a stamp on a monitor. On a wide
 * screen the height is what runs out first, so the floor lands low enough to
 * still see a whole A4 page; on a tall one it is the width.
 *
 * Returns 0 when there is nothing to go on, meaning "no floor".
 */
/**
 * How far past the page-fit you may still zoom out, as a fraction of it.
 *
 * Stopping exactly at the fit was too strict to use: at that zoom the sheet
 * fills the screen and you cannot see the next page coming, which is the
 * reason to zoom out at all. A fifth further shows the neighbouring sheets -
 * and because the pages are separated by a seam rather than a gap, what that
 * extra room shows is paper, not desk.
 */
export const ZOOM_OUT_HEADROOM = 0.8;

export function minNotebookZoom(
  vp: { width: number; height: number },
  page: { width: number; height: number }
): number {
  const usableW = vp.width - DESK_MARGIN_PX * 2;
  const usableH = vp.height - DESK_MARGIN_PX * 2;
  if (!(usableW > 0) || !(usableH > 0) || !page.width || !page.height) return 0;
  return Math.min(usableW / page.width, usableH / page.height) * ZOOM_OUT_HEADROOM;
}

export function clampNotebookCamera(
  cam: { x: number; y: number; zoom: number },
  vp: { width: number; height: number },
  page: { width: number; height: number },
  doc?: { pageCount: number; activeIndex: number }
)
```

```

): { x: number; y: number } {
  const z = cam.zoom;
  if (!z || !vp.width || !vp.height || !page.width || !page.height) {
    return { x: cam.x, y: cam.y };
  }
  const halfW = vp.width / 2 / z;
  const halfH = vp.height / 2 / z;
  const margin = DESK_MARGIN_PX / z;

  const count = Math.max(1, doc?.pageCount ?? 1);
  const index = Math.min(Math.max(0, doc?.activeIndex ?? 0), count - 1);
  const stride = page.height + PAGE_GAP;
  const docTop = -(index * stride);
  const docBottom = (count - 1 - index) * stride + page.height;

  return {
    x: boundAxis(cam.x, halfW - margin, page.width - halfW + margin),
    y: boundAxis(cam.y, docTop + halfH - margin, docBottom - halfH + margin),
  };
}

// -----
// One notebook view, shared by every pane that draws one
// -----
//
// A notebook is drawn by two components: the live canvas, and the split view's
// other pane. They used to compute their own layouts - the live canvas as a
// continuous strip anchored on the active page, the split pane as a single
// sheet at y = 0 - while sharing one stored camera. The same camera therefore
// meant two different pictures, and swapping which pane was live visibly moved
// the page. These helpers are the single definition both now use.

/** A sheet to draw, in the frame where the anchored (active) page sits at y = 0. */
export interface VisibleSheet {
  index: number;
  offsetY: number;
}

/**
 * The sheets a viewport can reach, including the anchored one.
 *
 * Mirrors the live canvas exactly: pages are one stride apart, and anything
 * more than a stride beyond the visible band is skipped, so a 200-page
 * notebook draws a handful of sheets rather than 200.
 */
export function visibleSheets(
  pageCount: number,
  activeIndex: number,
  pageHeight: number,
  cameraY: number,
  halfWorldHeight: number
): VisibleSheet[] {
  if (pageCount <= 0 || !(pageHeight > 0)) return [];
  const stride = pageHeight + PAGE_GAP;
  const topY = cameraY - halfWorldHeight;
  const bottomY = cameraY + halfWorldHeight;
  const out: VisibleSheet[] = [];
  for (let i = 0; i < pageCount; i++) {
    const offsetY = (i - activeIndex) * stride;
    if (offsetY + pageHeight < topY - stride) continue;
    if (offsetY > bottomY + stride) continue;
    out.push({ index: i, offsetY });
  }
  return out;
}

/**
 * Which way continuous scrolling should hand the anchor on: +1, -1 or 0.
 *
 * Once the view centre crosses the middle of the gap between two sheets the
 * next page becomes the anchored one and the camera slides one stride to
 * compensate, so the picture does not move. Same thresholds as the live canvas.
 */
export function reanchorDirection(
  cameraY: number,
  pageHeight: number,
  activeIndex: number,
  pageCount: number

```

```

): -1 | 0 | 1 {
  const forward = pageHeight + PAGE_GAP / 2;
  const backward = -PAGE_GAP / 2;
  if (cameraY > forward && activeIndex < pageCount - 1) return 1;
  if (cameraY < backward && activeIndex > 0) return -1;
  return 0;
}

=====
// === END FILE: src/lib/whiteboard/notebook.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/officeImport.ts ===
// File 108 of 194 | Lines: 738 | Size: 30204 bytes
=====

/**
 * Word and PowerPoint files -> a PDF -> a notebook.
 * =====
 * A .docx or .pptx is turned, ON THE DEVICE, into a PDF whose pages are the
 * document's pages (or slides), and that PDF then goes through the normal PDF
 * import. So an opened Word/PowerPoint file IS a PDF notebook afterwards:
 * export, share, split view, duplicate, delete/undo, page thumbnails and
 * "Open with" all work on it with no special cases anywhere else.
 *
 * Nothing is uploaded. Conversion needs no network.
 *
 * PowerPoint: slides are drawn with pptxviewjs (canvas). Two of its bugs are
 * worked around here, both measured on a real Hebrew lecture deck:
 * - It has no right-to-left support: Hebrew/Arabic lines were laid out with
 *   a left-to-right base, so punctuation landed at the wrong end. Lines
 *   containing RTL letters are drawn with an RTL base (same position).
 * - A text box holding an equation is stored twice (mc:AlternateContent): an
 *   Office-2010 version with the maths, and a fallback PICTURE of the box
 *   as PowerPoint drew it. The renderer cannot read the maths version and
 *   drew the whole box as a column one letter wide (7 of 25 slides of a real
 *   lecture deck). `preferFallbackContent` keeps the fallback - PowerPoint's own
 *   image of that box - so those slides look exactly as in PowerPoint.
 * - Placeholders get their layout position written in and a slot of their
 *   own (`normalizeSlideXml`), so nothing depends on the renderer's
 *   inheritance rules.
 *
 * Word: docx-preview lays the document out as HTML - the browser's own text
 * engine, so Hebrew, Arabic and mixed text are right - but it does not break
 * pages by height, so `paginateSection` does, between paragraphs and table
 * rows, never through a line. Each page is then drawn to an image.
 *
 * Text: the pages become pictures, so their text is captured on the way - a
 * box per word from the laid-out page (Word), every text-drawing call on the
 * slide canvas (PowerPoint) - and handed to the import, which stores it beside
 * the PDF (pdfTextStore.ts). That is what makes these pages selectable and
 * searchable.
 */

import { mergeFragments, type PageText, type TextRun } from "../pdfText";
import { attachOfficeText } from "../pdfTextStore";

export type OfficeKind = "docx" | "pptx";

const DOCX_MIME = "application/vnd.openxmlformats-officedocument.wordprocessingml.document";
const PPTX_MIME = "application/vnd.openxmlformats-officedocument.presentationml.presentation";
const PPSX_MIME = "application/vnd.openxmlformats-officedocument.presentationml.slideshow";

/** docx / pptx, "legacy" for the pre-2007 binary formats, else null. */
export function officeKindOf(file: { name: string; type?: string }): OfficeKind | "legacy" | null {
  const name = file.name.toLowerCase();
  const type = (file.type || "").toLowerCase();
  if (type === DOCX_MIME || /\.(\docx|docm|dotx)$/.test(name)) return "docx";
  if (type === PPTX_MIME || type === PPSX_MIME || /\.(\pptx|pptm|ppsx|potx)$/.test(name)) return "pptx";
  if (type === "application/msword" || type === "application/vnd.ms-powerpoint" || /\.(\doc|ppt|pps)$/.test(name)) {
    return "legacy";
  }
  return null;
}

```

```

export const LEGACY_OFFICE_MESSAGE =
  "This is an old Office format (.doc / .ppt). Open it in Word or PowerPoint and save it as .docx / .pptx, the
  n open that.";

/** What the file-pickers accept: PDF plus Word and PowerPoint. */
export const DOCUMENT_ACCEPT = [
  "application/pdf",
  ".pdf",
  DOCX_MIME,
  ".docx",
  PPTX_MIME,
  PPSX_MIME,
  ".pptx",
  ".ppsx",
].join(",");

export interface ConvertProgress {
  /** Human-readable: "Reading slides...", "Slide 3 of 25"... */
  label: string;
  done: number;
  total: number;
}

// -----
// PowerPoint XML clean-up (pure: unit-tested)
// -----

const RTL_CHARS = /[■-■■-■■-■]/;

/** True when a string contains Hebrew/Arabic/Syriac/Thaana letters. */
export function hasRtl(text: string): boolean {
  return RTL_CHARS.test(text);
}

interface PlaceholderGeom {
  idx: string | null;
  type: string | null;
  xfrm: string;
}

function attr(tag: string, name: string): string | null {
  const m = new RegExp(`\\s${name}="(["]*)"`).exec(tag);
  return m ? m[1] : null;
}

/** Placeholders (idx, type, <a:xfrm>) defined on a layout or master. */
export function placeholderGeometry(xml: string): PlaceholderGeom[] {
  const out: PlaceholderGeom[] = [];
  for (const sp of xml.match(/<p:sp>[\s\S]*?<\p:sp>/g) ?? []) {
    const ph = /<p:ph\b[^\>]*>/.exec(sp)?.[0];
    const xfrm = /<a:xfrm\b[\s\S]*?<\a:xfrm>/.exec(sp)?.[0];
    if (!ph || !xfrm) continue;
    out.push({ idx: attr(ph, "idx"), type: attr(ph, "type"), xfrm });
  }
  return out;
}

/** A placeholder with no type is a body placeholder (ECMA-376 19.3.1.36). */
const phType = (t: string | null) => t ?? "body";
const sameKind = (a: string, b: string) =>
  a === b || (a === "ctrTitle" && b === "title") || (a === "title" && b === "ctrTitle") || (a === "subTitle" &
    & b === "body");

function findGeometry(
  idx: string | null,
  type: string | null,
  layout: PlaceholderGeom[],
  master: PlaceholderGeom[]
): string | null {
  const t = phType(type);
  if (idx !== null) {
    const byIdx = layout.find((g) => g.idx === idx);
    if (byIdx) return byIdx.xfrm;
  }
  const byType = layout.find((g) => sameKind(phType(g.type), t)) ?? master.find((g) => sameKind(phType(g.type),
    t));
  return byType?.xfrm ?? null;
}

```

```

/** The visible text of a shape (all <a:t> runs). */
function shapeText(sp: string): string {
    return (sp.match(/<a:t>([\^>]*<\a:t>/g) ?? []).map((t) => t.slice(5, -6)).join("");
}

/**
 * A text-less shape filled with a picture, rewritten as `<p:pic>` at the same
 * place with the same crop. Null when the shape is anything else, or has no
 * position to put the picture at.
 */
export function pictureFromFilledBox(sp: string): string | null {
    const spPr = /<p:spPr\b[\^>]*>([\s\S]*?)<\p:spPr>/.exec(sp)?.[1];
    if (!spPr) return null;
    const blipFill = /<a:blipFill\b[\^>]*>([\s\S]*?)<\a:blipFill>/.exec(spPr)?.[0];
    const xfrm = /<a:xfrm\b[\s\S]*?<\a:xfrm>/.exec(spPr)?.[0];
    if (!blipFill || !xfrm || shapeText(sp).trim() !== "") return null;
    const cNvPr = /<p:cNvPr\b[\^>]*?(\s|>)[\s\S]*?<\p:cNvPr>/.exec(sp)?.[0] ?? '<p:cNvPr id="0" name="Picture"/>';
    const inner = blipFill.replace(/<a:blipFill\b[\^>]*>/, "").replace(/<\a:blipFill>$/, "");
    return (
        `<p:pic><p:nvPicPr>${cNvPr}<p:cNvPicPr><p:nvPr><\p:nvPicPr>` +
        `<p:blipFill>${inner}<\p:blipFill>` +
        `<p:spPr>${xfrm}<a:prstGeom prst="rect"><a:avLst><\a:prstGeom><\p:spPr><\p:pic>`
    );
}

/**
 * Gives every placeholder on a slide an explicit position and a slot of its
 * own. A placeholder without `<a:xfrm>` gets its layout's (or master's), and a
 * second shape claiming an already-used `idx` is moved to a free idx - the
 * renderer mixed up the geometry of shapes sharing one.
 */
export function normalizeSlideXml(slideXml: string, layoutXml: string, masterXml = ""): string {
    const layout = placeholderGeometry(layoutXml);
    const master = placeholderGeometry(masterXml);
    const seen = new Set<string>();
    let spare = 9000;
    return slideXml.replace(/<p:sp>[\s\S]*?<\p:sp>/g, (sp) => {
        const phTag = /<p:ph\b[\^>]*>/.exec(sp)?.[0];
        if (!phTag) return sp;
        const idx = attr(phTag, "idx");
        const type = attr(phTag, "type");
        let out = sp;
        const spPr = /<p:spPr\b[\^>]*?(\s|>)([\s\S]*?<\p:spPr>)?/.exec(out);
        const hasXfrm = spPr ? /<a:xfrm\b/.test(spPr[0]) : false;
        if (!hasXfrm) {
            const xfrm = findGeometry(idx, type, layout, master);
            if (xfrm) {
                if (spPr && spPr[1] === "/>") {
                    out = out.replace(spPr[0], spPr[0].replace(/>$/, `>${xfrm}<\p:spPr>`));
                } else if (spPr) {
                    out = out.replace(spPr[0], spPr[0].replace(/^(<p:spPr\b[\^>]*>)/, `_${xfrm}`));
                }
            }
        }
        // A box whose only content is a picture fill (PowerPoint's fallback image
        // of an equation box) becomes a real picture: the renderer draws
        // pictures, but not pictures used as a box's fill.
        const asPicture = pictureFromFilledBox(out);
        if (asPicture) return asPicture;

        const key = idx ?? `type:${attr(phTag, "type")}`;
        if (idx !== null && seen.has(key)) {
            // Already positioned above; a unique idx stops it borrowing the
            // layout slot another shape on this slide owns.
            let free = String(spare++);
            while (seen.has(free)) free = String(spare++);
            out = out.replace(phTag, phTag.replace(/sidx="[\^"]*"\/, ` idx="${free}"`));
            seen.add(free);
        }
        seen.add(key);
        return out;
    });
}

/**
 * Replaces every `mc:AlternateContent` with its `mc:Fallback` (or, with no

```

```

* fallback, its first `mc:Choice`). For equations the fallback is
* PowerPoint's own picture of the text box - exact - where the renderer drew
* the maths version as a broken one-letter column.
*/
export function preferFallbackContent(xml: string): string {
  if (!xml.includes("mc:AlternateContent")) return xml;
  return xml.replace(/<mc:AlternateContent\b[^>]*>([\s\S]*)<\/mc:AlternateContent>/g, (_all, inner: string) =
    > {
      const fallback = /<mc:Fallback\b[^>]*>([\s\S]*)<\/mc:Fallback>/.exec(inner);
      if (fallback) return fallback[1];
      const choice = /<mc:Choice\b[^>]*>([\s\S]*)<\/mc:Choice>/.exec(inner);
      return choice ? choice[1] : "";
    });
}

/** Resolves a relationship target ("../slideLayouts/x.xml") against a part. */
function resolveTarget(fromPart: string, target: string): string {
  const base = fromPart.split("/").slice(0, -1);
  for (const seg of target.split("/")) {
    if (seg === "..") base.pop();
    else if (seg !== ".") base.push(seg);
  }
  return base.join("/");
}

async function normalizePptx(bytes: ArrayBuffer): Promise<ArrayBuffer> {
  const { default: JSZip } = await import("jszip");
  const zip = await JSZip.loadAsync(bytes);
  const read = async (p: string) => (zip.file(p) ? await zip.file(p)!.async("string") : "");
  const slides = Object.keys(zip.files).filter((p) => /^ppt\/slides\/slide\d+\.xml$/ .test(p));
  for (const slide of slides) {
    const rels = await read(slide.replace(/slides\/(slide\d+\.xml)$/, "slides/_rels/$1.rels"));
    const layoutTarget = /Target="([^\"]*slideLayout[^\"]*)"/.exec(rels)?.[1];
    if (!layoutTarget) continue;
    const layoutPath = resolveTarget(slide, layoutTarget);
    const layoutRels = await read(layoutPath.replace(/slideLayouts\/(slideLayout\d+\.xml)$/, "slideLayouts/_rels/$1.rels"));
    const masterTarget = /Target="([^\"]*slideMaster[^\"]*)"/.exec(layoutRels)?.[1];
    const masterXml = masterTarget ? await read(resolveTarget(layoutPath, masterTarget)) : "";
    const xml = await read(slide);
    const fixed = normalizeSlideXml(preferFallbackContent(xml), preferFallbackContent(await read(layoutPath)),
      masterXml);
    if (fixed !== xml) zip.file(slide, fixed);
  }
  return zip.generateAsync({ type: "arraybuffer" });
}

// -----
// Drawing surfaces
// -----

/** An off-screen host: laid out (so text measures right) but never visible. */
function offscreenHost(): HTMLDivElement {
  const host = document.createElement("div");
  host.setAttribute("aria-hidden", "true");
  host.style.cssText =
    "position:fixed;left:-100000px;top:0;pointer-events:none;background:#fff;color:#000;z-index:-1;";
  document.body.appendChild(host);
  return host;
}

/** Lines with Hebrew/Arabic letters are drawn with an RTL base, same anchor. */
function patchRtlText(ctx: CanvasRenderingContext2D) {
  for (const fn of ["fillText", "strokeText"] as const) {
    const orig = ctx[fn].bind(ctx);
    ctx[fn] = ((text: string, x: number, y: number, maxWidth?: number) => {
      if (typeof text === "string" && hasRtl(text)) {
        const dir = ctx.direction;
        const align = ctx.textAlign;
        ctx.direction = "rtl";
        // The renderer measured with a left-to-right base: keep its anchor.
        ctx.textAlign = align === "start" ? "left" : align === "end" ? "right" : align;
        try {
          return maxWidth === undefined ? orig(text, x, y) : orig(text, x, y, maxWidth);
        } finally {
          ctx.direction = dir;
          ctx.textAlign = align;
        }
      }
    });
  }
}

```

```

    }
    return maxWidth === undefined ? orig(text, x, y) : orig(text, x, y, maxWidth);
  }) as typeof ctx.fillText;
}
}

interface RasterPage {
  /** JPEG data URL. */
  image: string;
  /** Page size in PDF points (1/72 in). */
  widthPt: number;
  heightPt: number;
  /** The page's text, captured while it was drawn. */
  text?: PageText;
}

const EMU_PER_PT = 12700;
/** Long side of a rendered slide, in pixels: sharp on a tablet at zoom. */
const SLIDE_PX = 1920;

/**
 * Loads pptxviewjs WITHOUT letting it damage the page. On load it assigns its
 * own classes over browser globals - `window.FileReader` among them - and after
 * one PowerPoint import every later file read in the app (PDFs, pictures)
 * failed with "readAsArrayBuffer is not a function". Whatever it overwrites is
 * put back; names it only adds are left alone (it may use them itself).
 */
let pptxModule: Promise<typeof import("pptxviewjs")> | null = null;
export function loadPptxViewer(): Promise<typeof import("pptxviewjs")> {
  if (!pptxModule) {
    const w = window as unknown as Record<string, unknown>;
    const before = new Map<string, unknown>();
    for (const key of Object.getOwnPropertyNames(window)) {
      try {
        before.set(key, w[key]);
      } catch {
        /* some window properties throw on read; they cannot be overwritten either */
      }
    }
    pptxModule = import("pptxviewjs").then((mod) => {
      restoreGlobals(w, before);
      return mod;
    });
  }
  return pptxModule;
}

/** Puts back every global that `before` knew and that has since changed. */
export function restoreGlobals(target: Record<string, unknown>, before: Map<string, unknown>): string[] {
  const restored: string[] = [];
  for (const [key, value] of before) {
    let now: unknown;
    try {
      now = target[key];
    } catch {
      continue;
    }
    if (now !== value) {
      try {
        target[key] = value;
        restored.push(key);
      } catch {
        /* read-only: it could not have been replaced either */
      }
    }
  }
  return restored;
}

type PageSink = (page: RasterPage) => void;

/**
 * Records the text drawn on a slide canvas: every fillText call, with where it
 * really landed (the canvas transform, alignment and font metrics applied), in
 * fractions of the canvas. Installed OVER the RTL patch, so it sees the
 * renderer's own alignment - which the patch keeps - and the logical string.
 */
function recordCanvasText(ctx: CanvasRenderingContext2D, canvas: HTMLCanvasElement) {

```

```

let frags: TextRun[] | null = null;
const orig = ctx.fillText.bind(ctx);
const note = (out: TextRun[], text: string, x: number, y: number, maxWidth?: number) => {
  const m = ctx.measureText(text);
  const w = maxWidth !== undefined ? Math.min(m.width, maxWidth) : m.width;
  if (!(w > 0)) return;
  const rtl = ctx.direction === "rtl";
  const align = ctx.textAlign;
  const left =
    align === "right" || (align === "end" && !rtl) || (align === "start" && rtl)
      ? x - w
      : align === "center"
        ? x - w / 2
        : x;
  const size = parseFloat(/(\d+(?:\.\d+)?)px/.exec(ctx.font)?.[1] ?? "") || 16;
  const ascent = m.fontBoundingBoxAscent || m.actualBoundingBoxAscent || size * 0.8;
  const descent = m.fontBoundingBoxDescent || m.actualBoundingBoxDescent || size * 0.2;
  const t = ctx.getTransform();
  const pts = [
    [left, y - ascent],
    [left + w, y - ascent],
    [left, y + descent],
    [left + w, y + descent],
  ].map(([px, py]) => [t.a * px + t.c * py + t.e, t.b * px + t.d * py + t.f]);
  const xs = pts.map((p) => p[0]);
  const ys = pts.map((p) => p[1]);
  const x0 = Math.max(0, Math.min(...xs));
  const x1 = Math.min(canvas.width, Math.max(...xs));
  const y0 = Math.max(0, Math.min(...ys));
  const y1 = Math.min(canvas.height, Math.max(...ys));
  if (x1 <= x0 || y1 <= y0) return;
  out.push({
    t: text,
    x: x0 / canvas.width,
    y: y0 / canvas.height,
    w: (x1 - x0) / canvas.width,
    h: (y1 - y0) / canvas.height,
  });
};
ctx.fillText = ((text: string, x: number, y: number, maxWidth?: number) => {
  if (frags && typeof text === "string" && text.trim()) {
    try {
      note(frags, text, x, y, maxWidth);
    } catch {
      /* recording must never break drawing */
    }
  }
  return maxWidth === undefined ? orig(text, x, y) : orig(text, x, y, maxWidth);
}) as typeof ctx.fillText;
return {
  start() {
    frags = [];
  },
  stop(): TextRun[] {
    const out = frags ?? [];
    frags = null;
    return out;
  },
};
}

async function rasterizePptx(bytes: ArrayBuffer, sink: PageSink, onProgress?: (p: ConvertProgress) => void): P
  romise<number> {
    onProgress?.({ label: "Reading slides...", done: 0, total: 1 });
    const fixed = await normalizePptx(bytes);
    const { PPTXViewer } = await loadPptxViewer();
    const host = offscreenHost();
    const canvas = document.createElement("canvas");
    host.appendChild(canvas); // the renderer refuses a detached canvas
    const ctx = canvas.getContext("2d");
    if (!ctx) throw new Error("This device cannot draw the slides.");
    patchRtlText(ctx);
    const recorder = recordCanvasText(ctx, canvas);
    try {
      const viewer = new PPTXViewer({ canvas });
      await viewer.loadFile(fixed);
      const count = viewer.getSlideCount();
      if (!count) throw new Error("This presentation has no slides.");
    }
  }

```

```

const proc = (viewer as unknown as { processor?: { getSlideDimensions?: () => { cx: number; cy: number } }
  }).processor;
const dims = proc?.getSlideDimensions?.() ?? { cx: 12192000, cy: 6858000 };
const cx = dims.cx > 0 ? dims.cx : 12192000;
const cy = dims.cy > 0 ? dims.cy : 6858000;
const scale = SLIDE_PX / Math.max(cx, cy);
canvas.width = Math.round(cx * scale);
canvas.height = Math.round(cy * scale);
for (let i = 0; i < count; i++) {
  onProgress?({ label: `Slide ${i + 1} of ${count}`, done: i, total: count });
  ctx.save();
  ctx.fillStyle = "#ffffff";
  ctx.fillRect(0, 0, canvas.width, canvas.height);
  ctx.restore();
  recorder.start();
  await viewer.render(canvas, { slideIndex: i });
  const runs = mergeFragments(recorder.stop(), canvas.width / canvas.height);
  const widthPt = cx / EMU_PER_PT;
  const heightPt = cy / EMU_PER_PT;
  sink({ image: canvas.toDataURL("image/jpeg", 0.9), widthPt, heightPt, text: { w: widthPt, h: heightPt, r
    uns, ordered: true } });
  // Let the page breathe between slides: a long deck must not freeze it.
  await new Promise((r) => setTimeout(r, 0));
}
return count;
} finally {
  host.remove();
}
}

// -----
// Word
// -----

/**
 * Splits one docx-preview section (which grows as tall as its content) into
 * page-sized sections. Whole paragraphs move to the next page; a table too tall
 * for a page is split between rows; a single block taller than a page keeps a
 * page to itself (and is clipped there) rather than looping.
 */
export function paginateSection(section: HTMLElement): HTMLElement[] {
  const article = section.querySelector(":scope > article") as HTMLElement | null;
  const pageH = parseFloat(section.style.minHeight) || 0;
  if (!article || !pageH) return [section];
  // docx-preview sizes pages as content-box: min-height is the height INSIDE
  // the margins. Fixing the height and comparing scroll to client height
  // measures overflow correctly whatever the box model.
  section.style.height = `${pageH}px`;
  section.style.overflow = "hidden";
  if (section.scrollHeight <= section.clientHeight + 1) return [section];
  const header = section.querySelector(":scope > header");
  const footer = section.querySelector(":scope > footer");
  const blocks = Array.from(article.children) as HTMLElement[];
  const pages: HTMLElement[] = [];

  const newPage = () => {
    const page = section.cloneNode(false) as HTMLElement;
    page.style.height = `${pageH}px`;
    page.style.overflow = "hidden";
    if (header) page.appendChild(header.cloneNode(true));
    const art = article.cloneNode(false) as HTMLElement;
    page.appendChild(art);
    if (footer) page.appendChild(footer.cloneNode(true));
    section.parentElement?.insertBefore(page, section);
    pages.push(page);
    return { page, art };
  };

  const overflows = (page: HTMLElement) => page.scrollHeight > page.clientHeight + 1;

  let cur = newPage();
  for (const block of blocks) {
    cur.art.appendChild(block);
    if (!overflows(cur.page)) continue;
    const alone = cur.art.children.length === 1;
    if (block.tagName === "TABLE" && block.querySelector("tr")) {
      // Keep the rows that fit here; carry the rest in a copy of the table.
      const rows = Array.from(block.querySelectorAll(":scope > tbody > tr, :scope > tr")) as HTMLElement[];
      let rest: HTMLElement[] = [];

```

```

    for (let i = rows.length - 1; i > 0 && overflows(cur.page); i--) {
      rest.unshift(rows[i]);
      rows[i].remove();
    }
    while (rest.length) {
      cur = newPage();
      const copy = block.cloneNode(false) as HTMLElement;
      const body = document.createElement("tbody");
      copy.appendChild(body);
      cur.art.appendChild(copy);
      const moved: HTMLElement[] = [];
      for (const r of rest) {
        body.appendChild(r);
        moved.push(r);
        if (overflows(cur.page) && moved.length > 1) {
          r.remove();
          moved.pop();
          break;
        }
      }
      rest = rest.slice(moved.length);
    }
    continue;
  }
  if (alone) {
    // Taller than a page on its own: it keeps this page, clipped.
    cur = newPage();
    continue;
  }
  block.remove();
  cur = newPage();
  cur.art.appendChild(block);
  if (overflows(cur.page)) cur = newPage();
}
// A trailing empty page (from the last "alone" block) is dropped.
const last = pages[pages.length - 1];
if (pages.length > 1 && last && last.querySelector("article")?.children.length === 0) {
  last.remove();
  pages.pop();
}
section.remove();
return pages;
}

/** Draws one page element: the browser's own renderer first, a fallback second. */
async function drawElement(el: HTMLElement, scale: number): Promise<string> {
  const w = el.offsetWidth;
  const h = el.offsetHeight;
  try {
    const { toCanvas } = await import("html-to-image");
    const canvas = await Promise.race([
      toCanvas(el, { pixelRatio: scale, backgroundColor: "#ffffff", width: w, height: h, cacheBust: false }),
      new Promise<never>((_, reject) => setTimeout(() => reject(new Error("timeout")), 6000)),
    ]);
    return canvas.toDataURL("image/jpeg", 0.9);
  } catch {
    // html-to-image draws through an SVG image; a WebView that stalls on it
    // gets html2canvas, which re-lays the text itself (slightly less exact
    // for mixed-direction lines, but never blank).
    const { default: html2canvas } = await import("html2canvas");
    const canvas = await html2canvas(el, {
      scale,
      backgroundColor: "#ffffff",
      logging: false,
      width: w,
      height: h,
      // The page only needs the document's own styles. The app's stylesheet
      // uses oklch() colours, which html2canvas cannot parse - it threw on
      // the very first page. Strip every stylesheet but the document's.
      onclone: (doc) => {
        doc.querySelectorAll("style, link[rel=stylesheet]").forEach((node) => {
          if (!(node as Element).closest("[data-office-styles]")) node.remove();
        });
      },
    });
    return canvas.toDataURL("image/jpeg", 0.9);
  }
}

```

```

const PX_TO_PT = 0.75;

/**
 * The words of a laid-out page with their boxes, in document (reading) order,
 * as fractions of the page. A word the page clips away (the overflow of a
 * block taller than a page) is left out, as it is from the picture.
 */
export function captureDomText(page: HTMLElement): PageText {
  const box = page.getBoundingClientRect();
  const W = box.width || page.offsetWidth || 1;
  const H = box.height || page.offsetHeight || 1;
  const runs: TextRun[] = [];
  const range = document.createRange();
  const walker = document.createTreeWalker(page, NodeFilter.SHOW_TEXT);
  for (let node = walker.nextNode() as Text | null; node; node = walker.nextNode() as Text | null) {
    const parent = node.parentElement;
    if (!parent || parent.closest("style, script")) continue;
    for (const m of node.data.matchAll(/\\S+/g)) {
      const at = m.index ?? 0;
      range.setStart(node, at);
      range.setEnd(node, at + m[0].length);
      const rects = Array.from(range.getClientRects()).filter((r) => r.width > 0 && r.height > 0);
      if (!rects.length) continue;
      // A word broken over two lines (a long URL) is shared out by width.
      const total = rects.reduce((n, r) => n + r.width, 0);
      let used = 0;
      rects.forEach((r, k) => {
        const take = k === rects.length - 1 ? m[0].length - used : Math.round((m[0].length * r.width) / total);
        const t = m[0].slice(used, used + take);
        used += take;
        const x = r.left - box.left;
        const y = r.top - box.top;
        if (!t || y >= H || y + r.height <= 0 || x >= W || x + r.width <= 0) return;
        runs.push({ t, x: x / W, y: y / H, w: r.width / W, h: r.height / H });
      });
    }
  }
  return { w: W * PX_TO_PT, h: H * PX_TO_PT, runs, ordered: true };
}

async function rasterizeDocx(bytes: ArrayBuffer, sink: PageSink, onProgress?: (p: ConvertProgress) => void): Promise<number> {
  onProgress?.({ label: "Reading document...", done: 0, total: 1 });
  const { renderAsync } = await import("docx-preview");
  const host = offscreenHost();
  const styles = document.createElement("div");
  styles.setAttribute("data-office-styles", "");
  const body = document.createElement("div");
  host.append(styles, body);
  try {
    await renderAsync(bytes, body, styles, {
      inWrapper: false,
      breakPages: true,
      ignoreLastRenderedPageBreak: false,
      experimental: true,
      renderHeaders: true,
      renderFooters: true,
      renderFootnotes: true,
      renderEndnotes: true,
      useBase64URL: true,
    });
    // Fonts must be ready before anything is measured or drawn.
    await (document as Document & { fonts?: FontFaceSet }).fonts?.ready;
    const sections = Array.from(body.querySelectorAll("section.docx")) as HTMLElement[];
    const pages = sections.flatMap((s) => paginateSection(s));
    if (!pages.length) throw new Error("This document has no pages.");
    for (let i = 0; i < pages.length; i++) {
      onProgress?.({ label: `Page ${i + 1} of ${pages.length}`, done: i, total: pages.length });
      const page = pages[i];
      let text: PageText | undefined;
      try {
        text = captureDomText(page);
      } catch {
        /* a page without its text is still a page */
      }
      const image = await drawElement(page, 2);
    }
  }
}

```

```

        sink({ image, widthPt: page.offsetWidth * PX_TO_PT, heightPt: page.offsetHeight * PX_TO_PT, text });
        await new Promise((r) => setTimeout(r, 0));
    }
    return pages.length;
} finally {
    host.remove();
}
}

// -----
// The one entry point
// -----

/** A PDF built one page at a time; each page image can be dropped once added. */
async function pdfBuilder() {
    const { jsPDF } = await import("jspdf");
    let pdf: InstanceType<typeof jsPDF> | null = null;
    return {
        add(p: RasterPage) {
            const orientation = p.widthPt > p.heightPt ? "landscape" : "portrait";
            if (!pdf) {
                pdf = new jsPDF({ unit: "pt", format: [p.widthPt, p.heightPt], orientation, compress: true });
            } else {
                pdf.addPage([p.widthPt, p.heightPt], orientation);
            }
            pdf.drawImage(p.image, "JPEG", 0, 0, p.widthPt, p.heightPt, undefined, "FAST");
        },
        blob(): Blob {
            if (!pdf) throw new Error("The document has no pages.");
            return pdf.output("blob");
        },
    };
}

/**
 * Converts a .docx or .pptx into a PDF File named after the original (so the
 * notebook it becomes is called "Lecture 3", not "Lecture 3.pdf").
 */
export async function convertOfficeToPdf(file: File, onProgress?: (p: ConvertProgress) => void): Promise<File>
{
    const kind = officeKindOf(file);
    if (kind === "legacy") throw new Error(LEGACY_OFFICE_MESSAGE);
    if (!kind) throw new Error(`"${file.name}" is not a Word or PowerPoint file.`);
    const bytes = await file.arrayBuffer();
    const builder = await pdfBuilder();
    const texts: PageText[] = [];
    const sink: PageSink = (page) => {
        builder.add(page);
        texts.push(page.text ?? { w: page.widthPt, h: page.heightPt, runs: [] });
    };
    let pageCount: number;
    try {
        pageCount = kind === "pptx" ? await rasterizePptx(bytes, sink, onProgress) : await rasterizeDocx(bytes, sink, onProgress);
    } catch (err) {
        const msg = (err as Error)?.message || String(err);
        throw new Error(
            `/corrupt|zip|end of central directory|signature/i.test(msg)
            ? `"${file.name}" could not be read - it may be damaged or password-protected.`
            : msg
        );
    }
    onProgress?.({ label: "Making the notebook...", done: pageCount, total: pageCount });
    const blob = builder.blob();
    const base = file.name.replace(/\.[^.]*/, "");
    const pdf = new File([blob], `${base}.pdf`, { type: "application/pdf" });
    attachOfficeText(pdf, texts);
    return pdf;
}

=====
// === END FILE: src/lib/whiteboard/officeImport.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/onedrive.ts ===
// File 109 of 194 | Lines: 353 | Size: 11737 bytes

```

```

=====

/**
 * OneDrive Sync (Microsoft account)
 * =====
 * Signs in with a Microsoft account and keeps ONE workspace file in the user's
 * own OneDrive, so the same email gives the same boards on every device.
 *
 * Why this shape:
 * - **App folder scope only** (`Files.ReadWrite.AppFolder`). We can read and
 *   write exactly one folder that belongs to this app – never the rest of the
 *   user's OneDrive. Least privilege, and the consent prompt says so.
 * - **No backend, no secret.** Browser apps cannot keep a client secret, so
 *   this uses the OAuth 2.0 authorization-code flow with PKCE, which is the
 *   only flow Microsoft still allows for SPAs. The implicit flow is deprecated.
 * - **No dependency.** MSAL would add ~100KB for what is three fetch calls.
 *
 * Storage cost is the user's own OneDrive quota, so syncing is free forever.
 */

import { parseWorkspace, serializeWorkspace, type WorkspaceSnapshot } from "./workspace";

const AUTH_HOST = "https://login.microsoftonline.com";
const GRAPH = "https://graph.microsoft.com/v1.0";
/** "common" = personal Microsoft accounts AND work/school accounts. */
const TENANT = "common";
const SCOPES = ["Files.ReadWrite.AppFolder", "User.Read", "offline_access"];
const FILE_NAME = "pronotes-workspace.json";

const LS_TOKENS = "pronotes.ms.tokens";
const LS_VERIFIER = "pronotes.ms.pkce";
const LS_RETURN = "pronotes.ms.return";

export interface StoredTokens {
  accessToken: string;
  refreshToken?: string;
  /** Epoch ms. */
  expiresAt: number;
  account?: { name?: string; email?: string };
}

export interface RemoteWorkspace {
  snapshot: WorkspaceSnapshot;
  /** Graph eTag – lets us detect that the remote changed under us. */
  eTag?: string;
  lastModified?: number;
}

export function getClientId(): string | null {
  const id = process.env.NEXT_PUBLIC_MS_CLIENT_ID;
  return id && id.trim() ? id.trim() : null;
}

export function isConfigured(): boolean {
  return !!getClientId();
}

function redirectUri(): string {
  return `${window.location.origin}${window.location.pathname}`;
}

// -----
// PKCE helpers
// -----

function base64Url(bytes: ArrayBuffer): string {
  return btoa(String.fromCharCode(...new Uint8Array(bytes)))
    .replace(/\+/g, "-")
    .replace(/\//g, "_")
    .replace(/=+$/, "");
}

async function makeVerifierAndChallenge() {
  const random = new Uint8Array(64);
  crypto.getRandomValues(random);
  const verifier = base64Url(random.buffer);
  const digest = await crypto.subtle.digest("SHA-256", new TextEncoder().encode(verifier));
  return { verifier, challenge: base64Url(digest) };
}

```

```

}

// -----
// Token storage
// -----

/**
 * Web storage throws (Safari private mode, quota, storage disabled): a saved
 * credential is a convenience, never a requirement, so every access below is
 * guarded and a failure just means "not signed in" / "sign in again".
 */
function storageGet(store: Storage, key: string): string | null {
  try {
    return store.getItem(key);
  } catch {
    return null;
  }
}

function storageSet(store: Storage, key: string, value: string): void {
  try {
    store.setItem(key, value);
  } catch {
    // Swallowed deliberately – but note what it costs: there is NO in-memory
    // copy of the tokens (loadTokens re-reads storage on every call), so a
    // failed write means the user is simply not signed in, not "signed in
    // until reload". Callers that must not silently lose the session (like
    // beginSignIn's verifier write) verify with a read-back instead of
    // trusting this.
  }
}

function storageRemove(store: Storage, key: string): void {
  try {
    store.removeItem(key);
  } catch {
    /* the key is unreadable anyway, so there is nothing left to clear */
  }
}

export function loadTokens(): StoredTokens | null {
  try {
    const raw = storageGet(localStorage, LS_TOKENS);
    if (!raw) return null;
    const t = JSON.parse(raw) as StoredTokens;
    return typeof t?.accessToken === "string" ? t : null;
  } catch {
    return null;
  }
}

function storeTokens(t: StoredTokens): void {
  storageSet(localStorage, LS_TOKENS, JSON.stringify(t));
}

export function signOut(): void {
  storageRemove(localStorage, LS_TOKENS);
  // The verifier (and return URL) live in sessionStorage – that is the only
  // place they are ever written (beginSignIn) and read
  // (completeSignInIfRedirected). Clearing them from localStorage, as this
  // used to do, cleared nothing.
  storageRemove(sessionStorage, LS_VERIFIER);
  storageRemove(sessionStorage, LS_RETURN);
}

// -----
// Sign-in
// -----

export async function beginSignIn(): Promise<void> {
  const clientId = getClientId();
  if (!clientId) throw new Error("Microsoft sign-in is not configured.");

  const { verifier, challenge } = await makeVerifierAndChallenge();
  storageSet(sessionStorage, LS_VERIFIER, verifier);
  // The stored verifier is the ONLY copy – nothing keeps it in memory – so
  // prove it landed BEFORE leaving for Microsoft. Otherwise the user logs in,
  // comes back, and sign-in fails silently for lack of a verifier: a wasted

```

```

// round trip with no explanation. SyncPanel's doSignIn shows this error.
if (storageGet(sessionStorage, LS_VERIFIER) !== verifier) {
  throw new Error(
    "This browser is blocking storage, so Microsoft sign-in can't finish. " +
    "Allow site data (or leave private browsing) and try again."
  );
}
storageSet(sessionStorage, LS_RETURN, window.location.href);

const params = new URLSearchParams({
  client_id: clientId,
  response_type: "code",
  redirect_uri: redirectUri(),
  response_mode: "query",
  scope: SCOPES.join(" "),
  code_challenge: challenge,
  code_challenge_method: "S256",
});
window.location.assign(`${AUTH_HOST}/${TENANT}/oauth2/v2.0/authorize?${params}`);
}

/**
 * Complete sign-in if we came back from Microsoft with `?code=`.
 * Returns true when a sign-in was completed on this page load.
 */
export async function completeSignInIfRedirected(): Promise<boolean> {
  const url = new URL(window.location.href);
  const code = url.searchParams.get("code");
  const error = url.searchParams.get("error");

  if (error) {
    // Clean the URL so a refresh does not replay the failure.
    cleanAuthParams();
    throw new Error(url.searchParams.get("error_description") || error);
  }
  if (!code) return false;

  const clientId = getClientId();
  const verifier = storageGet(sessionStorage, LS_VERIFIER);
  if (!clientId || !verifier) {
    cleanAuthParams();
    return false;
  }

  const body = new URLSearchParams({
    client_id: clientId,
    grant_type: "authorization_code",
    code,
    redirect_uri: redirectUri(),
    code_verifier: verifier,
    scope: SCOPES.join(" "),
  });

  const res = await fetch(`${AUTH_HOST}/${TENANT}/oauth2/v2.0/token`, {
    method: "POST",
    headers: { "Content-Type": "application/x-www-form-urlencoded" },
    body,
  });
  storageRemove(sessionStorage, LS_VERIFIER);
  cleanAuthParams();

  if (!res.ok) {
    throw new Error(`Microsoft sign-in failed (${res.status}).`);
  }
  const json = (await res.json()) as {
    access_token: string;
    refresh_token?: string;
    expires_in: number;
  };
  const tokens: StoredTokens = {
    accessToken: json.access_token,
    refreshToken: json.refresh_token,
    expiresAt: Date.now() + json.expires_in * 1000,
  };
  storeTokens(tokens);
  tokens.account = await fetchAccount(tokens.accessToken);
  storeTokens(tokens);
  return true;
}

```

```

}

function cleanAuthParams(): void {
  const url = new URL(window.location.href);
  for (const k of ["code", "error", "error_description", "session_state", "state"]) {
    url.searchParams.delete(k);
  }
  window.history.replaceState({}, "", url.toString());
}

async function fetchAccount(accessToken: string) {
  try {
    const res = await fetch(`${GRAPH}/me`, {
      headers: { Authorization: `Bearer ${accessToken}` },
    });
    if (!res.ok) return undefined;
    const me = (await res.json()) as {
      displayName?: string;
      userPrincipalName?: string;
      mail?: string;
    };
    return { name: me.displayName, email: me.mail || me.userPrincipalName };
  } catch {
    return undefined;
  }
}

/** A valid access token, refreshing silently when it has expired. */
async function getAccessToken(): Promise<string | null> {
  const tokens = loadTokens();
  if (!tokens) return null;
  // Refresh a minute early – a token that expires mid-request would surface as
  // a spurious "sync failed".
  if (tokens.expiresAt - 60_000 > Date.now()) return tokens.accessToken;
  if (!tokens.refreshToken) return null;

  const clientId = getClientId();
  if (!clientId) return null;

  const res = await fetch(`${AUTH_HOST}/${TENANT}/oauth2/v2.0/token`, {
    method: "POST",
    headers: { "Content-Type": "application/x-www-form-urlencoded" },
    body: new URLSearchParams({
      client_id: clientId,
      grant_type: "refresh_token",
      refresh_token: tokens.refreshToken,
      scope: SCOPES.join(" "),
    }),
  });
  if (!res.ok) {
    // The refresh token is dead (revoked / expired): force a fresh sign-in
    // rather than retrying forever.
    signOut();
    return null;
  }
  const json = (await res.json()) as {
    access_token: string;
    refresh_token?: string;
    expires_in: number;
  };
  const next: StoredTokens = {
    accessToken: json.access_token,
    refreshToken: json.refresh_token ?? tokens.refreshToken,
    expiresAt: Date.now() + json.expires_in * 1000,
    account: tokens.account,
  };
  storeTokens(next);
  return next.accessToken;
}

export function getSignedInAccount(): StoredTokens["account"] | null {
  return loadTokens()?.account ?? null;
}

export function isSignedIn(): boolean {
  return !!loadTokens();
}

```

```
// -----
// Graph file I/O – one JSON file in the app folder
// -----

const FILE_PATH = `${GRAPH}/me/drive/special/approot:${FILE_NAME}`;

/** Pull the workspace, or null when nothing has been synced yet. */
export async function downloadWorkspace(): Promise<RemoteWorkspace | null> {
  const token = await getAccessToken();
  if (!token) throw new Error("Not signed in to Microsoft.");

  const res = await fetch(`${FILE_PATH}/content`, {
    headers: { Authorization: `Bearer ${token}` },
  });
  if (res.status === 404) return null; // first ever sync
  if (!res.ok) throw new Error(`OneDrive download failed (${res.status}).`);

  const text = await res.text();
  const snapshot = parseWorkspace(text);
  return {
    snapshot,
    eTag: res.headers.get("etag") ?? undefined,
    lastModified: Date.parse(res.headers.get("last-modified") ?? "") || undefined,
  };
}

export async function uploadWorkspace(
  snapshot: WorkspaceSnapshot,
  deviceLabel?: string
): Promise<void> {
  const token = await getAccessToken();
  if (!token) throw new Error("Not signed in to Microsoft.");

  const res = await fetch(`${FILE_PATH}/content`, {
    method: "PUT",
    headers: {
      Authorization: `Bearer ${token}`,
      "Content-Type": "application/json",
    },
    body: serializeWorkspace(snapshot, deviceLabel),
  });
  if (!res.ok) throw new Error(`OneDrive upload failed (${res.status}).`);
}

=====
// === END FILE: src/lib/whiteboard/onedrive.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/palm.ts ===
// File 110 of 194 | Lines: 158 | Size: 6510 bytes
=====

/**
 * Palm rejection and deliberate touch
 * =====
 *
 * The problem this solves: on a tablet you rest the side of your hand on the
 * glass before and while you write. Every one of those contacts is a real
 * `pointerdown` with `pointerType === "touch"`, indistinguishable from a
 * deliberate tap unless you look at context. Landing on an object, it used to
 * select it – popping the transform box open mid-sentence and, if the hand
 * shifted a millimetre, dragging the text away.
 *
 * How the note apps that get this right (OneNote, Samsung Notes, GoodNotes,
 * Notability) actually do it – none of them relies on a single signal:
 *
 * 1. **The stylus wins.** While a pen is in contact, touch is not input.
 * 2. **The window outstays the pen.** You lift the pen between words while
 *    your hand stays put, so touch stays suppressed for a beat AFTER the pen
 *    leaves the glass. Suppressing only during contact – which is what this
 *    app did – leaves the gap wide open, because the palm usually lands
 *    BEFORE the nib does.
 * 3. **Retroactively.** A hand that lands first is only revealed as a palm
 *    when the pen arrives a moment later, so whatever that contact started
 *    gets rolled back rather than merely stopped. (Handled by the caller –
 *    see `WhiteboardCanvas.handlePointerDown`.)
 */
```

```
* 4. **Contact geometry.** A palm is a much bigger patch than a fingertip.
*
* The pen-based rules cost pure-touch users nothing, because a device with no
* stylus never sets `lastPenAt` – the window can never open. That is what lets
* all of this be always-on instead of hidden behind a setting nobody finds.
*/

/**
* How long after the stylus leaves the glass touch stays suppressed.
*
* Long enough to cover the lift between words (roughly 300-500ms of travel),
* short enough that putting the pen down and immediately pinching to zoom does
* not feel broken. The same ballpark the tablet note apps use.
*/
export const PEN_SUPPRESSION_MS = 800;

/**
* Context-aware fast-bypass suppression window for multi-touch gestures (R1-DEF-03).
* When the stylus is lifted and two fingertip contacts arrive, multi-touch
* pinch-to-zoom and pan unfreeze after 150ms (or immediately if fingertip-sized),
* eliminating the 800ms deadlock while preserving resting palm protection.
*/
export const PEN_FAST_BYPASS_MS = 150;

/**
* Contact patch, in CSS pixels, at or above which a touch is a palm rather
* than a fingertip. A fingertip on a tablet reports roughly 15-30; the edge of
* a hand reports 45+, often as a long thin ellipse (e.g. 30x70), which is why
* this tests the LARGER dimension and not the smaller one.
*
* Deliberately generous. A false positive here means "my finger cannot select
* anything", which is worse than the bug being fixed, so the geometry test is
* a supporting signal – the pen window above is the reliable one.
*/
export const PALM_CONTACT_PX = 45;

/** Android's platform long-press. Matching it is why it feels native. */
export const LONG_PRESS_MS = 500;

/**
* Movement that turns a press into a drag. Below it the finger is holding
* still (fingers always wobble a little); above it the user is panning and
* clearly did not mean to grab anything.
*/
export const LONG_PRESS_SLOP_PX = 12;

export type TouchVerdict = "allow" | "pen-down" | "pen-recent" | "contact-size";

export interface ContactGeometry {
  /** PointerEvent.width – contact patch width in CSS px. */
  width?: number;
  /** PointerEvent.height – contact patch height in CSS px. */
  height?: number;
}

export interface PenProximity {
  /** Is the stylus touching the glass right now? */
  penDown: boolean;
  /** Date.now() of the last stylus contact; 0 when no pen has ever been seen. */
  lastPenAt: number;
  now: number;
}

/**
* Is this contact patch too big to be a fingertip?
*
* Browsers that cannot measure the patch report the Pointer Events spec
* default of 1x1. That is "unknown", not "tiny" – calling it a fingertip would
* be exactly as unfounded as calling it a palm – so we abstain and let the pen
* window decide.
*/
export function contactLooksLikePalm(c: ContactGeometry): boolean {
  const w = typeof c.width === "number" && Number.isFinite(c.width) ? c.width : 0;
  const h = typeof c.height === "number" && Number.isFinite(c.height) ? c.height : 0;
  if (w <= 1 && h <= 1) return false;
  return Math.max(w, h) >= PALM_CONTACT_PX;
}
```

```

/**
 * Should this touch be allowed to interact with content, and if not, why?
 * The reason is kept (rather than a bare boolean) so the canvas can log it and
 * so the tests can assert WHICH rule fired.
 */
export function classifyTouch(
  c: ContactGeometry,
  p: PenProximity,
  suppressionWindowMs: number = PEN_SUPPRESSION_MS
): TouchVerdict {
  if (p.penDown) return "pen-down";
  const sincePen = p.now - p.lastPenAt;
  if (p.lastPenAt > 0 && sincePen >= 0 && sincePen < suppressionWindowMs) return "pen-recent";

  if (contactLooksLikePalm(c)) return "contact-size";
  return "allow";
}

/** Convenience wrapper: true when the touch must not reach content. */
export function isRejectedTouch(c: ContactGeometry, p: PenProximity): boolean {
  return classifyTouch(c, p) !== "allow";
}

/** Has the finger moved far enough that this is a drag, not a press? */
export function exceedsLongPressSlop(dx: number, dy: number): boolean {
  return Math.hypot(dx, dy) > LONG_PRESS_SLOP_PX;
}

export interface TouchTapSelectInput {
  /** classifyTouch verdict for this contact. */
  verdict: TouchVerdict;
  /** The finger landed on an object that is already selected. */
  hitSelected: boolean;
  /** Shift is held (multi-select toggle gesture). */
  shiftKey: boolean;
  /** An inline text/sticky editor is open. */
  editing: boolean;
}

/**
 * Does a fingertip tap select at once?
 *
 * A clean fingertip tap on an unselected object selects it on contact – the
 * tap-to-select, press-drag-to-move contract every tablet note app uses,
 * with finger-drag still panning. Anything else keeps the old behaviour: a
 * palm-sized patch still needs the deliberate long press (a resting hand must
 * never pop the transform box open), shift keeps toggling, and mid-edit a
 * touch is a resting hand.
 */
export function shouldSelectOnTouchTap(i: TouchTapSelectInput): boolean {
  if (i.verdict !== "allow") return false;
  if (i.hitSelected) return false;
  if (i.shiftKey) return false;
  if (i.editing) return false;
  return true;
}

=====
// === END FILE: src/lib/whiteboard/palm.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/pdf.ts ===
// File 111 of 194 | Lines: 161 | Size: 6166 bytes
=====

/**
 * PDF rendering
 * =====
 * Opens an imported PDF and rasterises single pages on demand.
 *
 * pdf.js is loaded with `await import()` so it never enters the main bundle –
 * it is ~1 MB, and someone who never imports a PDF should not pay for it. Its
 * worker is served from the app's own root (see PDF_WORKER_PATH) rather than a
 * CDN, because the app has to work with no internet: that is the whole premise
 * of it being local-first, and every other app in this category is criticised
 * for needing a connection.

```

```

*/

import type { PDFDocumentProxy } from "pdfjs-dist";

/** Rasterising above this loses more to memory than it gains in sharpness. */
export const PDF_MAX_RENDER_SCALE = 3;

/**
 * Where the pdf.js worker is served from. It is copied into public/ by
 * `npm run sync:pdf-worker`, which the build scripts run, so it ends up at the
 * app root in every package.
 */
export const PDF_WORKER_PATH = "/pdf.worker.min.mjs";

export interface PdfPageSize {
  width: number;
  height: number;
}

type PdfJs = typeof import("pdfjs-dist");

let pdfjsPromise: Promise<PdfJs> | null = null;

async function loadPdfJs(): Promise<PdfJs> {
  if (!pdfjsPromise) {
    pdfjsPromise = (async () => {
      const pdfjs = await import("pdfjs-dist");
      // The worker is shipped in public/ and served from the app's own
      // root, NOT from a CDN and NOT via `new URL(..., import.meta.url)`.
      //
      // A CDN would break the offline promise outright. import.meta.url works
      // in the Next bundle but cannot be compiled to CommonJS, which is what
      // the node test suite uses – and the store imports this module, so the
      // whole suite failed to compile. A plain root-relative path works in
      // every target: the dev server, the static export, Electron's
      // `pronotes://app/` scheme, and the APK's asset root.
      pdfjs.GlobalWorkerOptions.workerSrc = `${PDF_WORKER_PATH}`;
      return pdfjs;
    })();
  }
  return pdfjsPromise;
}

/**
 * Opens a PDF from raw bytes.
 *
 * The buffer is copied first: pdf.js takes ownership of what it is given and
 * detaches it, which would leave the caller's copy – the one being written to
 * IndexedDB – as a zero-length buffer.
 */
export async function openPdf(bytes: ArrayBuffer): Promise<PDFDocumentProxy> {
  const pdfjs = await loadPdfJs();
  return pdfjs.getDocument({ data: bytes.slice(0) }).promise;
}

/** Open stored documents, so a 40 MB PDF is parsed once however many parts read it. */
const storedDocs = new Map<string, Promise<PDFDocumentProxy>>();

/**
 * Opens a PDF kept in the PDF store, once. The page pictures and the page text
 * both read through here - two separate caches would parse (and hold) every
 * document twice.
 */
export function openStoredPdf(docId: string): Promise<PDFDocumentProxy> {
  let d = storedDocs.get(docId);
  if (!d) {
    d = (async () => {
      const { getPdf } = await import("./pdfstore");
      const stored = await getPdf(docId);
      if (!stored) throw new Error("This PDF is no longer stored on this device.");
      return openPdf(stored.bytes);
    })();
    storedDocs.set(docId, d);
    // A failed open must not be cached forever, or a transient error would
    // leave the notebook permanently blank until a restart.
    d.catch(() => storedDocs.delete(docId));
  }
  return d;
}

```

```

}

/** The stored document if something already has it open, else undefined. */
export function peekStoredPdf(docId: string): Promise<PDFDocumentProxy> | undefined {
  return storedDocs.get(docId);
}

/** Page size in CSS pixels at 100%, respecting the page's own rotation. */
export async function pdfPageSize(doc: PDFDocumentProxy, pageNumber: number): Promise<PdfPageSize> {
  const page = await doc.getPage(pageNumber);
  const viewport = page.getViewport({ scale: 1 });
  return { width: Math.round(viewport.width), height: Math.round(viewport.height) };
}

/**
 * Renders one page to a PNG data URL.
 *
 * `targetWidth` is the width the page will be drawn at on screen; the raster
 * is made at `dpr` times that so it stays sharp on a high-DPI tablet without
 * rendering the whole document at maximum resolution.
 */
export async function renderPdfPage(
  doc: PDFDocumentProxy,
  pageNumber: number,
  targetWidth: number,
  dpr = 2
): Promise<string> {
  const page = await doc.getPage(pageNumber);
  const base = page.getViewport({ scale: 1 });
  const scale = Math.min(PDF_MAX_RENDER_SCALE, (targetWidth * dpr) / base.width);
  const viewport = page.getViewport({ scale });

  const canvas = document.createElement("canvas");
  canvas.width = Math.max(1, Math.floor(viewport.width));
  canvas.height = Math.max(1, Math.floor(viewport.height));
  const ctx = canvas.getContext("2d");
  if (!ctx) throw new Error("Could not get a 2D context to render the PDF page");

  // PDFs assume paper: without this, a page with no background paints as
  // transparent and the canvas shows through in dark mode.
  ctx.fillStyle = "#ffffff";
  ctx.fillRect(0, 0, canvas.width, canvas.height);

  await page.render({ canvas, canvasContext: ctx, viewport }).promise;
  return canvas.toDataURL("image/png");
}

/** Reads a File into an ArrayBuffer, rejecting anything that is not a PDF. */
export function readPdfFile(file: File): Promise<ArrayBuffer> {
  return new Promise((resolve, reject) => {
    const looksLikePdf =
      file.type === "application/pdf" || /\.pdf$/i.test(file.name);
    if (!looksLikePdf) {
      reject(new Error(`${file.name} is not a PDF.`));
      return;
    }
    const reader = new FileReader();
    reader.onload = () => resolve(reader.result as ArrayBuffer);
    reader.onerror = () => reject(new Error(`Could not read "${file.name}".`));
    reader.readAsArrayBuffer(file);
  });
}

/**
 * The page size a notebook should use for this PDF, clamped so a poster-sized
 * page cannot produce a sheet too large to work with.
 */
export function notebookSizeForPdf(size: PdfPageSize, maxWidth = 1400): PdfPageSize {
  if (size.width <= maxWidth) return size;
  const scale = maxWidth / size.width;
  return { width: Math.round(size.width * scale), height: Math.round(size.height * scale) };
}

=====
// === END FILE: src/lib/whiteboard/pdf.ts ===
=====

```

```

=====
// === START FILE: src/lib/whiteboard/pdfText.ts ===
// File 112 of 194 | Lines: 801 | Size: 29209 bytes
=====

/**
 * Page text: geometry, reading order, selection and search
 * =====
 * The text of a PDF / Word / PowerPoint page, as runs of text with boxes, and
 * everything the "Select text" tool and search need to do with it. Pure: no
 * DOM, no IndexedDB, no pdf.js - so it is unit-tested in node.
 *
 * Coordinates are fractions of the PAGE (0..1 of its width and height, top-left
 * origin), never pixels or points. The same record then lines up with the page
 * at every zoom, on any sheet size, letterboxed or not.
 *
 * Where the text comes from:
 * - a real PDF: pdf.js `getTextContent()` (`runsFromPdfItems`);
 * - a converted Word / PowerPoint file: captured while it was drawn, since the
 *   PDF it becomes holds only pictures (see officeImport.ts).
 *
 * Right-to-left text
 * -----
 * Each run's string is expected in LOGICAL (reading) order, which is what
 * pdf.js, the DOM and PowerPoint's text all give. Two ways a PDF breaks that
 * are repaired here:
 * - glyphs stored one per item, left to right: a Hebrew word then arrives
 *   last-letter-first. Runs of one line are re-ordered visually, with a
 *   right-to-left base when the line is mostly Hebrew/Arabic (`orderLine`).
 * - whole strings stored in visual order. Hebrew betrays this: final letters
 *   (■ ■ ■ ■) only ever END a word, so a page whose words keep STARTING with
 *   them is reversed, and its RTL runs are turned back (`looksVisualOrder`).
 */

import { PEN_TYPE_PRESETS, type StrokeElement } from "../types";

/** One piece of text on a page, in page fractions. `t` is in reading order. */
export interface TextRun {
  t: string;
  x: number;
  y: number;
  w: number;
  h: number;
}

/** A page's text. `w` / `h` are the page's own size, used only for its shape. */
export interface PageText {
  w: number;
  h: number;
  runs: TextRun[];
  /**
   * The runs are already in reading order - text captured from Word's layout
   * (document order) or PowerPoint's drawing (the order it was written in) -
   * so lines are never re-sorted by position. A PDF's order is only a hint.
   */
  ordered?: boolean;
}

export type Dir = "l" | "r" | "n";

export interface LaidRun extends TextRun {
  /** Offset of the run's first character in `PageLayout.text`. */
  start: number;
  end: number;
  line: number;
  dir: Dir;
}

export interface PageLayout {
  page: PageText;
  /** The whole page in reading order: runs joined by spaces and newlines. */
  text: string;
  runs: LaidRun[];
}

export interface Rect {
  x: number;
  y: number;
}

```

```

    w: number;
    h: number;
}

export interface LineRect extends Rect {
    line: number;
}

// -----
// Direction
// -----

const RTL_LETTER = /[\u0590-\u08FF\uFB1D-\uFDFF\uFE70-\uFEFF]/;
const RTL_LETTER_G = /[\u0590-\u08FF\uFB1D-\uFDFF\uFE70-\uFEFF]/g;
const LTR_LETTER_G = /[A-Za-zÀ-█-█-█]/g;

/** Whether a run reads right-to-left, left-to-right, or has no letters at all. */
export function dirOf(s: string): Dir {
    const r = (s.match(RTL_LETTER_G) ?? []).length;
    const l = (s.match(LTR_LETTER_G) ?? []).length;
    if (r === 0 && l === 0) return /\d/.test(s) ? "l" : "n";
    return r >= l ? "r" : "l";
}

export function hasRtlText(s: string): boolean {
    return RTL_LETTER.test(s);
}

/**
 * A string stored in visual order, turned into reading order: reversed, except
 * that numbers and Latin words inside it keep their left-to-right order.
 */
export function visualToLogical(s: string): string {
    // Brackets need no swapping: stored in visual order as their logical
    // characters, plain reversal already reads "(3)" (checked in the tests).
    const rev = Array.from(s).reverse();
    const out: string[] = [];
    let i = 0;
    while (i < rev.length) {
        if (/[A-Za-z0-9]/.test(rev[i])) {
            let j = i;
            // A number keeps its separators ("3.14", "1,000") with it.
            while (j < rev.length && (/[A-Za-z0-9]/.test(rev[j]) || (/[:,./]/.test(rev[j]) && /[A-Za-z0-9]/.test(rev
                [j + 1] ?? "")))) j++;
            out.push(...rev.slice(i, j).reverse());
            i = j;
        } else {
            out.push(rev[i]);
            i++;
        }
    }
    return out.join("");
}

const HEBREW_FINALS = /[\u05F0-\u05F9]/;

/**
 * True when a page's Hebrew reads backwards: more words START with a final
 * letter than end with one. Words of one letter say nothing and are skipped.
 */
export function looksVisualOrder(text: string): boolean {
    let starts = 0;
    let ends = 0;
    for (const word of text.match(/[\u05F0-\u05F9]{2,}/g) ?? []) {
        if (HEBREW_FINALS.test(word[0])) starts++;
        if (HEBREW_FINALS.test(word[word.length - 1])) ends++;
    }
    return starts >= 2 && starts > ends * 2;
}

// -----
// Reading order
// -----

function overlapY(a: Rect, b: Rect): number {
    const top = Math.max(a.y, b.y);
    const bottom = Math.min(a.y + a.h, b.y + b.h);
    return Math.max(0, bottom - top) / Math.max(1e-9, Math.min(a.h, b.h));
}

```

```

}

/** Horizontal gap between two boxes, 0 when they touch or overlap. */
function gapX(a: Rect, b: Rect): number {
  return Math.max(0, Math.max(a.x, b.x) - Math.min(a.x + a.w, b.x + b.w));
}

/**
 * Splits runs into lines WITHOUT re-sorting the page: a run joins the line
 * before it only if it sits on the same baseline band and close by. Sorting
 * the whole page by position would interleave the two columns of a two-column
 * paper; the document's own order already reads column by column.
 */
export function grouplines(runs: TextRun[], aspect = 1): TextRun[][] {
  const lines: TextRun[][] = [];
  let cur: TextRun[] = [];
  let box: Rect | null = null;
  for (const r of runs) {
    // Gaps are compared in the same units as heights, so a wide page and a
    // tall one judge "close by" alike.
    const near = box !== null && overlapY(box, r) >= 0.5 && gapX(box, r) * aspect <= 2.5 * Math.max(box.h, r.h);
    if (!near && cur.length) {
      lines.push(cur);
      cur = [];
      box = null;
    }
    cur.push(r);
    box = box
      ? {
          x: Math.min(box.x, r.x),
          y: Math.min(box.y, r.y),
          w: Math.max(box.x + box.w, r.x + r.w) - Math.min(box.x, r.x),
          h: Math.max(box.y + box.h, r.y + r.h) - Math.min(box.y, r.y),
        }
      : { x: r.x, y: r.y, w: r.w, h: r.h };
  }
  if (cur.length) lines.push(cur);
  return lines;
}

/**
 * One line's runs in reading order. Left to right by position; for a line
 * that is mostly right-to-left, right to left, with any stretch of Latin words
 * or numbers inside it turned back to left-to-right (the Unicode bidi rule L2,
 * applied to whole runs).
 */
export function orderLine(runs: TextRun[]): TextRun[] {
  if (runs.length < 2) return runs.slice();
  const dirs = runs.map((r) => dirOf(r.t));
  const idx = runs.map((_, i) => i).sort((a, b) => runs[a].x + runs[a].w / 2 - (runs[b].x + runs[b].w / 2));
  if (!lineIsRtl(runs)) {
    // A left-to-right line: a Hebrew/Arabic phrase inside it still reads
    // right to left ("The phrase ■■■■ ■■■■ means").
    return reverseStretches(idx, dirs, "r").map((i) => runs[i]);
  }
  idx.reverse();
  const out: number[] = [];
  let i = 0;
  while (i < idx.length) {
    if (dirs[idx[i]] === "l") {
      let j = i;
      while (j < idx.length && dirs[idx[j]] !== "r") j++;
      // Trailing neutrals belong to the right-to-left text that follows.
      let k = j;
      while (k > i && dirs[idx[k - 1]] === "n") k--;
      out.push(...idx.slice(i, k).reverse());
      i = k;
    } else {
      out.push(idx[i]);
      i++;
    }
  }
  return out.map((n) => runs[n]);
}

/**
 * The line's base direction. A clear majority decides; when neither script

```

```

* clearly dominates, the line starts where its first word is - the rightmost
* word of a Hebrew line, the leftmost of an English one - so
* "■■■■■■: Binary Search Tree ■■■■ ■■■" (more Latin letters) still reads
* right to left, from "■■■■■■".
*/
function lineIsRtl(runs: TextRun[]): boolean {
  const r = runs.reduce((n, x) => n + (x.t.match(RTL_LETTER_G) ?? []).length, 0);
  const l = runs.reduce((n, x) => n + (x.t.match(LTR_LETTER_G) ?? []).length, 0);
  if (r === 0) return false;
  if (l === 0) return true;
  if (r >= 3 * l) return true;
  if (l >= 3 * r) return false;
  let right: TextRun | null = null;
  for (const x of runs) {
    const d = dirOf(x.t);
    if (d === "n" || /^[\d\s.,;/%+\-()]*$/ .test(x.t)) continue;
    if (!right || x.x + x.w > right.x + right.w) right = x;
  }
  return right ? dirOf(right.t) === "r" : r > l;
}

/** Positions (in order) with each stretch of `dir` runs reversed; neutrals between them go with them. */
function reverseStretches(idx: number[], dirs: string[], dir: string): number[] {
  const out: number[] = [];
  let i = 0;
  while (i < idx.length) {
    if (dirs[idx[i]] === dir) {
      let j = i;
      let lastStrong = i;
      while (j < idx.length && dirs[idx[j]] !== (dir === "r" ? "l" : "r")) {
        if (dirs[idx[j]] === dir) lastStrong = j;
        j++;
      }
      out.push(...idx.slice(i, lastStrong + 1).reverse());
      i = lastStrong + 1;
    } else {
      out.push(idx[i]);
      i++;
    }
  }
  return out;
}

const EDGE_NEUTRALS = /^(^[\p{L}\p{N}]*)([\s\S]*?)(^[\p{L}\p{N}]*)$/u;

/**
 * In a right-to-left line, punctuation at the ends of a number or Latin word
 * belongs to the Hebrew around it, not to the number: "■■■■■■ 12." is drawn
 * ". 12 ■■■■■■", and pdf.js hands over ".12" as one left-to-right piece, which
 * copied as "■■■■■■ .12". The punctuation is split off (with its share of the
 * box) so the line order puts it where it is read.
 */
function splitEdgeNeutrals(run: TextRun): TextRun[] {
  const m = EDGE_NEUTRALS.exec(run.t);
  if (!m || !m[2] || (!m[1] && !m[3])) return [run];
  const edges = charEdges(run.t);
  const piece = (from: number, to: number): TextRun => ({
    t: run.t.slice(from, to),
    x: run.x + run.w * edges[from],
    y: run.y,
    w: run.w * (edges[to] - edges[from]),
    h: run.h,
  });
  const a = m[1].length;
  const b = a + m[2].length;
  return [piece(0, a), piece(a, b), piece(b, run.t.length)].filter((r) => r.t.trim() !== "");
}

/** Arabic presentation forms (what many PDFs store) -> plain letters. */
const PRESENTATION_FORM = /[\uFB50-\uFDFF\uFE70-\uFEFC]/;
const PRESENTATION_FORMS = /[\uFB50-\uFDFF\uFE70-\uFEFC]+/g;

/** Lays a page out for reading: line order, separators, offsets. */
export function layoutPage(page: PageText): PageLayout {
  const aspect = page.h > 0 ? page.w / page.h : 1;
  const clean = page.runs
    .filter((r) => r.t.trim() !== "" && r.w > 0 && r.h > 0)
    .map((r) => (PRESENTATION_FORM.test(r.t) ? { ...r, t: r.t.replace(PRESENTATION_FORMS, (s) => s.normalize("

```

```

        NFKC")) } : r));
let lines = groupLines(clean, aspect);
if (!page.ordered) {
  lines = lines.map((line) =>
    orderLine(lineIsRtl(line) ? line.flatMap((r) => (dirOf(r.t) === "l" ? splitEdgeNeutrals(r) : [r])) : line)
  );
  // Whole strings stored backwards: turn every right-to-left run around, once.
  const probe = lines.map((l) => l.map((r) => r.t).join(" ").join("\n"));
  if (looksVisualOrder(probe)) {
    lines = lines.map((l) => orderLine(l.map((r) => (dirOf(r.t) === "r" ? { ...r, t: visualToLogical(r.t) } : r))));
  }
}

const out: LaidRun[] = [];
let text = "";
lines.forEach((line, li) => {
  if (li > 0) text += "\n";
  line.forEach((r, ri) => {
    if (ri > 0) {
      const prev = line[ri - 1];
      const gap = gapX(prev, r) * aspect;
      const joined = /\s$/.test(prev.t) || /\^s$/.test(r.t);
      if (!joined && gap > 0.15 * Math.min(prev.h, r.h)) text += " ";
    }
    const start = text.length;
    text += r.t;
    out.push({ ...r, start, end: text.length, line: li, dir: dirOf(r.t) });
  });
});
return { page, text, runs: out };
}

// -----
// Character geometry
// -----

/**
 * Relative advance of each character of a string. The browser plugs in a
 * canvas measurer; without one (tests, or no canvas) every character counts
 * the same, which is still right to within a letter.
 */
export type CharMeasurer = (s: string) => number[];

let measurer: CharMeasurer | null = null;
const edgeCache = new Map<string, number[]>();

export function setCharMeasurer(m: CharMeasurer | null) {
  measurer = m;
  edgeCache.clear();
}

/**
 * Edges of each character from the run's reading START, as fractions of the
 * run's width: n + 1 numbers from 0 to 1.
 */
export function charEdges(s: string): number[] {
  const hit = edgeCache.get(s);
  if (hit) return hit;
  const n = s.length;
  let widths = measurer ? measurer(s) : [];
  if (widths.length !== n || widths.some((w) => !(w >= 0))) widths = new Array(n).fill(1);
  const total = widths.reduce((a, b) => a + b, 0) || n || 1;
  const edges = [0];
  let acc = 0;
  for (const w of widths) {
    acc += w;
    edges.push(acc / total);
  }
  if (edgeCache.size > 4000) edgeCache.clear();
  edgeCache.set(s, edges);
  return edges;
}

/** x of the boundary before character `i` of a run (0..length). */
function boundaryX(run: LaidRun, i: number): number {
  const f = charEdges(run.t)[Math.max(0, Math.min(run.t.length, i))];

```

```

    return run.dir === "r" ? run.x + run.w * (1 - f) : run.x + run.w * f;
}

/** Page-fraction box of characters [a, b) of a run. */
function runSlice(run: LaidRun, a: number, b: number): Rect {
    const x0 = boundaryX(run, a);
    const x1 = boundaryX(run, b);
    return { x: Math.min(x0, x1), y: run.y, w: Math.abs(x1 - x0), h: run.h };
}

// -----
// Hit testing and selection
// -----

/**
 * The text offset nearest a point (page fractions). A point beside a line
 * lands at that line's nearer end; a point in the margin lands on the closest
 * line - so a drag that overshoots the text still selects up to it.
 */
export function offsetAt(layout: PageLayout, px: number, py: number): number | null {
    const { runs, page } = layout;
    if (!runs.length) return null;
    const aspect = page.h > 0 ? page.w / page.h : 1;
    let best: LaidRun | null = null;
    let bestD = Infinity;
    for (const r of runs) {
        const dx = Math.max(0, r.x - px, px - (r.x + r.w)) * aspect;
        const dy = Math.max(0, r.y - py, py - (r.y + r.h));
        // Being on the right line matters more than being near the right word.
        const d = dx + dy * 4;
        if (d < bestD) {
            bestD = d;
            best = r;
        }
    }
    if (!best) return null;
    const edges = charEdges(best.t);
    // Where along the run, measured from its reading start.
    const f = best.dir === "r" ? (best.x + best.w - px) / best.w : (px - best.x) / best.w;
    let i = 0;
    while (i < edges.length - 1 && f > (edges[i] + edges[i + 1]) / 2) i++;
    return best.start + i;
}

const WORD_CHAR = /[p{L}\p{N}\p{M}]_'\p{L}\p{N}\p{M}]/u;

/** The word around an offset, [start, end). Empty when there is none. */
export function wordRangeAt(text: string, offset: number): [number, number] {
    let s = Math.max(0, Math.min(text.length, offset));
    let e = s;
    if (!WORD_CHAR.test(text[s] ?? "") && WORD_CHAR.test(text[s - 1] ?? "")) s = e = s - 1;
    if (!WORD_CHAR.test(text[s] ?? "")) return [offset, offset];
    while (s > 0 && WORD_CHAR.test(text[s - 1])) s--;
    while (e < text.length && WORD_CHAR.test(text[e])) e++;
    return [s, e];
}

/** Boxes covering text [start, end), one per run touched. */
export function rangeRects(layout: PageLayout, start: number, end: number): LineRect[] {
    const out: LineRect[] = [];
    if (end <= start) return out;
    for (const r of layout.runs) {
        if (r.end <= start || r.start >= end) continue;
        const a = Math.max(start, r.start) - r.start;
        const b = Math.min(end, r.end) - r.start;
        out.push({ ...runSlice(r, a, b), line: r.line });
    }
    return out;
}

/**
 * Merges the boxes of each line into the stretches a pen would draw: pieces of
 * one line closer than about a letter become one. Mixed-direction lines may
 * select two separate stretches, and they stay separate.
 */
export function lineSegments(rects: LineRect[], aspect = 1): LineRect[] {
    const byLine = new Map<number, LineRect[]>();
    for (const r of rects) {

```

```

    const list = byLine.get(r.line) ?? [];
    list.push(r);
    byLine.set(r.line, list);
  }
  const out: LineRect[] = [];
  for (const [line, list] of byLine) {
    list.sort((a, b) => a.x - b.x);
    let cur = { ...list[0] };
    for (const r of list.slice(1)) {
      if ((r.x - (cur.x + cur.w)) * aspect <= 0.6 * Math.max(cur.h, r.h)) {
        const x1 = Math.max(cur.x + cur.w, r.x + r.w);
        const y0 = Math.min(cur.y, r.y);
        const y1 = Math.max(cur.y + cur.h, r.y + r.h);
        cur = { x: cur.x, y: y0, w: x1 - cur.x, h: y1 - y0, line };
      } else {
        out.push(cur);
        cur = { ...r };
      }
    }
    out.push(cur);
  }
  return out;
}

/**
 * Where the caret for a selection edge is drawn: the boundary before `offset`
 * (`side` "start") or after `offset - 1` ("end"), with its line's box.
 */
export function caretAt(layout: PageLayout, offset: number, side: "start" | "end"): { x: number; y: number; h:
  number } | null {
  const runs = layout.runs;
  if (!runs.length) return null;
  let run: LaidRun | undefined;
  if (side === "start") run = runs.find((r) => offset < r.end) ?? runs[runs.length - 1];
  else run = [...runs].reverse().find((r) => offset > r.start) ?? runs[0];
  const i = Math.max(0, Math.min(run.t.length, offset - run.start));
  return { x: boundaryX(run, i), y: run.y, h: run.h };
}

/** The selected text, as it should land on the clipboard. */
export function selectedText(layout: PageLayout, start: number, end: number): string {
  return layout.text
    .slice(Math.max(0, start), Math.max(0, end))
    .replace(/\u00AD\u200B-\u200F\u202A-\u202E\u2066-\u2069/g, "")
    .replace(/\t+\n/g, "\n")
    .trim();
}

// -----
// Search
// -----

/** Invisible or decorative characters that must never stop a match. */
const IGNOREABLE = /\u00AD\u0640\u200B-\u200F\u202A-\u202E\u2066-\u2069\uFEFF/;
const MARK = /\p{M}/u;

/**
 * Letters Arabic text is spelled with interchangeably - and that PDF fonts map
 * to their Persian look-alikes (a PDF's "■" often comes out as "■").
 */
const LETTER_FOLD: Record<string, string> = {
  "\u06CC": "\u064A",
  "\u0649": "\u064A",
  "\u06BE": "\u0647",
  "\u06A9": "\u0643",
};

/**
 * Folds text for matching - lower case, no accents, no Hebrew niqqud or Arabic
 * harakat, Arabic presentation forms and ligatures to plain letters, every run
 * of whitespace to one space - and keeps, for each folded character, the index
 * of the original character it came from.
 */
export function foldForSearch(s: string): { folded: string; map: number[] } {
  let folded = "";
  const map: number[] = [];
  for (let i = 0; i < s.length; i++) {
    const ch = s[i];

```

```

    if (IGNORABLE.test(ch)) continue;
    if (/\\s/.test(ch)) {
      if (folded.length > 0 && folded[folded.length - 1] !== " ") {
        folded += " ";
        map.push(i);
      }
      continue;
    }
    for (const c of ch.normalize("NFKD").toLowerCase()) {
      if (MARK.test(c) || IGNORABLE.test(c)) continue;
      folded += LETTER_FOLD[c] ?? c;
      map.push(i);
    }
  }
  return { folded, map };
}

export function foldQuery(q: string): string {
  return foldForSearch(q).folded.trim();
}

/**
 * The words of a search, as the notebook search reads them: quotes dropped,
 * split on spaces, each word once. A page matches when it has every word, in
 * any order - the same rule as for text boxes and notes.
 */
export function queryTerms(query: string): string[] {
  const out: string[] = [];
  for (const t of query.replace(/["'"]/g, " ").split(/\\s+/)) {
    if (t && foldQuery(t) && !out.some(o => foldQuery(o) === foldQuery(t))) out.push(t);
  }
  return out;
}

/** Every match of `query` in `text`, as [start, end) offsets into `text`. */
export function findAll(text: string, query: string, max = Infinity): Array<[number, number]> {
  const q = foldQuery(query);
  if (!q) return [];
  const { folded, map } = foldForSearch(text);
  const out: Array<[number, number]> = [];
  let from = 0;
  while (out.length < max) {
    const at = folded.indexOf(q, from);
    if (at < 0) break;
    const start = map[at];
    const end = map[at + q.length - 1] + 1;
    out.push([start, end]);
    from = at + Math.max(1, q.length);
  }
  return out;
}

/**
 * A readable one-line excerpt around a match, about `size` characters, cut at
 * word boundaries where it can be, with "..." where text was left out.
 */
export function snippetAround(
  text: string,
  start: number,
  end: number,
  size = 80
): { snippet: string; matchStart: number; matchLength: number } {
  const matchLen = end - start;
  const room = Math.max(0, size - matchLen);
  let a = Math.max(0, start - Math.floor(room / 2));
  let b = Math.min(text.length, end + (room - (start - a)));
  a = Math.max(0, Math.min(a, b - size));
  // Pull each cut back to a space, but never by more than a word's worth.
  if (a > 0) {
    const sp = text.indexOf(" ", a);
    if (sp >= 0 && sp < start && sp - a < 15) a = sp + 1;
  }
  if (b < text.length) {
    const sp = text.lastIndexOf(" ", b);
    if (sp >= end && b - sp < 15) b = sp;
  }
  const flat = (s: string) => s.replace(/\\s/g, " ");
  const lead = a > 0 ? "..." : "";

```

```

    const snippet = lead + flat(text.slice(a, b)) + (b < text.length ? "..." : "");
    return { snippet, matchStart: lead.length + (start - a), matchLength: matchLen };
}

// -----
// Sources
// -----

/** The subset of a pdf.js text item this module reads. */
export interface PdfTextItemLike {
    str: string;
    transform: number[];
    width: number;
    fontName?: string;
}

export interface PdfTextStyleLike {
    ascent?: number;
    descent?: number;
    vertical?: boolean;
}

function mul(m1: number[], m2: number[]): number[] {
    return [
        m1[0] * m2[0] + m1[2] * m2[1],
        m1[1] * m2[0] + m1[3] * m2[1],
        m1[0] * m2[2] + m1[2] * m2[3],
        m1[1] * m2[2] + m1[3] * m2[3],
        m1[0] * m2[4] + m1[2] * m2[5] + m1[4],
        m1[1] * m2[4] + m1[3] * m2[5] + m1[5],
    ];
}

/**
 * pdf.js text items -> runs. `viewport` is the page's viewport transform at
 * scale 1 (so the page's own /Rotate is honoured) and its size.
 *
 * An item's box is its baseline start, the width pdf.js measured, and the
 * font's ascent / descent - the same geometry pdf.js's own text layer uses.
 */
export function runsFromPdfItems(
    items: PdfTextItemLike[],
    styles: Record<string, PdfTextStyleLike | undefined>,
    viewport: { transform: number[]; width: number; height: number }
): TextRun[] {
    const out: TextRun[] = [];
    const vt = viewport.transform;
    const scale = Math.hypot(vt[0], vt[1]) || 1;
    for (const it of items) {
        if (typeof it.str !== "string" || !it.str.trim() || !Array.isArray(it.transform)) continue;
        const style = (it.fontName && styles[it.fontName]) || {};
        if (style.vertical) continue;
        const tx = mul(vt, it.transform);
        const fontH = Math.hypot(tx[2], tx[3]);
        if (!(fontH > 0)) continue;
        const ascent = style.ascent && style.ascent > 0 ? style.ascent : style.descent ? 1 + style.descent : 0.8;
        const descent = style.descent !== undefined && style.descent < 0 ? style.descent : ascent - 1;
        const angle = Math.atan2(tx[1], tx[0]);
        const ux = Math.cos(angle);
        const uy = Math.sin(angle);
        // "Up" on screen for this text direction (y grows downwards).
        const upx = uy;
        const upy = -ux;
        const w = it.width * scale;
        const up = ascent * fontH;
        const down = -descent * fontH;
        const bx = tx[4];
        const by = tx[5];
        const xs = [bx + upx * up, bx + ux * w + upx * up, bx - upx * down, bx + ux * w - upx * down];
        const ys = [by + upy * up, by + uy * w + upy * up, by - upy * down, by + uy * w - upy * down];
        const x0 = Math.min(...xs);
        const x1 = Math.max(...xs);
        const y0 = Math.min(...ys);
        const y1 = Math.max(...ys);
        if (!(x1 > x0) || !(y1 > y0)) continue;
        out.push({
            t: it.str,
            x: x0 / viewport.width,

```

```

        y: y0 / viewport.height,
        w: (x1 - x0) / viewport.width,
        h: (y1 - y0) / viewport.height,
    });
}
return out;
}

/**
 * Text drawn a piece at a time (PowerPoint slides are drawn with one canvas
 * call per line, word, or even letter) -> one run per stretch of a line.
 * Pieces join when they share a line and touch; repeats of the same piece in
 * the same place (a slide drawn twice, fill + outline) are dropped.
 */
export function mergeFragments(frag: TextRun[], aspect = 1): TextRun[] {
    const seen = new Set<string>();
    const out: TextRun[] = [];
    for (const f of frags) {
        if (!f.t || !(f.w > 0) || !(f.h > 0)) continue;
        const key = `${f.t}|${f.x.toFixed(4)}|${f.y.toFixed(4)}`;
        if (seen.has(key)) continue;
        seen.add(key);
        const last = out[out.length - 1];
        if (last && !/s/.test(f.t) && f.t.length <= 2 && dirOf(f.t) === dirOf(last.t) && overlapY(last, f) >= 0.7) {
            // Letter-by-letter drawing: glue the letter on when it starts where the
            // previous piece ended.
            // Touching, not a space apart: a space is ~0.25em, and gluing across
            // one turned "is a test of it" into "isa testofit". Right-to-left
            // letters are drawn leftwards, so their gap is measured the other way.
            const gap =
                (dirOf(f.t) === "r" ? last.x - (f.x + f.w) : f.x - (last.x + last.w)) * aspect;
            if (gap >= -0.3 * f.h && gap <= 0.06 * f.h) {
                const x0 = Math.min(last.x, f.x);
                const x1 = Math.max(last.x + last.w, f.x + f.w);
                const y0 = Math.min(last.y, f.y);
                const y1 = Math.max(last.y + last.h, f.y + f.h);
                out[out.length - 1] = { t: last.t + f.t, x: x0, y: y0, w: x1 - x0, h: y1 - y0 };
                continue;
            }
        }
        out.push({ ...f });
    }
    return out;
}

// -----
// Sheet mapping
// -----

// -----
// Marking up: highlight, underline, strike
// -----

export type MarkKind = "highlight" | "underline" | "strike";

/**
 * Strokes that mark selected lines, from line boxes already in sheet (world)
 * units: a highlighter stroke as tall as the line, or a fineliner rule under
 * the baseline or through the lower-case letters. Ordinary strokes on purpose -
 * saved, undoable, erasable and exported like ink you drew yourself.
 */
export function markStrokes(
    kind: MarkKind,
    segs: Rect[],
    color: { fill: string; ink: string },
    makeId: () => string,
    now = Date.now()
): StrokeElement[] {
    return segs
        .filter((s) => s.w > 0.5 && s.h > 0.5)
        .map((s) => {
            const style = kind === "highlight" ? "highlighter" : "fineliner";
            // A stroke renders at strokeWidth x its pen's size scale.
            const scale = PEN_TYPE_PRESETS[style].sizeScale ?? 1;
            const width = kind === "highlight" ? s.h : Math.max(1, s.h * 0.07);
            // Font boxes run ascent + descent with the baseline about 80% down, so
            // an underline sits just under it and a strike through the x-height.

```

```

const y = kind === "highlight" ? s.y + s.h / 2 : kind === "underline" ? s.y + s.h * 0.88 : s.y + s.h * 0
    .56;
const steps = 8;
const points: [number, number, number][] = Array.from({ length: steps + 1 }, (_, i) => [(s.w * i) / step
    s, 0, 0.5]);
return {
    id: makeId(),
    type: "stroke",
    x: s.x,
    y,
    rotation: 0,
    opacity: 1,
    points,
    color: kind === "highlight" ? color.fill : color.ink,
    strokeWidth: width / scale,
    strokeStyle: style,
    smoothing: 50,
    createdAt: now,
    updatedAt: now,
} satisfies StrokeElement;
});
}

```

```

/**
 * Where the page sits on its sheet. Pages are drawn "contain" and centred
 * (PdfPageLayer's preserveAspectRatio="xMidYMid meet"), so a page inserted
 * into a notebook of another shape is letterboxed; text must follow exactly.
 */
export function pageBoxOnSheet(page: { w: number; h: number }, sheetW: number, sheetH: number): Rect {
    if (!(page.w > 0) || !(page.h > 0)) return { x: 0, y: 0, w: sheetW, h: sheetH };
    const s = Math.min(sheetW / page.w, sheetH / page.h);
    const w = page.w * s;
    const h = page.h * s;
    return { x: (sheetW - w) / 2, y: (sheetH - h) / 2, w, h };
}

```

```

=====
// === END FILE: src/lib/whiteboard/pdfText.ts ===
=====

```

```

=====
// === START FILE: src/lib/whiteboard/pdfTextIndex.ts ===
// File 113 of 194 | Lines: 246 | Size: 9847 bytes
=====

```

```

/**
 * The text of a notebook's PDF / Word / PowerPoint pages, on demand
 * =====
 * One place that answers "what text is on this page, and where": for the
 * Select-text tool, for the find highlight, and for the notebook search.
 *
 * Lazy by design. A 90-page lecture PDF is never read in one go when you open
 * it: a page's text is extracted the first time something asks for that page
 * (you pick up the text tool on it, or a search walks past it), then kept in
 * memory and in IndexedDB (pdfTextStore.ts), so the next search is instant and
 * survives a restart.
 */

```

```

import type { PDFDocumentProxy } from "pdfjs-dist";
import type { Board, PdfPageRef } from "../types";
import { getPages } from "../notebook";
import {
    findAll,
    layoutPage,
    queryTerms,
    runsFromPdfItems,
    snippetAround,
    type PageLayout,
    type PageText,
    type PdfTextItemLike,
    type PdfTextStyleLike,
} from "../pdfText";

```

```

export interface PdfTextHit {
    pageIndex: number; // 0-based page index within the notebook
    pageId?: string; // the notebook page id, if known
    snippet: string; // ~80 chars around the match, readable order
}

```

```

    matchStart: number;    // index of the match inside snippet
    matchLength: number;
    /** Which match on its page this is (0 = the first), so find can go to it. */
    ordinal?: number;
}

interface DocState {
    source: "office" | "pdf";
    pages: Array<PageText | null>;
    layouts: Map<number, PageLayout | null>;
    inflight: Map<number, Promise<PageText>>;
    saveTimer: ReturnType<typeof setTimeout> | null;
    /**
     * The stored record could not be read. Nothing is written back then: the
     * record may hold a Word/PowerPoint file's text, which exists nowhere else.
     */
    readFailed?: boolean;
}

const docs = new Map<string, Promise<DocState>>();

/** Reads the stored record once per document; a failed read means "none yet". */
function docState(docId: string): Promise<DocState> {
    let d = docs.get(docId);
    if (!d) {
        d = (async (): Promise<DocState> => {
            const fresh: DocState = { source: "pdf", pages: [], layouts: new Map(), inflight: new Map(), saveTimer:
                null };
            try {
                const { getDocText, PDF_TEXT_VERSION } = await import("./pdfTextStore");
                const rec = await getDocText(docId);
                if (rec && Array.isArray(rec.pages) && (rec.source === "office" || rec.version === PDF_TEXT_VERSION))
                {
                    return { ...fresh, source: rec.source, pages: rec.pages };
                }
            } catch {
                // No IndexedDB (or a broken record): extract again, in memory only,
                // and ask again next time instead of trusting this for the session.
                queueMicrotask(() => docs.delete(docId));
                return { ...fresh, readFailed: true };
            }
            return fresh;
        })();
        docs.set(docId, d);
    }
    return d;
}

/** Writes a document's extracted pages back, a moment after the last one. */
function scheduleSave(docId: string, st: DocState) {
    if (st.source !== "pdf" || st.readFailed) return;
    if (st.saveTimer) clearTimeout(st.saveTimer);
    st.saveTimer = setTimeout(async () => {
        st.saveTimer = null;
        try {
            const { putDocText, PDF_TEXT_VERSION } = await import("./pdfTextStore");
            await putDocText({ id: docId, source: "pdf", version: PDF_TEXT_VERSION, pages: st.pages });
        } catch {
            // A cache: failing to keep it only means reading the page again later.
        }
    }, 1500);
}

async function extractPdfPage(docId: string, pageNumber: number, opened?: PDFDocumentProxy): Promise<PageText>
{
    const doc = opened ?? (await (await import("./pdf")).openStoredPdf(docId));
    if (pageNumber < 1 || pageNumber > doc.numPages) return { w: 1, h: 1, runs: [] };
    const page = await doc.getPage(pageNumber);
    const viewport = page.getViewport({ scale: 1 });
    const content = await page.getTextContent();
    const items = content.items.filter((it): it is typeof it & PdfTextItemLike => "str" in it);
    const runs = runsFromPdfItems(items, content.styles as Record<string, PdfTextStyleLike>, {
        transform: viewport.transform,
        width: viewport.width,
        height: viewport.height,
    });
    return { w: viewport.width, h: viewport.height, runs };
}

```

```

/** The raw text of one page. Never rejects: a page it cannot read has none. */
export async function getPageText(ref: PdfPageRef, opened?: PDFDocumentProxy): Promise<PageText> {
  const st = await docState(ref.docId);
  const i = ref.pageNumber - 1;
  const have = st.pages[i];
  if (have) return have;
  // A converted Office file has ALL its text in its record; a page missing
  // from it has none, and pdf.js would find nothing in a picture anyway.
  if (st.source === "office") return { w: 1, h: 1, runs: [] };
  let p = st.inflight.get(i);
  if (!p) {
    p = extractPdfPage(ref.docId, ref.pageNumber, opened)
      .then((text) => {
        st.pages[i] = text;
        st.layouts.delete(i);
        scheduleSave(ref.docId, st);
        return text;
      })
    // A page that failed to read (memory, a race with a delete) has no text
    // THIS time only: nothing is kept, so the next ask tries again rather
    // than calling it a scan forever.
    .catch(() => ({ w: 1, h: 1, runs: [] }) as PageText);
    st.inflight.set(i, p);
    p.finally(() => st.inflight.delete(i));
  }
  return p;
}

/** A page's text laid out for reading and selection; null when it has none. */
export async function getPageLayout(ref: PdfPageRef, opened?: PDFDocumentProxy): Promise<PageLayout | null> {
  const text = await getPageText(ref, opened);
  const st = await docState(ref.docId);
  const i = ref.pageNumber - 1;
  if (st.layouts.has(i)) return st.layouts.get(i) ?? null;
  const layout = text.runs.length ? layoutPage(text) : null;
  st.layouts.set(i, layout);
  return layout;
}

/**
 * Whether a page-less-of-text document is a Word / PowerPoint file converted
 * before text was captured: those PDFs were written by jsPDF. Lets the tool
 * say "open the file again" instead of "this is a scan".
 */
export async function isImageOnlyOfficeImport(docId: string): Promise<boolean> {
  try {
    const st = await docState(docId);
    if (st.source === "office") return false;
    const { openStoredPdf } = await import("./pdf");
    const doc = await openStoredPdf(docId);
    const meta = await doc.getMetadata();
    const info = (meta?.info ?? {}) as { Producer?: string; Creator?: string };
    return /jspdf/i.test(`${info.Producer ?? ""} ${info.Creator ?? ""}`);
  } catch {
    return false;
  }
}

/**
 * Stores the captured text of a converted Word / PowerPoint file under the id
 * its PDF was given, and makes it the document's text at once.
 */
export async function saveOfficeText(docId: string, pages: PageText[]): Promise<void> {
  docs.set(docId, Promise.resolve({ source: "office", pages, layouts: new Map(), inflight: new Map(), saveTime
    r: null }));
  const { putDocText, PDF_TEXT_VERSION } = await import("./pdfTextStore");
  await putDocText({ id: docId, source: "office", version: PDF_TEXT_VERSION, pages });
}

/** Most matches listed for one page, so one busy page cannot hide the rest. */
const HITS_PER_PAGE = 5;

/**
 * A document for reading text in a search. One that is already open (shown on
 * screen) is shared; otherwise it is opened just for this search and closed
 * after it - searching "everywhere" must not leave every PDF in the
 * workspace open for the rest of the session.

```

```

*/
async function openForSearch(docId: string): Promise<{ doc: PDFDocumentProxy; close: () => void }> {
  const { peekStoredPdf, openPdf } = await import("./pdf");
  const shared = peekStoredPdf(docId);
  if (shared) return { doc: await shared, close: () => {} };
  const { getPdf } = await import("./pdfstore");
  const stored = await getPdf(docId);
  if (!stored) throw new Error("This PDF is no longer stored on this device.");
  const doc = await openPdf(stored.bytes);
  return { doc, close: () => void doc.loadingTask.destroy().catch(() => {}) };
}

/**
 * Text hits for `query` in a notebook's PDF/Office pages. Case- and
 * diacritic-insensitive; with several words, a page must contain all of them
 * (any order), and the hits are placed on the first word. `signal` stops a
 * search that is no longer wanted between pages.
 */
export async function searchNotebookPdfText(
  board: Board,
  query: string,
  limit = 100,
  options: { signal?: AbortSignal } = {}
): Promise<PdfTextHit[]> {
  const hits: PdfTextHit[] = [];
  const terms = queryTerms(query ?? "");
  if (terms.length === 0) return hits;
  const pages = getPages(board);
  const opened = new Map<string, { doc: PDFDocumentProxy; close: () => void }>();
  try {
    for (let i = 0; i < pages.length && hits.length < limit; i++) {
      if (options.signal?.aborted) break;
      const ref = pages[i].pdf;
      if (!ref?.docId) continue;
      // Only open the document when a page's text is not known yet.
      const st = await docState(ref.docId);
      let doc: PDFDocumentProxy | undefined;
      if (!st.pages[ref.pageNumber - 1] && st.source === "pdf") {
        let o = opened.get(ref.docId);
        if (!o) {
          try {
            o = await openForSearch(ref.docId);
          } catch {
            continue;
          }
        }
        opened.set(ref.docId, o);
      }
      doc = o.doc;
    }
    const layout = await getPageLayout(ref, doc);
    if (!layout) continue;
    if (terms.slice(1).some((t) => findAll(layout.text, t, 1).length === 0)) continue;
    const lead = findAll(layout.text, terms[0], Math.min(HITS_PER_PAGE, limit - hits.length));
    lead.forEach(([s, e], ordinal) => {
      hits.push({ pageIndex: i, pageId: pages[i].id, ordinal, ...snippetAround(layout.text, s, e) });
    });
  } finally {
    for (const o of opened.values()) o.close();
  }
  return hits;
}

=====
// === END FILE: src/lib/whiteboard/pdfTextIndex.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/pdfTextStore.ts ===
// File 114 of 194 | Lines: 135 | Size: 5039 bytes
=====

/**
 * Page text storage (IndexedDB)
 * =====
 * The text of imported documents, one record per stored PDF (same id), in a
 * database of its own next to `pronotes-pdfs`.

```

```

*
* Two kinds of record live here:
* - "office": the text of a Word / PowerPoint file, captured while it was
*   converted. The PDF that file became is pictures only, so this record is
*   the ONLY copy of its text - it is never thrown away while the PDF lives.
* - "pdf": a cache of what pdf.js extracted from a real PDF, filled a page at
*   a time as pages are looked at or searched. Losing it costs a re-read.
*
* Why a sidecar rather than invisible text inside the PDF: jsPDF's built-in
* fonts cannot encode Hebrew or Arabic, so embedding would mean shipping a
* Unicode font into every converted file (hundreds of KB each) and trusting
* pdf.js to read our own positioning back. A record of runs with boxes needs no
* font, keeps Hebrew exactly as the browser laid it out, and is read with no
* parsing at all.
*
* A separate database (not a second store in `pronotes-pdfs`) so that opening
* it can never trigger a version upgrade of the PDF store itself.
*/

import type { PageText } from "../pdfText";

const DB_NAME = "pronotes-pdfText";
const DB_VERSION = 1;
const STORE = "docs";

/**
 * Bumped when extraction changes, so cached "pdf" records are re-read. Never
 * affects "office" records: their source is gone and cannot be re-read.
 */
export const PDF_TEXT_VERSION = 1;

export interface StoredDocText {
  /** The id of the PDF in `pronotes-pdfs`. */
  id: string;
  source: "office" | "pdf";
  version: number;
  /** Indexed by page number - 1. `null` = not extracted yet ("pdf" only). */
  pages: Array<PageText | null>;
}

function openDb(): Promise<IDBDatabase> {
  return new Promise((resolve, reject) => {
    if (typeof indexedDB === "undefined") {
      reject(new Error("No IndexedDB"));
      return;
    }
    const req = indexedDB.open(DB_NAME, DB_VERSION);
    req.onupgradeneeded = () => {
      const db = req.result;
      if (!db.objectStoreNames.contains(STORE)) db.createObjectStore(STORE, { keyPath: "id" });
    };
    req.onsuccess = () => resolve(req.result);
    req.onerror = () => reject(req.error ?? new Error("Could not open the text store"));
  });
}

/**
 * One request in its own transaction. Settles when the TRANSACTION does, not
 * the request: a write whose commit then aborts (a full disk) must report
 * failure - for a Word file's text there is no second copy.
 */
function tx<T>(mode: IDBTransactionMode, run: (store: IDBObjectStore) => IDBRequest<T>): Promise<T> {
  return openDb().then(
    (db) =>
      new Promise<T>((resolve, reject) => {
        let result: T;
        const fail = (err: unknown) => {
          db.close();
          reject(err ?? new Error("Text store request failed"));
        };
        let t: IDBTransaction;
        try {
          t = db.transaction(STORE, mode);
        } catch (err) {
          fail(err);
          return;
        }
        const req = run(t.objectStore(STORE));

```

```

    req.onsuccess = () => {
      result = req.result;
    };
    t.oncomplete = () => {
      db.close();
      resolve(result);
    };
    t.onabort = () => fail(t.error ?? req.error);
    t.onerror = () => fail(t.error ?? req.error);
  })
);
}

export function getDocText(id: string): Promise<StoredDocText | undefined> {
  return tx<StoredDocText | undefined>("readonly", (s) => s.get(id));
}

export function putDocText(rec: StoredDocText): Promise<void> {
  return tx("readwrite", (s) => s.put(rec) as unknown as IDBRequest<void>).then(() => undefined);
}

export function deleteDocText(id: string): Promise<void> {
  return tx("readwrite", (s) => s.delete(id) as unknown as IDBRequest<void>).then(() => undefined);
}

/** Drops the text of every document that is no longer stored. */
export async function pruneDocTexts(keepIds: Set<string>): Promise<void> {
  const keys = await tx<IDBValidKey[]>("readonly", (s) => s.getAllKeys());
  for (const k of keys) if (typeof k === "string" && !keepIds.has(k)) await deleteDocText(k);
}

// -----
// Hand-over from the Office converter to the PDF import
// -----

/**
 * The converter returns a plain PDF `File`, and the PDF import that stores it
 * (and chooses its id) knows nothing about Office. The captured text rides
 * along keyed on that File object; the import takes it once it has an id.
 * A WeakMap, so a conversion that is never imported leaves nothing behind.
 */
const pendingOfficeText = new WeakMap<object, PageText[]>();

export function attachOfficeText(file: object, pages: PageText[]) {
  pendingOfficeText.set(file, pages);
}

export function takeOfficeText(file: object): PageText[] | undefined {
  const pages = pendingOfficeText.get(file);
  pendingOfficeText.delete(file);
  return pages;
}

=====
// === END FILE: src/lib/whiteboard/pdfTextStore.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/pdfstore.ts ===
// File 115 of 194 | Lines: 115 | Size: 4481 bytes
=====

/**
 * PDF byte storage (IndexedDB)
 * =====
 * Imported PDFs are kept here, NOT in the workspace that goes to localStorage.
 *
 * Why this exists at all: the obvious implementation is to render every page
 * to a PNG and drop those into the notebook as image elements. A 90-page
 * lecture PDF at A4 is roughly 200-500 KB per page once rasterised – 20-45 MB
 * – against a localStorage budget of about 5 MB. It would not fail politely
 * either: the quota error arrives on SAVE, so you would import a PDF, annotate
 * it for an hour, and then lose the lot.
 *
 * So the original bytes are stored once in IndexedDB (hundreds of MB
 * available), each notebook page holds only a `{ docId, pageNumber }`
 * reference, and pages are rasterised on demand for display. Your ink stays in

```

```

* the normal element list, completely separate from the PDF, so it survives
* even if the source document is later removed.
*/

const DB_NAME = "pronotes-pdfs";
const DB_VERSION = 1;
const STORE = "documents";

export interface StoredPdf {
  id: string;
  name: string;
  bytes: ArrayBuffer;
  pageCount: number;
  addedAt: number;
}

/** Metadata without the payload – cheap to list. */
export type PdfSummary = Omit<StoredPdf, "bytes">;

function openDb(): Promise<IDBDatabase> {
  return new Promise((resolve, reject) => {
    if (typeof indexedDB === "undefined") {
      reject(new Error("This browser has no IndexedDB, so PDFs cannot be stored."));
      return;
    }
    const req = indexedDB.open(DB_NAME, DB_VERSION);
    req.onupgradeneeded = () => {
      const db = req.result;
      if (!db.objectStoreNames.contains(STORE)) {
        db.createObjectStore(STORE, { keyPath: "id" });
      }
    };
    req.onsuccess = () => resolve(req.result);
    req.onerror = () => reject(req.error ?? new Error("Could not open the PDF store"));
  });
}

function tx<T>(mode: IDBTransactionMode, run: (store: IDBObjectStore) => IDBRequest<T>): Promise<T> {
  return openDb().then(
    (db) =>
      new Promise<T>((resolve, reject) => {
        const t = db.transaction(STORE, mode);
        const req = run(t.objectStore(STORE));
        req.onsuccess = () => resolve(req.result);
        req.onerror = () => reject(req.error ?? new Error("PDF store request failed"));
        t.oncomplete = () => db.close();
      })
  );
}

export function putPdf(doc: StoredPdf): Promise<void> {
  return tx("readwrite", (s) => s.put(doc) as unknown as IDBRequest<void>).then(() => undefined);
}

export function getPdf(id: string): Promise<StoredPdf | undefined> {
  return tx<StoredPdf | undefined>("readonly", (s) => s.get(id));
}

export function deletePdf(id: string): Promise<void> {
  return tx("readwrite", (s) => s.delete(id) as unknown as IDBRequest<void>).then(() => undefined);
}

/** Every stored PDF, without its bytes. */
export async function listPdfs(): Promise<PdfSummary[]> {
  const all = await tx<StoredPdf[]>("readonly", (s) => s.getAll() as IDBRequest<StoredPdf[]>);
  return all
    .map(({ id, name, pageCount, addedAt }) => ({ id, name, pageCount, addedAt }))
    .sort((a, b) => b.addedAt - a.addedAt);
}

/**
 * Removes PDFs nothing references any more.
 *
 * Deleting a notebook does not reach in here, so without this a discarded
 * import would occupy storage forever with nothing able to open it.
 */
export async function pruneUnusedPdfs(referencedIds: Set<string>): Promise<string[]> {
  const stored = await listPdfs();

```

```

const orphans = stored.filter((d) => !referencedIds.has(d.id)).map((d) => d.id);
for (const id of orphans) await deletePdf(id);
// A document's text goes with it - and only then. Keyed on what is still
// STORED, not on what is referenced, so text can never outlive its PDF nor
// be dropped while the PDF (and an Undo that restores it) still exists.
try {
  const { pruneDocTexts } = await import("./pdfTextStore");
  // Listed again NOW: a document imported while the deletes ran (a Word
  // file converting) must keep the text it just stored.
  const current = await listPdfs();
  await pruneDocTexts(new Set(current.map((d) => d.id)));
} catch {
  // Housekeeping only.
}
return orphans;
}

/** Total bytes held, for showing the user what their imports cost. */
export async function pdfStorageBytes(): Promise<number> {
  const all = await tx<StoredPdf[]>("readonly", (s) => s.getAll() as IDBRequest<StoredPdf[]>);
  return all.reduce((sum, d) => sum + (d.bytes?.byteLength ?? 0), 0);
}

=====
// === END FILE: src/lib/whiteboard/pdfstore.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/persistence.ts ===
// File 116 of 194 | Lines: 916 | Size: 34852 bytes
=====

/**
 * Whiteboard Persistence Layer
 * =====
 *
 * Saves the entire whiteboard state (folders, boards, elements) to localStorage
 * with a debounced autosave. Provides load/save/clear operations.
 *
 * Schema versioning: if the schema changes in a breaking way, bump SCHEMA_VERSION
 * and add a migration in `migrate()`.
 */

import { isAbandonedTextBox } from "../textbox";
import { DEFAULT_PAGE_SIZE, STROKE_TOOL_IDS, cleanToolbarItems } from "../types";
import {
  HEARTBEAT_KEY,
  STORAGE_PRESSURE_RATIO,
  createIdbWorkspaceBackend,
  decodeWorkspace,
  encodeWorkspace,
  measureStoragePressure,
  workspaceDbAvailable,
  type WorkspaceBackend,
} from "../workspaceDb";
import type {
  Board,
  BoardBackground,
  Folder,
  NotebookPage,
  PageSizeId,
  DockPosition,
  PageTemplate,
  PinnedBarPosition,
  PinnedPen,
  StyleState,
  ToolType,
  ToolbarItemId,
  TouchSettings,
  WhiteboardElement,
} from "../types";

const STORAGE_KEY = "z-whiteboard-state-v1";
const SCHEMA_VERSION = 1;

const VALID_BOARD_PATTERNS: string[] = ["dots", "grid", "lines", "plain"];
const VALID_PAGE_SIZES: string[] = ["a4", "a4-landscape", "letter", "letter-landscape", "square"];

```

```

const VALID_DOCKS: string[] = ["bottom", "top", "left", "right"];
const BOARD_PATTERN_MIN_SIZE = 6;
const BOARD_PATTERN_MAX_SIZE = 120;

/**
 * Tool ids accepted from a save. DERIVED from STROKE_TOOL_IDS rather than
 * hand-listed: the hand-written version still named only the pre-rework pens,
 * so a saved `tool` of "ballpoint" / "gel" / "fineliner" / "fountain" was
 * rejected and the pen silently reset to the default on every reload.
 */
const NON_STROKE_TOOLS: string[] = [
  "select", "lasso", "pan", "eraser", "rectangle", "ellipse", "diamond",
  "triangle", "star", "cloud", "line", "arrow", "text", "sticky", "image",
  "connect", "laser", "textselect",
];
const VALID_TOOLS: string[] = [...NON_STROKE_TOOLS, ...STROKE_TOOL_IDS];

/** Only keep a plain object; anything else would poison the merged defaults. */
function objectOrUndefined<T>(v: unknown): T | undefined {
  return v && typeof v === "object" && !Array.isArray(v) ? (v as T) : undefined;
}

function boolOrUndefined(v: unknown): boolean | undefined {
  return typeof v === "boolean" ? v : undefined;
}

/**
 * A pin is a pen, so this is exactly the stroke tools. Hand-listing it meant a
 * pinned ballpoint / gel / fineliner / fountain failed validation and was
 * DELETED from the favourites bar on the next load.
 */
const VALID_PIN_TOOLS: string[] = STROKE_TOOL_IDS;

export interface PersistedState {
  version: number;
  folders: Folder[];
  boards: Board[];
  activeBoardId: string | null;
  activeFolderId: string | null;
  customLinkTypes?: Array<{ id: string; name: string; color: string }>;
  pinnedPens?: PinnedPen[];
  pinnedBarPos?: PinnedBarPosition | null;
  toolbarPosition?: DockPosition;
  pageTemplates?: PageTemplate[];
  splitBoardId?: string | null;
  splitRatio?: number;
  splitSide?: "left" | "right";
  splitOrientation?: "side" | "stacked";
  recentColors?: string[];
  style?: Partial<StyleState>;
  tool?: ToolType;
  lastStrokeTool?: ToolType;
  touchSettings?: Partial<TouchSettings>;
  snapToGrid?: boolean;
  minimapOpen?: boolean;
  legendOpen?: boolean;
  zoomDockOpen?: boolean;
  shapeRecognition?: boolean;
  defaultPaper?: BoardBackground;
  pinnedBarOpen?: boolean;
  pinnedBarVertical?: boolean;
  propsPanelCollapsed?: boolean;
  toolPropsBarCollapsed?: boolean;
  /** The tool bar's buttons, in order. Absent = all of them, default order. */
  toolbarTools?: ToolbarItemId[];
  /** Buttons the user took off the bar, so tools added later can still appear. */
  toolbarHidden?: ToolbarItemId[];
}

export function loadState(): PersistedState | null {
  if (typeof window === "undefined") return null;
  try {
    const raw = window.localStorage.getItem(STORAGE_KEY);
    if (!raw) return null;
    const parsed = JSON.parse(raw) as PersistedState;
    // Always run migrate() - it's idempotent and ensures any missing fields
    // (e.g., tags/starred/accessedAt added after initial release) are backfilled.
    return migrate(parsed);
  }

```